The mathematics of YeeLab

Voter models, exact ranking probabilities, voting methods, and how Beta and normal voters differ

This document explains what YeeLab computes and why. Chapter 1 defines the model: the Yee diagram, the two voter distributions (normal and Beta) and the spread rules that fix the Beta distributions. Chapter 2 shows how the share of every ranking of the candidates is computed exactly for every pixel, and Chapter 3 how the voting methods turn these shares into winners. Chapter 4 lets a share of the voters vote strategically, by bullet voting, burial or compromise. Chapter 5 explains why Beta and normal voters give different diagrams. Chapter 6 compares the spread rules and explains why `rms` is the default. Chapter 7 asks when a candidate's region is convex or in one piece, and how that relates to the monotonicity of the method. Chapter 8 shows how the web UI draws the regions as curves fast enough to follow a dragged candidate. Chapter 9 describes the diagram in which every pixel is drawn at the geometric median of its voters. Chapter 10 defines how the web UI compares a method's diagram with the Voronoi diagram.

The settings that appear in the mathematics, with their defaults (file names are in the package `src/yeelab/`):

quantity	symbol	set by	default
medians per axis	n	`PIXELS` (`margin/shares.py`)	300
deviation	D	Model parameter, *Deviation* slider	0.2
voter distribution		Model parameter, *Voters* control	Beta
Beta spread rule		Model parameter; the web UI always uses `rms`	`rms`
what a pixel is (Beta, UI)		*Point in diagram is*, `UI_PIXEL_MEDIAN` (`web/app.py`)	geometric median
interpolation nodes per axis	N	`NODES`	49
Gauss–Legendre points per edge	Q	`QUAD_NODES`	24
score levels (UI)	L	*Score categories*, `SCORE_LEVELS` (`web/app.py`)	6
power of the score ballot (UI)	p	*Score power p*, `SCORE_POWER` (`build/methods.py`)	1.5
bullet, burying, compromising voters (UI)	b, q, y	*Bullet voters*, *Burying voters*, *Compromising voters* (`web/app.py`)	0, 0, 0
voter grid of the score shares	K	`GRID_CELLS`, `GRID_SUB` (`margin/shares.py`)	256, 8
contour grid per axis (UI)	G	`DRAG_GRID` / `FINAL_GRID` (`web/app.py`)	160 / 320

Contents

1. The model . . . 3
   - 1.1. Yee diagrams . . . 3
   - 1.2. Normal voters . . . 3
   - 1.3. Beta voters . . . 3
   - 1.4. The deviation and the centre pixel . . . 4
   - 1.5. Spread rules . . . 4
   - 1.6. Solving for the parameters (`beta_params_at`) . . . 5
2. Exact ranking probabilities . . . 7
   - 2.1. Ranking cells (`ranking_cells`) . . . 7

2.2. Cell probability as a boundary integral ... 8
2.3. Edge integrals (`TabulatedBeta.edge_terms`) ... 9
2.4. Interpolation in the median ... 10
2.5. Normal voters (`normal.py`) ... 10
2.6. Accuracy and tests ... 12
3. Voting methods (`yeelab.build`) ... 13
3.1. First past the post (`fptp`) ... 13
3.2. Instant runoff (`irv`) ... 13
3.3. Borda count (`borda`) ... 13
3.4. Pairwise majorities ... 13
3.5. Baldwin and Nanson (`baldwin`, `nanson`) ... 13
3.6. Minimax and Black (`minimax`, `black`) ... 14
3.7. Descending acquiescing coalitions (`dac`) ... 14
3.8. Score voting (`score`) ... 15
3.9. Score ballots on real ballots ... 20
3.10. Majority judgment (`mj`) ... 23
3.11. IRV set + KOTH + Score (`irv_set_koth_score`) ... 25
3.12. Ties ... 26
4. Strategic voting ... 27
4.1. Bullet voting (`Bullet`) ... 27
4.2. Burial (`Burial`) ... 30
4.3. Compromise (`Compromise`) ... 32
4.4. STAR's runoff ... 34
5. Beta versus normal voters ... 35
5.1. The two models ... 35
5.2. Pairwise majorities ... 36
5.3. Curved borders ... 37
5.4. Summary ... 40
6. Choosing the spread rule ... 41
6.1. The two rules on the default candidates ... 41
6.2. The round IRV edge of `mean_abs` ... 42
6.3. What keeps a border straight ... 42
6.4. Benchmark ... 43
6.5. Other ways to fix the spread ... 44
6.6. Conclusion ... 44
7. Shapes of win regions ... 45
7.1. Regions, shapes and monotonicity ... 45
7.2. Why monotonicity does not fix the shape ... 45
7.3. Where the regions are convex ... 45
7.4. Monotone methods with non-convex regions ... 46
7.5. Monotone methods with regions in pieces ... 48
7.6. Condorcet methods under Beta voters ... 48
7.7. Summary ... 49
8. Real-time diagrams in the web UI ... 50
8.1. What each method needs (`margin/shares.py`) ... 50
8.2. Dragging: an edge cache ... 51
8.3. Tabulated Beta CDF (`margin/beta_tables.py`) ... 51
8.4. Compiled kernels (numba) ... 52
8.5. Borders as zero sets (`build/`, `margin/regions.py`) ... 52
8.6. Timings ... 59
9. Pixels at the geometric median ... 60
9.1. The map g ... 60

9.2. The coloured region 61
9.3. The borders between the candidates 62
9.4. Computing g (`margin/geometric.py`) 63
10. Comparing a diagram with Voronoi 65
10.1. Assignment regret (`margin/metrics.py`) 65
10.2. Symmetric mean distance between boundaries 65

1. The model

1.1. Yee diagrams

The political space is the unit square $[0,1]^2$. There are C candidates at fixed points $c_1, ..., c_C \in [0,1]^2$. The square is divided into $n \times n$ pixels, and every pixel is its own election:

- the voters of the pixel are spread around the pixel centre $m = \big(\big(k+\frac{1}{2}\big)/n, \big(l+\frac{1}{2}\big)/n\big)$, $k, l = 0, ..., n-1$;
- every voter ranks the candidates by Euclidean distance, closest first;
- the pixel is coloured by the winner of a voting method.

A pixel does not have a finite number of voters. Its electorate is a probability distribution, and all that a voting method needs from it is, for every strict ranking r of the candidates,

$$P_{kl}(r) = \text{share of the voters of pixel } (k,l) \text{ whose ranking is } r. \tag{1}$$

Chapter 2 computes these shares exactly, without sampling voters, so the diagrams have no Monte Carlo noise. A voter can only be tied between two candidates on a line (Section 2.1), and lines have probability zero, so ties are ignored.

If every voter sat exactly at the pixel centre, every method would elect the candidate nearest to m, and the diagram would be the **Voronoi diagram** of the candidates (*Voronoi* in the web UI). It is the reference against which the other diagrams are compared.

1.2. Normal voters

In the original Yee diagrams the voters of a pixel are isotropic normal around the pixel centre,

$$(X, Y) \sim \mathcal{N}(m, \sigma^2 I), \tag{2}$$

so some of them lie outside the square. The spread is set by the **deviation** D, the mean absolute deviation of each coordinate from the pixel centre. For $X \sim \mathcal{N}(m_x, \sigma^2)$, $E|X - m_x| = \sigma\sqrt{2/\pi}$, so

$$\sigma = D\sqrt{\frac{\pi}{2}} \quad (D = 0.2 \Rightarrow \sigma = 0.251, \quad D = 0.3 \Rightarrow \sigma = 0.376) \tag{3}$$

(`normal.sigma_from_deviation`).

1.3. Beta voters

YeeLab's own model keeps every voter inside the square. The two coordinates are independent and Beta distributed,

$$X \sim \mathrm{Beta}(a_x, b_x), \quad Y \sim \mathrm{Beta}\big(a_y, b_y\big), \tag{4}$$

where (a_x, b_x) depend only on the pixel column and $\big(a_y, b_y\big)$ only on the pixel row. Both come from the same function of the median, so it is enough to study one coordinate $X \sim \mathrm{Beta}(a, b)$ with median $m \in (0,1)$. We write f, F for the pdf and CDF of X and g, G for those of Y. The CDF of $\mathrm{Beta}(a,b)$ is the regularised incomplete beta function $I_x(a,b)$ (`scipy.special.betainc`), its inverse is `betaincinv`.

A Beta distribution has two parameters, so two conditions fix it:

1. the **median** is the pixel centre,

$$I_m(a,b) = \frac{1}{2}; \tag{5}$$

2. a **spread rule** fixes how far the voters spread (Section 1.4, Section 1.5).

The median plays the role that the mean plays for normal voters: a line through m parallel to an axis has exactly half of the pixel's voters on each side. For oblique lines this is no longer true, and Chapter 5 shows that this is the root of most differences between the two models.

If $X \sim \mathrm{Beta}(a,b)$ then $1-X \sim \mathrm{Beta}(b,a)$. So the pixel with median $1-m$ has the parameters of median m swapped, and the rules below only need to be defined for $m \geq \frac{1}{2}$.

1.4. The deviation and the centre pixel

At the centre pixel, $m = \frac{1}{2}$, symmetry forces $a = b = a_0$, and every spread rule fixes a_0 by the same condition as the normal model, $E|X - \frac{1}{2}| = D$. For a symmetric Beta this has a closed form,

$$E|X - \frac{1}{2}| = \frac{2^{-2a_0}}{a_0 B(a_0, a_0)} = D, \tag{6}$$

which decreases from $\frac{1}{2}$ ($a_0 \to 0$) to 0 ($a_0 \to \infty$), so $0 < D < \frac{1}{2}$ is required. It is solved for a_0 by bracketing (`centre_shape`). The uniform distribution, $a_0 = 1$, has $D = \frac{1}{4}$ and separates two shapes:

- $D < \frac{1}{4}$ gives $a_0 > 1$: the voters of the centre pixel are bell-shaped, dense in the middle and thin at the walls. The default $D = 0.2$ gives $a_0 = 1.724$.
- $D > \frac{1}{4}$ gives $a_0 < 1$: the density is U-shaped and infinite at both walls, with more voters near the walls than in the middle. $D = 0.3$ gives $a_0 = 0.602$ ($\mathrm{Beta}(\frac{1}{2}, \frac{1}{2})$ has $D = 1/\pi \approx 0.318$).

Away from the centre, as the median approaches a wall, one of the parameters drops below 1 for every D (at $D = 0.2$ with `rms`, $b = 0.42$ at $m = 0.9$), and the density becomes infinite at that wall. Much of the numerics of Chapter 2 is shaped by this.

1.5. Spread rules

1.5.1. One free parameter

With the median fixed by Equation 5, one parameter is left. The convenient one is the **concentration** $\kappa = a + b$:

$$\mathrm{Var}\ X = \frac{\mu(1-\mu)}{\kappa + 1}, \quad \mu = EX = \frac{a}{\kappa}. \tag{7}$$

For given κ and m, Equation 5 determines a and b (Section 1.6). A spread rule is therefore a curve $\kappa(m)$ for $m \in [\frac{1}{2}, 1)$ that starts at $\kappa(\frac{1}{2}) = 2a_0$; a larger κ means voters closer together.

1.5.2. Why the spread must change near a wall

Near a wall, "the same spread for every pixel" cannot hold in every sense. When the median m is close to 1, the half of the voters above the median lies within $1-m$ of it and contributes almost nothing to any measure of spread; the whole spread has to come from the other half, the one towards the centre of the square. For the mean absolute deviation,

$$E|X-m| = \frac{1}{2}E[m - X \mid X < m] + \frac{1}{2}E[X - m \mid X > m] \leq \frac{1}{2}E[m - X \mid X < m] + \frac{1-m}{2}, \tag{8}$$

so keeping $E|X-m| = D$ forces

$$E[m - X \mid X < m] \geq 2D - (1-m). \tag{9}$$

At the centre this far half lies at mean distance D from the median; next to a wall it must lie twice as far. For the power mean $(E|X-m|^p)^{\frac{1}{p}}$ the same argument gives a factor $2^{\frac{1}{p}}$: 2 for $p = 1$, $\sqrt{2} \approx 1.41$ for the root mean square, 1.26 for $p = 3$. This holds for every distribution on $[0,1]$, not only for Beta. Every rule therefore decides, in effect, how far the far half of the voters is pushed out near a wall, and this is what the two rules differ in.

1.5.3. The two rules

`rms` (default). The root mean square distance of the voters from the median is the same as at the centre pixel,

$$\sqrt{E(X-m)^2} = s = \frac{1}{2\sqrt{2a_0+1}}, \quad E(X-m)^2 = \operatorname{Var} X + (\mu - m)^2, \tag{10}$$

where s is the standard deviation of $\mathrm{Beta}(a_0, a_0)$ ($s = 0.237$ for $D = 0.2$, 0.337 for $D = 0.3$). The square weights distant voters more, so near a wall a few voters far out make up the spread and the rest of the far half stays where it is. It is the rule with the clearest meaning, and of the rules compared in Section 6.4 its borders are among the straightest. The web UI uses it for all its Beta voters.

`mean_abs` (legacy). The mean absolute deviation is D for every median, $E|X - m| = D$. This was the first rule, and it is kept to reproduce older results. Because it keeps the mean absolute deviation fixed, Equation 9 applies in full: near a wall the far half of the voters is pushed twice as far out, towards the opposite wall. At median 0.98 the share of voters below 0.1 is back at 0.192, more than at the centre, where `rms` has 0.049 (Figure 1). This push bends Condorcet borders, adds cycles and gives IRV a round edge that `rms` does not have (Chapter 6). It is not offered in the web UI; it remains available as a model spread rule.

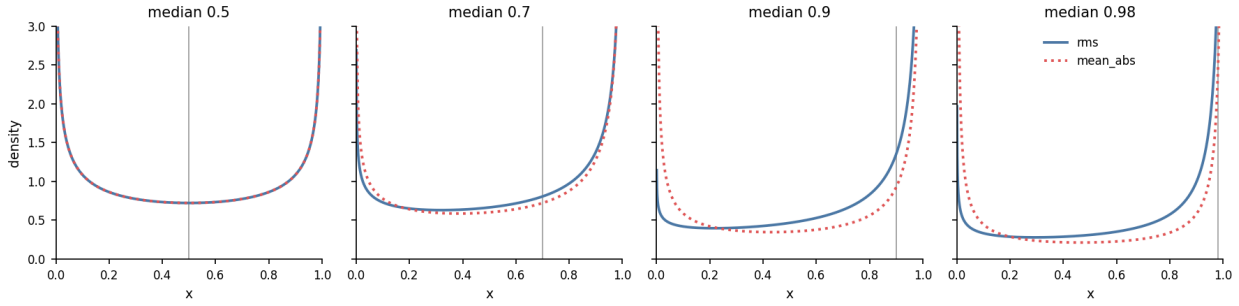


Figure 1: Density of one coordinate of a pixel's voters, $D = 0.3$, for medians moving towards the wall at 1 (grey: the median). Both rules agree at the centre. Near the wall `mean_abs` moves many voters to the opposite wall; `rms` moves far fewer.

At $D = 0.3$ (`docs/figures.py`):

median	rule	$\kappa = a + b$	mean	$E\lvert X - m\rvert$	$\sqrt{E(X-m)^2}$	$P(X < 0.1)$
0.50	all	1.204	0.500	0.300	0.337	0.175
0.70	`rms`	1.197	0.621	0.283	0.337	0.095
	`mean_abs`	1.012	0.612	0.300	0.355	0.121
0.90	`rms`	1.062	0.750	0.229	0.337	0.051
	`mean_abs`	0.554	0.685	0.300	0.430	0.146
0.98	`rms`	0.799	0.812	0.186	0.337	0.049
	`mean_abs`	0.307	0.698	0.300	0.490	0.192

1.6. Solving for the parameters (`beta_params_at`)

Only the distinct values of $\max(m, 1-m) \geq \frac{1}{2}$ are solved, in increasing order; medians below $\frac{1}{2}$ get the mirrored parameters.

The median for a given concentration (`_b_for_median`). For a fixed $\kappa = a + b$ the median of $\mathrm{Beta}(\kappa - b, b)$ decreases from 1 to $\frac{1}{2}$ as b grows from 0 to $\kappa/2$, so Equation 5 has exactly one root, found by bracketing b (Brent's method).

`rms` (`_solve_rms`). Along the Beta distributions with median m, the RMS distance Equation 10, which is explicit in (a, b), decreases as κ grows. An outer bracketing search over $\log \kappa$, starting from the solution at the previous median, finds the κ whose RMS distance is s.

`mean_abs` (`_solve_beta`). Split the absolute deviation at the median and use $F(m) = \frac{1}{2}$:

$$\begin{aligned} E|X - m| &= E[X - m] - 2E[(X - m)\mathbb{1}\{X < m\}] \\ &= \mu - m - 2E[X\mathbb{1}\{X < m\}] + 2mF(m) \\ &= \mu - 2E[X\mathbb{1}\{X < m\}]. \end{aligned} \tag{11}$$

Since $x \cdot x^{a-1}(1-x)^{b-1}/B(a,b) = \mu \cdot$ (density of $\mathrm{Beta}(a+1,b)$), $E[X\mathbb{1}\{X < m\}] = \mu I_m(a+1,b)$, and therefore

$$E|X - m| = \frac{a}{a+b}(1 - 2I_m(a+1,b)) = D. \tag{12}$$

Equation 5 and Equation 12 are solved together for (a,b) with Levenberg–Marquardt (`scipy.optimize.root(method="lm")`). A two-dimensional solver needs a good initial guess, so the medians are solved from the centre solution (a_0, a_0) outwards, each solution being the initial guess for the next median, and intermediate medians are inserted wherever two consecutive medians are more than 0.25 apart in $\mathrm{logit}(m)$ (`CONTINUATION_STEP`). This keeps the solver on the correct branch, also for a single median close to 0 or 1.

The tests check that every rule keeps its quantity fixed (to 10^{-9}, with the moments computed by independent quadrature), the median (to 10^{-12}), the mirror symmetry, and that a lone median gets the same parameters as a full sweep.

2. Exact ranking probabilities

This chapter describes how the margin pipeline computes the shares (Equation 1) for Beta voters (`ranking_cells.py` and `margin/beta_tables.py`, Section 2.1 to Section 2.4) and for normal voters (`normal.py`, Section 2.5). The only approximations are Gauss–Legendre quadrature and polynomial interpolation, both of which converge exponentially fast; the normal model needs no quadrature at all. For Beta voters the steps are:

1. Solve the Beta parameters at the interpolation nodes (Section 1.6, Section 2.4).
2. Cut the square by the $\binom{C}{2}$ bisectors into cells, each with one ranking (Section 2.1).
3. Write the probability of each cell as a sum of integrals over its edges; each edge is shared by two cells (Section 2.2).
4. Compute every edge integral for all pairs of node parameters (Section 2.3).
5. Interpolate from the $N \times N$ nodes to the $n \times n$ evaluated medians (Section 2.4).

2.1. Ranking cells (`ranking_cells`)

2.1.1. Bisectors

A voter at p prefers c_i to c_j exactly when

$$|p - c_i|^2 < |p - c_j|^2 \quad \Longleftrightarrow \quad 2\big(c_j - c_i\big) \cdot p < |c_j|^2 - |c_i|^2 \ . \tag{13}$$

The boundary of Equation 13 is the perpendicular bisector of $c_i c_j$, a straight line. A voter's full ranking is determined by the signs of all $\binom{C}{2}$ comparisons, so the ranking can only change when the voter crosses one of these bisectors.

The $\binom{C}{2}$ lines cut the unit square into convex polygons ("cells"). Inside a cell every comparison has a fixed sign, hence **every point of a cell has the same ranking**. Points on a bisector have a tie, but lines have zero area and so zero probability.

The crucial observation is that **the cells do not depend on the pixel**: they depend only on the candidates. Only the probability mass that each pixel's distribution puts on each cell changes. So

$$P_{kl}(r) = P_{kl}((X, Y) \in K_r) = \iint_{K_r} f_k(x) g_l(y) \, \mathrm{d}x \, \mathrm{d}y, \tag{14}$$

where K_r is the cell with ranking r.

2.1.2. Construction

The construction starts with the square as a single polygon and, for every pair (i, j), splits every current polygon into its parts on either side of the bisector (half-plane clipping, `_clip`, a Sutherland–Hodgman step for one line). Slivers with area below 10^{-14} are discarded. All polygons stay convex and counter-clockwise (CCW). The ranking of a cell is found by sorting the candidates by distance from the cell's vertex average, which lies strictly inside the convex cell.

Figure 2 shows the arrangement for the candidates A–E used throughout this document.

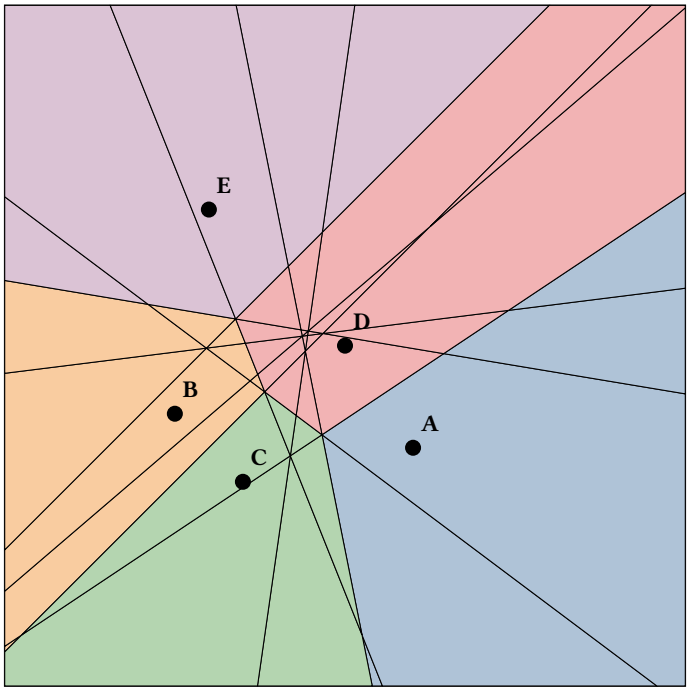


Figure 2: The 43 ranking cells of the default candidates A–E (rows of `CANDIDATES`). Colour marks the first choice; every cell has its own full ranking.

2.2. Cell probability as a boundary integral

Computing the double integral Equation 14 directly for every cell and every pixel would be expensive. Green's theorem turns it into a sum of integrals along the cell's edges, and each edge is shared by two cells.

2.2.1. Green's theorem

For a region K with CCW boundary ∂K and a 1-form $\omega = L\,\mathrm{d}x + M\,\mathrm{d}y$,

$$\oint_{\partial K} L\,\mathrm{d}x + M\,\mathrm{d}y = \iint_K \left(\partial_x M - \partial_y L\right)\mathrm{d}x\,\mathrm{d}y. \tag{15}$$

Choose

$$\omega = -f(x)G(y)\,\mathrm{d}x. \tag{16}$$

Then $\partial_x M - \partial_y L = f(x)g(y)$, which is the joint density, so

$$P(K) = \oint_{\partial K} \omega = \sum_{\text{edges } s\to e} \int_s^e \omega. \tag{17}$$

An equally valid choice is

$$\omega' = F(x)g(y)\,\mathrm{d}y, \quad \omega' - \omega = Fg\,\mathrm{d}y + fG\,\mathrm{d}x = \mathrm{d}(FG). \tag{18}$$

Because ω' and ω differ by an exact form, on any segment

$$\int_s^e \omega = \int_s^e \omega' - [F(x)G(y)]_s^e. \tag{19}$$

Which form is integrated numerically is chosen edge by edge (Section 2.3).

Sanity check. The probabilities of all cells sum to one: interior edges are traversed once in each direction and cancel, leaving the boundary of the square. On it $\mathrm{d}x = 0$ on the vertical sides, $G(0) = 0$ on the bottom, and the top ($y = 1$, traversed from $x = 1$ to $x = 0$) gives $-\int_1^0 f(x)\,\mathrm{d}x = 1$.

2.2.2. Sharing edges (`margin/shares.py`)

Every edge is stored once under a canonical orientation (lexicographically smaller endpoint first). A cell that traverses the edge in the canonical direction adds its integral with sign $+1$; the neighbouring cell traverses it the other way and adds it with sign -1. The edges are integrated in parallel threads (`betainc` / `betaincinv` release the GIL).

The result of each edge integral is a matrix over the interpolation nodes: entry $[k, l]$ uses the X-parameters of node k and the Y-parameters of node l. Because X and Y are independent, one set of N parameter pairs serves both axes, and the whole $N \times N$ grid of node shares comes out of $O(N)$ one-dimensional quantities combined in an outer-product fashion.

2.3. Edge integrals (`TabulatedBeta.edge_terms`)

Let the edge go from $s = (x_s, y_s)$ to $e = (x_e, y_e)$.

2.3.1. Axis-parallel edges (closed form)

- **Vertical** ($x_s = x_e$): $\mathrm{d}x = 0$, so $\int \omega = 0$.
- **Horizontal** ($y_s = y_e$): $G(y)$ is constant, so

$$\int_s^e \omega = -G(y_s)(F(x_e) - F(x_s)). \tag{20}$$

These include all edges on the boundary of the square.

2.3.2. General edges: substitution $u = F(x)$

On the line through s and e, $y = \ell(x) = y_s + (x - x_s)(y_e - y_s)/(x_e - x_s)$. Integrating Equation 16 directly is a bad idea: $f(x)$ is infinite at 0 or 1 when $a < 1$ or $b < 1$ (Section 1.4), and quadrature would converge slowly. Substituting $u = F(x)$, $\mathrm{d}u = f(x)\,\mathrm{d}x$ absorbs the density:

$$\int_s^e \omega = -\int_{F(x_s)}^{F(x_e)} G(\ell(F^{-1}(u)))\,\mathrm{d}u. \tag{21}$$

The integrand is now bounded by 1 and smooth unless the edge reaches $y = 0$ or $y = 1$, where G itself behaves like y^a or $1 - (1 - y)^b$ and is not smooth.

2.3.3. Edges touching $y = 0$ or $y = 1$: substitution $v = G(y)$

For those edges the roles of the axes are swapped. Integrate ω' from Equation 18 with $v = G(y)$, $\mathrm{d}v = g(y)\,\mathrm{d}y$, and $x = \ell^{-1}(y)$:

$$\int_s^e \omega' = \int_{G(y_s)}^{G(y_e)} F(\ell^{-1}(G^{-1}(v)))\,\mathrm{d}v, \tag{22}$$

and convert back to ω with Equation 19. Now the endpoint on $y \in \{0, 1\}$ is harmless because it is mapped to $v \in \{0, 1\}$ and the singular behaviour of G is absorbed by the substitution, while x stays in the interior of $(0, 1)$ so F is smooth there.

2.3.4. Edges touching both kinds of boundary

An edge can run from a vertical side of the square ($x \in \{0, 1\}$) to a horizontal one ($y \in \{0, 1\}$). Neither substitution handles both endpoints, so the edge is split at its midpoint; each half touches only one kind of boundary and is handled as above. (An edge ending exactly at a corner of the square is integrated with the v form.)

2.3.5. Quadrature

The remaining one-dimensional integrals over $[u_s, u_e]$ (or $[v_s, v_e]$) are computed with Q-point Gauss–Legendre quadrature (`QUAD_NODES`, default $Q = 24$):

$$\int_{u_s}^{u_e} h(u)\,\mathrm{d}u \approx \frac{u_e - u_s}{2} \sum_{q=1}^{Q} w_q h\left(u_s + (u_e - u_s)\frac{t_q + 1}{2}\right), \tag{23}$$

with Legendre nodes t_q and weights w_q on $[-1, 1]$. Note that the interval $[F_k(x_s), F_k(x_e)]$ depends on the node parameter k, so the quadrature nodes are placed separately for every interpolation node.

Cost. For the u form, F_k^{-1} is evaluated at Q nodes for each of the N column-parameters (QN quantile evaluations), then G_l is evaluated at each resulting y for each of the N row-parameters (QN^2 CDF evaluations). An edge therefore costs $O(QN^2)$ Beta CDF evaluations, and the total cost is $O(E \cdot Q \cdot N^2)$ for E distinct edges.

2.4. Interpolation in the median

Evaluating all n medians directly, $O(Qn^2)$ per edge would be costly. But the cell probabilities are **smooth** functions of the medians (m_x, m_y): (a, b) depends smoothly on m, and the integrals Equation 14 depend smoothly on (a, b). (The **winner** of a voting method jumps between evaluation points, but the probabilities do not.) So they are computed exactly only on an $N \times N$ grid of medians (`NODES`, default $N = 49$) and interpolated to the $n \times n$ evaluation grid. If $N \geq n$ no interpolation is used.

2.4.1. Nodes (`node_medians`)

The interpolation variable is $z = \mathrm{logit}(m) = \log(m/(1-m))$. It stretches the regions near 0 and 1, where (a, b) change fastest. The nodes are the Chebyshev–Lobatto points in z spanning the first and last target median:

$$t_j = -\cos\left(\pi \frac{j}{N-1}\right), \quad z_j = Z t_j, \quad m_j = \mathrm{expit}(z_j), \quad j = 0, ..., N-1, \tag{24}$$

where $Z = \mathrm{logit}\left(1 - \frac{1}{2n}\right)$. (The code writes t_j as $\sin\left(\pi \frac{2j-(N-1)}{2(N-1)}\right)$, which is the same thing.) Chebyshev–Lobatto points cluster at the ends of the interval, which further concentrates nodes near the walls and avoids the Runge phenomenon.

2.4.2. Barycentric interpolation (`_interpolation_matrix`)

For Chebyshev–Lobatto nodes the barycentric formula of the second kind is

$$p(z) = \frac{\sum_j \frac{\lambda_j}{z - z_j} h_j}{\sum_j \frac{\lambda_j}{z - z_j}}, \quad \lambda_j = (-1)^j, \quad \lambda_0, \lambda_{N-1} \text{ halved}, \tag{25}$$

where h_j are the values at the nodes. (The weights are invariant under the affine map $t \to Zt$, up to a common factor that cancels.) When a target coincides with a node, the node value is used directly. Evaluated at all n target medians this is a fixed $n \times N$ matrix L, and on the 2-D tensor grid

$$P_{\text{grid}}[\cdot, \cdot, r] = L\, P_{\text{nodes}}[\cdot, \cdot, r]\, L^T, \tag{26}$$

applied as two matrix products (`ranking_cells.interpolate_to`). The final product is carried out in `float32` because the result is by far the largest array; the rounding ($\approx 10^{-7}$) can only affect evaluation points whose winning margin is equally small. Values are finally clipped to $[0, 1]$.

Because the probabilities are analytic in z, the interpolation error decreases exponentially in N. The interpolation reproduces constants exactly, so the probabilities at each evaluation point still sum to one up to rounding. Narrow voters change faster with the median: for Beta voters with $D < 0.1$ the web UI uses $2N - 1 = 97$ nodes, because with 49 the shares at a target median with $D = 0.05$ would sum to up to 1.025.

Edge integrals are cached in memory between requests. Interpolation to the evaluation grid happens after the node shares have been computed.

2.5. Normal voters (`normal.py`)

`normal.py` computes the shares for normal voters (Section 1.2).

2.5.1. Cells in a larger box (`normal_cells`)

Voters can lie outside the square, so the bisector arrangement is built in the box $[-L, 1+L]^2$ with $L = 10\sigma$ (`BOX`). For every pixel centre in $[0,1]^2$ the mass outside the box is at most $4\Phi(-10) \approx 3 \cdot 10^{-23}$. The similarity map $p \mapsto (p+L)/(1+2L)$ maps perpendicular bisectors to perpendicular bisectors and preserves the order of distances, so `ranking_cells` is applied to the mapped candidates and the polygons are mapped back.

2.5.2. Exact cell probabilities with Owen's T function

No quadrature is needed. For a convex CCW polygon K with vertices $p_1, \ldots, p_r$ and any point m, the indicator of K is, almost everywhere, the signed sum of the indicators of the triangles (m, p_k, p_{k+1}) (a fan from m; triangles oriented clockwise count negatively). Hence

$$P(K) = \sum_{\text{edges } s \to e} P_{\pm}(m, s, e). \tag{27}$$

Each triangle (m, s, e) is split at the foot q of the perpendicular from m onto the edge line into two right triangles with legs ρ (from m to q) and t (from q along the line). The isotropic normal is invariant under rotations, so in units of σ a right triangle is $\{0 \le z_1 \le h, 0 \le z_2 \le a z_1\}$ with $h = \rho/\sigma$, $a = t/\rho$, for standard normal (Z_1, Z_2). The wedge $\{z_1 > 0, 0 < z_2 < a z_1\}$ holds $\arctan(a)/(2\pi)$ of the mass, and the part of it beyond $z_1 = h$ is by definition Owen's T function,

$$T(h, a) = \frac{1}{2\pi} \int_0^a \frac{\exp(-h^2(1+x^2)/2)}{1+x^2}\,\mathrm{d}x = P(Z_1 > h, 0 < Z_2 < aZ_1) \quad (h, a \ge 0). \tag{28}$$

Therefore the right triangle has probability

$$R(\rho, t) = \frac{\arctan(t/\rho)}{2\pi} - T(\rho/\sigma, t/\rho). \tag{29}$$

Both terms are odd in t; T is even in h and odd in a, so Equation 29 with a **signed** ρ is the signed probability. For the edge $s \to e$ with unit direction u and left normal $u^{\perp} = (-u_y, u_x)$ let

$$\rho = (m - s) \cdot u^{\perp}, \quad t_s = (s - m) \cdot u, \quad t_e = t_s + |e - s|, \tag{30}$$

where $\rho > 0$ when m lies left of the edge, i.e. inside a CCW cell. Then

$$P_{\pm}(m, s, e) = R(\rho, t_e) - R(\rho, t_s), \tag{31}$$

and 0 when $\rho = 0$ (the triangle degenerates). For the $N \times N$ grid of node medians ρ, t_s, t_e are outer sums, so each edge costs two evaluations of Owen's T per node. They are compiled (Section 8.4): for $0 \le a \le 1$ the integral above with 12-point Gauss–Legendre, which agrees with scipy's `owens_t` to 10^{-16} for every h (the integrand is analytic in x), and for $a > 1$ the identity

$$T(h, a) + T(ah, 1/a) = \frac{1}{2}(\Phi(h) + \Phi(ah)) - \Phi(h)\Phi(ah) \quad (h, a \ge 0), \tag{32}$$

which leaves $T(ah, 1/a)$ with $1/a < 1$. Five candidates take about 0.02 s on 49×49 nodes.

2.5.3. Nodes

Unlike the Beta parameters, nothing changes faster near 0 or 1, so the nodes are Chebyshev–Lobatto points in the median itself, $m_j = \frac{1}{2} + \left(\frac{1}{2} - \frac{1}{2n}\right) t_j$, and the barycentric interpolation of Section 2.4 is applied in m instead of $\mathrm{logit}(m)$ (`ranking_cells.interpolate_to(..., transform=...)`). The logit nodes would be far too sparse in the middle for small σ (maximum error at 120 pixels, $N = 49$):

nodes in	$\sigma = 0.376$	$\sigma = 0.125$	$\sigma = 0.063$
m	$1.7 \cdot 10^{-7}$	$3.6 \cdot 10^{-7}$	$2.9 \cdot 10^{-7}$
$\mathrm{logit}(m)$	$1.6 \cdot 10^{-7}$	$7.5 \cdot 10^{-5}$	$1.4 \cdot 10^{-2}$

The floor of about 10^{-7} is the `float32` rounding of the result.

2.6. Accuracy and tests

The tests (`tests/`) check the probabilities against quantities that are known independently:

- the cells partition the square (or the box) and every point inside a cell has the cell's ranking;
- the shares at every evaluated median sum to one (to 10^{-12} before interpolation);
- for a bisector parallel to an axis the share preferring one candidate is a Beta CDF, and at median $\frac{1}{2}$ it is exactly one half (to 10^{-8});
- for normal voters the pairwise shares match their closed form (Equation 56) (to 10^{-12}), and every Condorcet method draws the Voronoi diagram;
- the shares match 10^6 sampled voters within five standard errors (`monte_carlo_check` does the same for any pixels, with $2 \cdot 10^6$ voters, whose standard error is at most $1/\left(2\sqrt{M}\right) \approx 3.5 \cdot 10^{-4}$);
- interpolated shares match the exact ones at evaluated medians (to $2 \cdot 10^{-5}$ for Beta voters with 33 nodes, $2 \cdot 10^{-6}$ for normal voters with 49 nodes).

3. Voting methods (`yeelab.build`)

For each evaluated median, a method operates on the voter shares it needs and returns the winner and its margin. Most methods need only pairwise or first-choice shares; IRV and DAC need the complete ranking profile. Because an electorate is a distribution, the calculations use shares instead of vote counts.

3.1. First past the post (`fptp`)

The first-choice share of candidate i is

$$s_i = \sum_{r : r_1 = i} P(r), \tag{33}$$

and the candidate with the largest share wins.

3.2. Instant runoff (`irv`)

With a set S of eliminated candidates, every ballot counts for its highest ranked candidate outside S,

$$s_i^S = \sum_{r\text{: first of } r \text{ outside } S \text{ is } i} P(r). \tag{34}$$

The candidate with the smallest share is added to S, and after $C-1$ rounds the remaining candidate wins. The rankings are complete, so no ballot is ever exhausted and a candidate with a majority is never eliminated: running to the end gives the same winner as stopping at a majority. The rounds run in a compiled loop over the pixels (Section 8.4): every ranking points to its highest ranked candidate outside S, and eliminating a candidate moves only the rankings that point to it. All $C-1$ rounds together take at most RC steps per pixel.

3.3. Borda count (`borda`)

With $\mathrm{pos}_r(i) \in \{0, \ldots, C-1\}$ the position of i in ranking r,

$$\mathrm{score}_i = \sum_r P(r)(C - 1 - \mathrm{pos}_r(i)), \tag{35}$$

and the highest score wins.

3.4. Pairwise majorities

Condorcet methods only use the pairwise shares

$$\pi_{ij} = \sum_{r : i \text{ above } j \text{ in } r} P(r), \quad \pi_{ij} + \pi_{ji} = 1, \tag{36}$$

the share of voters preferring c_i to c_j; c_i beats c_j when $\pi_{ij} > \frac{1}{2}$.

Condorcet winner (`condorcet`). The candidate who beats every other candidate. If there is none the pixel is marked with `CYCLE` (black in the figures). As a function of the pixel, $\pi_{ij} = \frac{1}{2}$ holds only on a curve, so apart from pixel centres that happen to lie exactly on such a curve (Section 3.12) the majority relation is a tournament. A tournament in which nobody beats everyone always contains a cycle (a tournament without cycles is a strict order and has a top), so a black pixel is a Condorcet cycle unless it is such a tie.

Schulze (`schulze`). The defeat strength of c_i over c_j is π_{ij} if $\pi_{ij} > \pi_{ji}$ and 0 otherwise. The strength of the strongest path is found with the Floyd–Warshall recursion $p_{ij} \leftarrow \max\big(p_{ij}, \min\big(p_{ik}, p_{kj}\big)\big)$ for $k = 1, \ldots, C$, and the winner is a candidate with $p_{ij} \geq p_{ji}$ for all j. Schulze elects the Condorcet winner whenever there is one, so its diagram differs from the Condorcet winner diagram only on the black pixels, where it resolves the cycle.

3.5. Baldwin and Nanson (`baldwin`, `nanson`)

These two are only in the web UI (Chapter 8), where they are built from blocks (`yeelab/build/`) as `Eliminate(Tally(BordaCount()), how="min")` and `how="mean"`. Both repeat the Borda count among the remaining candidates S. There a ballot gives c_i one point per remaining candidate below it, and the expected number of those is the sum of the chances of being above each of them:

$$\text{score}_i^S = \sum_r P(r)\big(|S| - 1 - \text{pos}_r^S(i)\big) = \sum_{j \in S, j \neq i} \pi_{ij}, \tag{37}$$

with $\text{pos}_r^S(i)$ the position of c_i among S in ranking r. For S = all candidates this is the Borda score, so Borda, Baldwin and Nanson need only the pairwise shares. As $\pi_{ij} + \pi_{ji} = 1$, the scores in S add up to the number of pairs, and their mean is $(|S| - 1)/2$.

Baldwin drops the candidate with the lowest score (ties: the first) and counts again, until one is left. **Nanson** drops, in every round, each candidate whose score is at most the mean; if that is all of them, all scores are equal and the first candidate wins (a set of zero area). A Condorcet winner beats every other remaining candidate, so its score is above the mean, and both methods elect it: like Schulze, they differ from the Condorcet winner only where there is a cycle.

Their margins (Section 8.5) are the smallest gap of any round: for Baldwin the gap between the two lowest scores, as for IRV, and for Nanson the distance of the score closest to the mean,

$$\mu_{\text{Baldwin}} = \min_S \Big(\text{score}_{(2)}^S - \text{score}_{(1)}^S\Big), \quad \mu_{\text{Nanson}} = \min_S \min_{i \in S} \left|\text{score}_i^S - \frac{|S| - 1}{2}\right|, \tag{38}$$

over the sets S of the rounds, with $\text{score}_{(1)}^S \leq \text{score}_{(2)}^S$ the two lowest. The winner changes only where the decision of some round flips, a tie of the two lowest or a score at the mean, and there the gap of that round is 0.

3.6. Minimax and Black (`minimax`, `black`)

These two are only in the web UI as well, built from blocks as `Highest(Weakest(Margins(Pairwise())))` and `Conditional(Unbeaten(Margins(Pairwise())), when_true=Unbeaten(Margins(Pairwise())), when_false=Highest(Tally(BordaCount())))`. Both compare candidates by the margins of their head-to-head results,

$$\ell_{ij} = \pi_{ij} - \pi_{ji} = 2\pi_{ij} - 1 = -\ell_{ji}, \tag{39}$$

which are positive where c_i beats c_j.

Minimax. The score of c_i is its weakest link, its narrowest win or, if it loses somewhere, minus its worst defeat,

$$\text{weak}_i = \min_{j \neq i} \ell_{ij}, \tag{40}$$

and the highest score wins (ties: the first): the winner is the candidate whose worst defeat is the smallest. A Condorcet winner has a positive score and every other candidate a negative one, as it loses to the Condorcet winner, so minimax elects the Condorcet winner whenever there is one. In a cycle it elects the candidate with the smallest worst defeat, which is the candidate that the Condorcet winner method finds closest to being one (Section 8.5), so minimax extends it to cycles. Unlike Schulze it looks at a single defeat and not at paths of defeats.

Black. The Condorcet winner if there is one, otherwise the Borda winner. Every pixel has a winner, never a cycle, and like Schulze, Baldwin and Nanson, Black differs from the Condorcet winner diagram only where there is a cycle.

3.7. Descending acquiescing coalitions (`dac`)

This method is only in the web UI as well, built from blocks as `DescendingCoalitions()`: Woodall's descending acquiescing coalitions (DAC). A voter *acquiesces* to a set T of candidates if they rank no candidate outside T above one in T. Every set scores the share of the voters who acquiesce to it, and the sets are taken from the highest score down: each makes the candidates outside it ineligible, unless that would leave no one eligible. When one candidate is left, it wins.

A complete ranking acquiesces exactly to the sets of its first k candidates, so the score of T is the share of the rankings that begin with the members of T,

$$a_T = \sum_{r:\,\{r_1,\ldots,r_{|T|}\}=T} P(r). \tag{41}$$

For a single candidate that is its first-choice share s_i, and for all candidates but c_i the share of the voters who rank c_i last. The empty set and the set of all candidates get every voter and change nothing. On complete rankings these coalitions are also the solid coalitions of Woodall's DSC, so the two methods agree.

Rounds. A set changes the eligible candidates $\mathcal{E}$ only if it *cuts* $\mathcal{E}$: it holds some of them but not all. Every set with a higher score than one that cuts $\mathcal{E}$ was taken while $\mathcal{E}$ was larger, and holds all of $\mathcal{E}$ or none of it since. So the method is a sequence of at most $C-1$ rounds: the set T with the highest score among those that cut $\mathcal{E}$ (ties: the lowest bit mask) keeps $\mathcal{E} \cap T$. At the median $(0.51, 0.51)$ of Section 3.11 ($D = 0.3$), where D is a centrist squeezed between A and E:

round	set T	a_T	eligible after
1	A, B, C, D (E last)	40.8 %	A, B, C, D
2	A, D (the first two)	30.9 %	A, D
3	A (first)	28.2 %	A

A wins, narrowly: 27.7 % of the voters rank A last, which is the score of the set of B, C, D and E, and would have left D. D beats A head to head (Section 3.11), so DAC is not a Condorcet method.

Majorities. Two sets that more than half of the voters acquiesce to share a voter, and the sets one voter acquiesces to are nested. So the sets with more than half of the voters form a chain, and they are taken first. A candidate with more than half of the first choices therefore wins, and where more than half of the voters rank some set of candidates above all others (a mutual majority), one of them wins. In the diagram of A–E with the defaults of the UI (Beta voters, `rms`, $D = 0.2$), DAC elects the FPTP winner at 97 % of the pixels and the IRV winner at 88 %, and of the pixels with a Condorcet winner it elects that winner at 89 %. At $D = 0.3$ these are 88 %, 78 % and 77 %.

Computing. The sets are bit masks, 2^C of them (256 for 8 candidates). At every pixel each ranking adds its share to the $C-1$ sets it begins (the empty set and the full set change nothing), and each round scans the masks for the highest score of a set that cuts $\mathcal{E}$. This runs in a compiled loop like that of IRV (Section 8.4): $R(C-1)$ additions and at most $2(C-1)2^C$ comparisons per pixel. With 8 candidates that takes about half the time of the IRV rounds.

Margin. The winner is in $\mathcal{E} \cap T$ for the set T of every round. A *rival* of the round is a set U that cuts $\mathcal{E}$ but holds none of $\mathcal{E} \cap T$: had it come before T, the winner would be in $\mathcal{E} \cap U$ instead, among the candidates the round drops. The gap of the round is

$$a_T - \max_U a_U \tag{42}$$

over its rivals, and the margin μ is the smallest gap of all rounds. In the example the margin is $0.282 - 0.277 = 0.005$, the gap of the last round. A set that cuts $\mathcal{E}$ but is no rival can come before T and leave the winner: then the two rounds trade places, they reach the same eligible candidates, and their gaps together compare the same scores, so μ stays continuous. Unlike the margin of IRV, it vanishes only where a round's set ties with a rival, that is, where the winner changes. No ranking begins both T and a rival U, since the sets it begins are nested and T and U are not. So a change of the shares by $\delta = \sum_r |\Delta P(r)|$ in all moves every gap by at most δ, and the winner stays while $\delta < \mu$.

3.8. Score voting (`score`)

Score voting is only in the web UI as well, built from blocks as `Highest(Tally(Score(6, power=1.5)))`. A score ballot is not a ranking: the voter gives every candidate a score from 0 to $L-1$, and the candidate with the highest mean score wins. The scores depend on how far the candidates are and not only on their order, so the model needs one more assumption than the ranked methods (`score.py`). The UI has one score method and two sliders for it: *Score categories* sets the number of scores L, from 2 to 11, by default 6, and *Score power p* sets

the power p of Equation 44, from 1 to 2 in steps of 0.05, by default 1.5 (`SCORE_POWER` in `build/methods.py`). With $L = 2$ a score ballot is an approval ballot, and approval voting is score voting with two levels.

The ballot. A voter at p puts the candidates in order of distance,

$$r_{(1)} \le r_{(2)} \le \ldots \le r_{(C)}, \quad r_i = |p - c_i|, \tag{43}$$

and gives the closest the top score $L - 1$ and the farthest 0. Every other candidate gets a score by its *part of the way* from the farthest to the closest, $t_i = \big(r_{(C)} - r_i\big)/\big(r_{(C)} - r_{(1)}\big)$, raised to the power p:

$$\text{score}_i = \text{round}\big(t_i^p (L-1)\big), \quad t_i = \frac{r_{(C)} - r_i}{r_{(C)} - r_{(1)}}, \tag{44}$$

with a half rounded to the even score. A voter as far from every candidate gives them all the top score. $p = 1$ (`POWER` in `score.py`, the default of the block) is in proportion to the distance; above 1 the top scores stay with the candidates near the closest. Only the closest and the farthest candidate set the scale, whatever p.

The ballot has the properties one expects of a sincere score ballot, at every p: the closest candidate gets the top score and the farthest 0, a closer candidate never gets a lower score, the ballot does not change when all distances are scaled or shifted, so no unit of distance has to be chosen, and a copy of a candidate changes no other score. As the voter moves, the scores change one at a time, each where its own part of the way crosses a border, and a small move changes a score by one level at most. With more levels $\text{score}_i/(L-1)$ tends to t_i^p, not to t_i: the power is not a rounding effect, it bends the scale itself. `script/score_edge_cases.py` checks these properties on edge cases and random voters.

What p does. A score changes where its part of the way is half a score,

$$t_i = \left(\frac{k + \frac{1}{2}}{L - 1}\right)^{1/p}, \quad k = 0, \ldots, L-2. \tag{45}$$

p	$L = 2$: approves beyond	$L = 3$	$L = 6$
1	0.50	0.25, 0.75	0.10, 0.30, 0.50, 0.70, 0.90
1.5	0.63	0.40, 0.83	0.22, 0.45, 0.63, 0.79, 0.93
2	0.71	0.50, 0.87	0.32, 0.55, 0.71, 0.84, 0.95

Table 1: The borders of Equation 45: the parts of the way where a score changes, for the ends and the default of the slider.

With two levels the voter approves the candidates beyond $2^{-1/p}$ of the way: halfway for $p = 1$, 0.63 for $p = 1.5$ and 0.71 for $p = 2$ (Table 1). The farthest candidate sets the scale, and it is usually far from every candidate the voter considers, so with $p = 1$ halfway is a generous cut: real voters give the top scores much more sparingly, and $p = 1.5$ is the one power that fits their ballots best (Section 3.9). In the plane the generous cut favours a candidate in the middle of the others: the voters on every side have it closer than halfway, so it is approved by many whose nearest candidate is someone else (Figure 3).

The slider stops at 1 and at 2. No ballot of Section 3.9 is fitted better by a power below 1, and above 2 the ballots with a neutral middle score fit much worse (Section 3.9): with three levels $p = 2$ puts the border of the middle score exactly at halfway, where a voter who gives a candidate the middle opinion of a scale puts it, and the half is rounded down.

Curved borders. A score changes where Equation 45 holds, $r_{(C)} - r_i = \big(\big(k + \frac{1}{2}\big)/(L-1)\big)^{1/p} \cdot \big(r_{(C)} - r_{(1)}\big)$: a curve in the plane, as the distances are not squared. (The bisector of Equation 13 is straight because there the squares $|p|^2$ cancel.) The voters who give a candidate a score are therefore not a union of polygons (Figure 3, top), and the mean score is not a sum of the edge terms of Section 2.2.

Distances, not their squares. Take a voter who stands on a candidate, with the others at the squared distances 0.35, 0.45, 0.55 and 1. The distances are 0, 0.59, 0.67, 0.74 and 1: the candidate at the voter is far ahead of the

rest. On the squares the three in the middle move a part of the way of 0.1 to 0.3 closer to the top score. Real ratings agree with the distances: their squares fit them worst (Section 3.9).

Shares from a grid. Let q_i be the mean score of c_i as a part of the top score, and $u_i = 1 - q_i$ the part not given; with two levels u_i is the share of the voters who do not approve c_i. Like a ranking, a ballot depends only on where the voter is, so the points $b_i(x, y)$ that a voter at (x, y) gives below the top score are the same for every pixel, and $u_i = E[b_i(X, Y)]/(L - 1)$. The plane is cut into rectangles by the lines $x_0 < x_1 < ... < x_K$, the same along both axes. X and Y are independent, so the rectangle $[x_k, x_{k+1}] \times [x_l, x_{l+1}]$ holds the share $\mu_k \nu_l$ of the voters, with $\mu_k = F(x_{k+1}) - F(x_k)$ from the exact CDF F of X, and ν_l likewise from that of Y. With a_{ikl} the mean of $b_i/(L - 1)$ over that rectangle,

$$u_i \approx \sum_{k,l} \mu_k \, a_{ikl} \, \nu_l. \tag{46}$$

For all medians at once these are two matrix products per candidate (`unscored_shares` in `margin/shares.py`). a comes from a *ballot table* (`ballot_table` in `score.py`): for every rectangle the distinct ballots of its voters and the part of them with each. A rectangle whose four corners have the same ballot holds all of that ballot, and the others, about 2% of them, the ballots at 8×8 points, most with two or three distinct ones. Every score share is a mean over the table (`tabulate`): the part not given here, STAR's comparisons and the shares of burial and compromise (Chapter 4). Computing the ballots is most of the work, and the table depends only on the candidates, the ballot and the grid, not on the medians, so it is kept: all score shares of a ballot come from one pass, and the fine grid after a drag reuses the table of the last step.

The grid has 256 equal cells across the square. For Beta voters the cell at each wall is halved again and again down to 10^{-6}: the density of a Beta with $a < 1$ is infinite at the wall, and the voters of a pixel next to a wall sit within a small part of an equal cell. For normal voters the cells grow by a factor of 1.1 each beyond the square, out to the box of Section 2.5.

Only a is approximate: within a rectangle the borders are replaced by a mean over the rectangle. Where exact shares are known the grid is within $2 \cdot 10^{-4}$ of them for $D \geq 0.2$ and within $5 \cdot 10^{-4}$ for $D = 0.05$ (`tests/test_score.py`): with two candidates every ballot gives the first choice the top score and the other 0, so u_i is a share of Equation 33. With two levels the share who approve a candidate is also between the exact shares of the voters with it first and with it among the $C - 1$ closest.

Why the part not given. Narrow voters far from the candidates all give the same candidates the top score. With four candidates and $D = 0.05$ the voters of a pixel in a far corner approve the same two, and both q_i are 1 up to 10^{-20} or less. Double precision cannot tell $1 - 10^{-20}$ from 1, so on a large part of the square the two q_i would be equal, or differ only by rounding. The winner there is the candidate with the smaller u_i, and Equation 46 has no negative terms, so a u_i of 10^{-40} is as exact, in relative terms, as one of 0.4. Two things keep it so.

- μ_k is exact in both tails. Below the median it is $F(x_{k+1}) - F(x_k)$. Above the median F is close to 1 and that difference would be 0; there $\mu_k = S(x_k) - S(x_{k+1})$ with $S = 1 - F$ computed directly, as $I_{1-x}(b, a)$ for Beta voters and $\Phi(-z)$ for normal ones.
- The u_i are not interpolated. The interpolant of Section 2.4 is exact to about 10^{-5}, far more than these shares. They are computed at the points where the borders are traced (Section 8.5), with μ and ν of the voters of those points; for Beta voters their parameters are solved at each of these medians (Section 1.6). One more point is one more row of each matrix product.

With two candidates and normal voters u_i is known: Φ of the distance to the bisector over σ. The grid follows it within 6% down to 10^{-36} (`tests/test_score.py`). At $D = 0.05$ and $G = 320$ such a tail falls by a factor of about 1.5 from one traced point to the next, so 6% moves a border by a small part of that step.

Winner. The candidate with the largest q_i wins, that is the smallest u_i, and the margin (Section 8.5) is its lead over the second: the second smallest u_i minus the smallest. The tally of a score ballot in `yeelab.build` is $-u_i = q_i - 1$, the mean score counted down from 1, that of `Tally(ballot)`; `Tally(ballot, statistic="median")`

takes the median of the same scores instead (Section 3.10). With two levels the q_i do not sum to 1: their sum is the mean number of approved candidates.

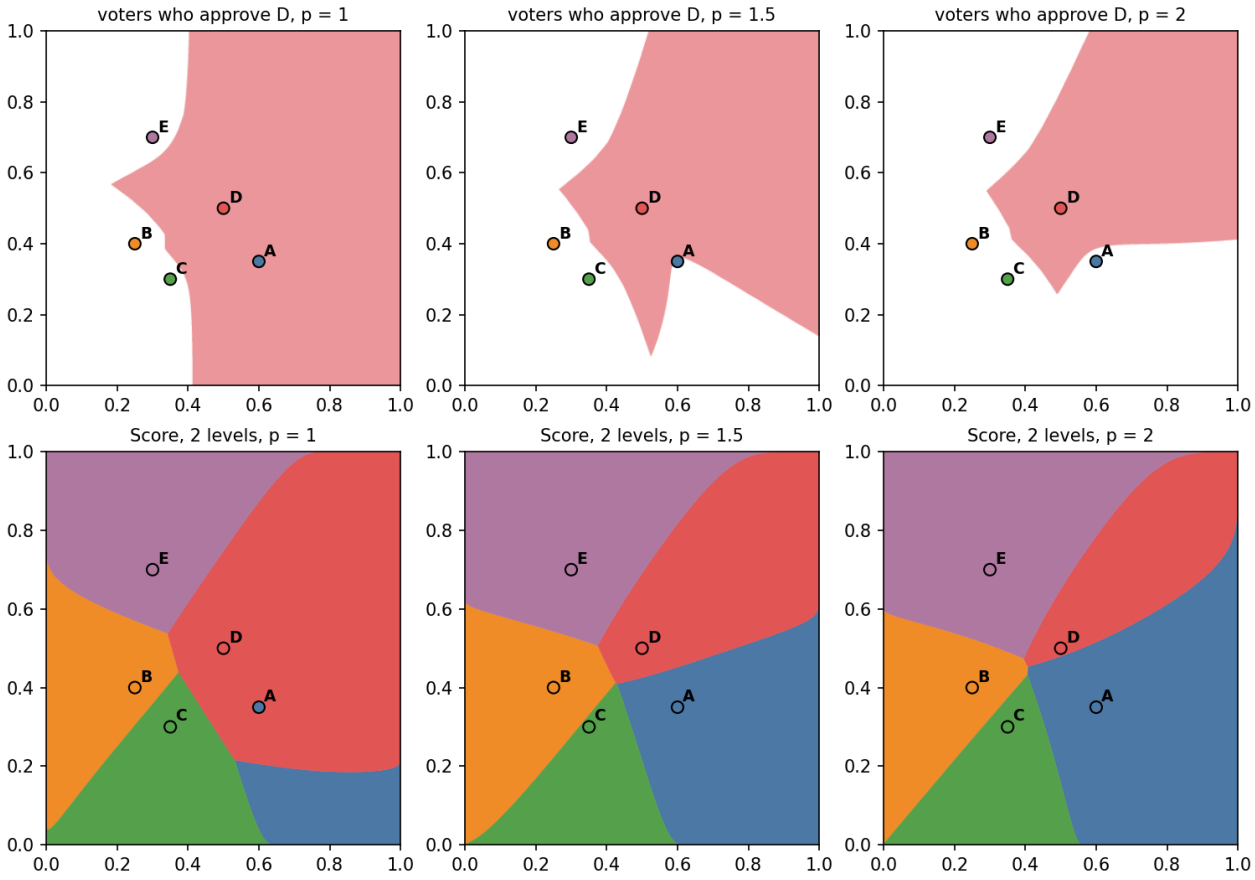


Figure 3: Two levels at the ends and the default of the slider, $p = 1$, 1.5 and 2. Top: the voters who approve D, those beyond $2^{-1/p}$ of the way from the farthest candidate to the closest; the borders are curves. Bottom: the diagrams (candidates A–E, Beta voters, `rms`, $D = 0.2$).

Two levels. D is in the middle of A–E, and its Voronoi cell is 21% of the square (Figure 3). With $p = 1$ D is approved by 64% of the voters of the square and wins 42% of it, more than FPTP (14%), Schulze (25%) or Borda (31%), and A only 8%: a voter near A has B, C and E far away, which puts D closer than halfway, so D is approved as well. With $p = 1.5$ the cut moves to 0.63 of the way, D is approved by 45% and wins 23%, and A 25%; with $p = 2$, 36% and 16%, and A 31%. A voter approves 2.3 candidates on average with $p = 1$, 1.9 with 1.5 and 1.6 with 2. The part of the square not won by the nearest candidate falls from 24% to 9% and 8%.

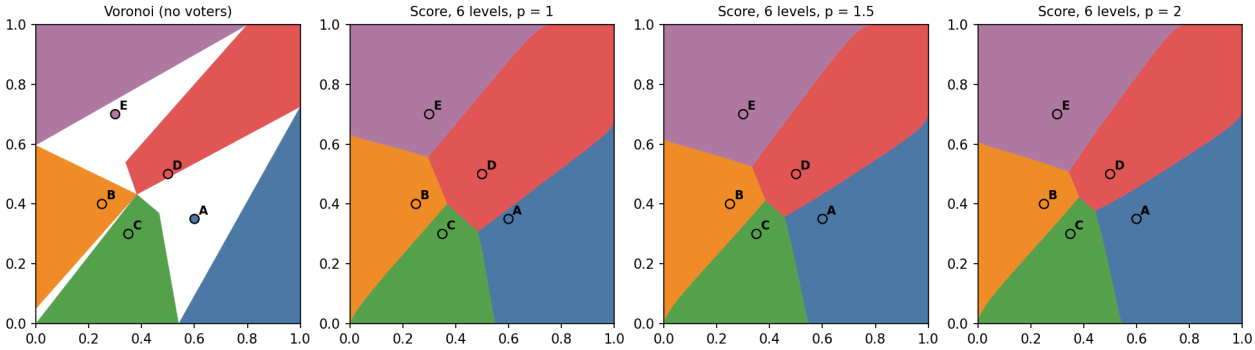


Figure 4: Six levels, the default, at $p = 1$, 1.5 and 2, next to the Voronoi diagram (candidates A–E, Beta voters, `rms`, $D = 0.2$).

Six levels. With six levels the power matters less, as more levels give every candidate more of its part of the way (Figure 4): D wins 29%, 24% and 22% of the square with $p = 1$, 1.5 and 2, and A 24%, 26% and 27%; the part

not won by the nearest candidate is 10%, 5% and 3%. The mean scores of the voters of the square fall with p, D from 3.15 to 2.62 and 2.37 of 5, as every candidate but the closest gets less.

Against the Voronoi diagram. The same holds for other candidates. Over 60 random layouts of 3 to 7 candidates, every other one with two candidates 0.08 to 0.2 apart, with the voters of the UI's default ($D = 0.2$), the part of the square not won by the nearest candidate is (`docs/figures.py`):

	2 levels	lost	6 levels	lost
`Score`, $p = 1$	34.4%	1.43	14.2%	0.12
`Score`, $p = 1.5$	23.6%	0.92	9.0%	0.05
`Score`, $p = 2$	18.4%	0.65	7.6%	0.07
`ScoreAvg`	37.7%	1.38	14.1%	0.10
`ScoreDH`	29.5%	1.45	15.6%	0.28
`ScoreHybrid`	15.1%	0.68	12.2%	0.17
`ScoreCluster`	34.5%	1.48	13.7%	0.15

Table 2: The part of the square not won by the nearest candidate (at the median of the voters), and *lost*: the candidates per diagram who win less than a quarter of their Voronoi cell, over 60 random layouts (Beta voters, `rms`, $D = 0.2$).

A higher p is closer to the Voronoi diagram, but only up to a point: the Voronoi diagram is what the voters' own nearest candidate would draw, and a ballot that gives the top score to the closest candidate alone would draw it best at small D. That is not how voters score (Section 3.9), so the Voronoi diagram only says in which direction to bend; the data say how far.

Other ballots. `score.py` has four other ways from distances to scores. The web UI does not offer them, but `script/score_edge_cases.py`, `docs/ballots.py` and the comparisons above use them, as `ScoreAvg`, `ScoreDH`, `ScoreHybrid` and `ScoreCluster` in `yeelab.build`.

- `ScoreAvg` puts the mean distance $\overline{r} = \frac{1}{C}\sum_j r_j$ in the middle of the scale, so with two levels it approves the candidates closer than $\overline{r}$: the best approval ballot of a voter with the utility $-r_i$ who takes every pair of candidates to be as likely to tie (Weber). Every candidate moves the scale through $\overline{r}$, so a copy of a candidate changes the other scores, and the far candidates of an election raise the mean and with it the cut: of all these ballots it fits real ones worst.
- `ScoreDH` shares the $L - 1$ steps from the top score to 0 out among the gaps between neighbours in the order of distance, each in turn to the gap with the largest gap/(its steps $+\delta$), $\delta = 0.8$ (D'Hondt at 1); with two levels it approves the candidates above the largest gap. Candidates close together share a score, so two near rivals are approved alike and one of them can lose its region altogether, and a small move of the voter can change several scores at once.
- `ScoreHybrid` does what `ScoreDH` does for the candidates closer than halfway, on the upper half of the scale, and what $p = 1$ does for the others, on the lower half. Its cut is far from the farthest candidate, as on real approval ballots (it is the best rule with two levels in Table 2), but it inherits the jumps of `ScoreDH`, and of the 15 kinds of real ballot it fits only the Italian approval ballots better than $p = 1.5$.
- `ScoreCluster` gives the scores of $p = 1$ unless they split a cluster of candidates much closer to each other than to those around them, at a cost μ per split and with κ setting how far apart a cluster must stand. It has two parameters more and fits real ballots no better than $p = 1$.

`script/score_edge_cases.py` checks ten properties of a ballot on edge cases (small moves, clusters swept across the scale, clones, many levels, scaling, the order of the scores). `Score` meets all ten at every power of the slider, `ScoreAvg` and `ScoreCluster` nine, `ScoreHybrid` eight and `ScoreDH` six. On real ballots `Score` with $p = 1.5$ misses 19.6% fewer points than with $p = 1$, `ScoreHybrid` 10.8% fewer, `ScoreCluster` 0.2% more, `ScoreDH` 7.5% more and `ScoreAvg` 22.9% more (Section 3.9).

3.9. Score ballots on real ballots

Equation 44 is an assumption about how a voter turns distances into scores. It can be checked where the same people gave both something a distance follows from and a coarser ballot or rating of the same candidates. Seven public data sets do (Table 3); `docs/ballots.py` downloads them and prints every number of this section (`uv run --with pandas --with rdata --with pyreadstat python docs/ballots.py`). The ballot depends on the distances only up to a common scale and shift, so any of these distances gives the same ballot as the distance in a plane would.

data	people and candidates	distance	ballots
Voter Autrement 2017, online	37 726 participants, 11 candidates of the French presidential election	100 minus their opinion, 0–100	approval; evaluation on 0/1/2, −1/0/1, 0/1/2/3 and −1/0/1/2
Voter Autrement 2022	2 284 participants, 12 candidates	100 minus their opinion, 0–100	approval; scores on 0/1/2, −1/0/1, 0/1/2/3, −1/0/1/2; majority judgment on 5 and 7 grades
Votare Altrimenti 2022	1 021 respondents, 14 Italian parties	100 minus their opinion, 0–100	approval; scores 0–4; evaluative 0–4
`perfume_ideal`, `cream_id`, SensoMineR	103 and 86 consumers, 14 perfumes and 9 creams	perceived to ideal profile	liking 1–9 and 0–10
CSES Integrated Module	235 030 respondents, 3 to 9 parties each	left–right, as the respondent placed self and party	like–dislike 0–10
Kuhlmann et al. 2017	879 people, 26 personality items	top of a slider 1–101 minus the answer	Likert 1–5
Zhang et al., CES-D 8	394 people, 8 items	top of either of two sliders 0–100 minus the answer	Likert 0–4; scale 0–14

Table 3: Data sets that pair a distance with a ballot or rating of the same candidates by the same people: three voting experiments (15 kinds of ballot), and ratings given without a stake (8 kinds).

In the voting experiments every participant cast a few ballots of kinds drawn at random and gave each candidate an opinion on a slider from 0 to 100. A ballot of Equation 44 uses its lowest and its highest score, so only the voters whose real ballot does too count: 10 108 of the 10 479 Voter Autrement 2017 participants with an approval ballot and every opinion, for instance. A rating on more than four levels is first stretched from the person's lowest to their highest rating, as few people use the whole of a long scale when nothing is at stake. The measure is the mean distance between the ballot of Equation 44 and the real one, in points of the scale, per voter and then over the voters.

The power. Table 4 gives, for every kind of ballot, the points missed at the ends and the default of the slider and the best power.

ballots	L	voters	$p = 1$	1.5	2	best p
VA 2017 approval	2	10 108	0.150	0.092	0.083	1.90
VA 2017 0/1/2	3	2 636	0.223	0.144	0.148	1.75
VA 2017 −1/0/1	3	6 074	0.196	0.168	0.211	1.35
VA 2017 0/1/2/3	4	4 399	0.312	0.215	0.221	1.65
VA 2017 −1/0/1/2	4	5 389	0.284	0.221	0.256	1.40
VA 2022 approval	2	1 329	0.139	0.099	0.091	2.00
VA 2022 0/1/2	3	365	0.186	0.152	0.205	1.50
VA 2022 −1/0/1	3	729	0.180	0.182	0.268	1.20
VA 2022 0/1/2/3	4	617	0.351	0.246	0.257	1.55
VA 2022 −1/0/1/2	4	712	0.309	0.274	0.312	1.25
VA 2022 MJ, 5 grades	5	1 140	0.424	0.376	0.405	1.45
VA 2022 MJ, 7 grades	7	1 131	0.597	0.543	0.611	1.40
Italy 2022 approval	2	753	0.151	0.103	0.088	2.90
Italy 2022 scores 0–4	5	708	0.487	0.442	0.462	1.50
Italy 2022 evaluative 0–4	5	635	0.537	0.513	0.543	1.35
perfumes, liking	9	103	1.773	1.860	2.019	0.95
creams, liking	11	86	2.185	2.344	2.536	1.00
CSES, like–dislike	11	235 030	2.902	2.874	2.942	1.35
Kuhlmann, Likert 1–5	5	570	0.416	0.544	0.688	1.00
Zhang, slider c, Likert 0–4	5	383	0.873	0.904	0.948	0.95
Zhang, slider c, scale 0–14	15	390	2.251	2.419	2.664	0.95
Zhang, slider d, Likert 0–4	5	383	0.870	0.895	0.942	1.05
Zhang, slider d, scale 0–14	15	390	2.182	2.336	2.580	1.00

Table 4: Points missed per candidate by the ballot of Equation 44 at $p = 1$, 1.5 and 2, and the power that misses fewest (searched from 0.8 to 3 in steps of 0.05). VA: Voter Autrement; MJ: majority judgment.

Every one of the 15 kinds of election ballot is fitted better by a power above 1, from 1.20 to 2.90. One power for all of them (Table 5) misses 19.6% fewer points than $p = 1$ at $p = 1.5$, and is worse on no kind by more than 0.8%. The optimum is flat from 1.4 to 1.7; beyond 2 it falls apart, as the scales with a neutral middle fit much worse.

p	1.2	1.3	1.4	1.5	1.6	1.8	2.0	2.5
elections: mean	−14.3%	−17.1%	−18.7%	−19.6%	−19.5%	−18.2%	−10.6%	+0.4%
elections: worst kind	−4.0%	−1.8%	−0.1%	+0.8%	+3.5%	+8.8%	+48.8%	+65.6%
elections: kinds better	15	15	15	14	14	14	9	7
ratings: mean	+1.5%	+3.1%	+5.4%	+7.8%	+10.0%	+14.6%	+18.8%	+28.1%

Table 5: The change of the points missed against $p = 1$, the same power for every kind of ballot: the mean over the 15 kinds of election ballot and the worst of them, the number of kinds it fits better than $p = 1$, and the mean over the 8 kinds of rating.

Where voters cut. On the scale of Equation 44, from 1 at the closest candidate to 0 at the farthest, the farthest candidate a Voter Autrement 2017 voter approves is at 0.76 (median) and the closest one the voter does not approve at 0.60; for 80% of the voters every approved candidate is closer than every other one. They approve 2.85 candidates. Approving above 0.7 of the way gets 91.7% of the decisions right, above halfway 85.0%. With

$p = 1$ the ballot approves 4.22 candidates and gets 23.5% of the ballots wholly right; with $p = 1.5$ it approves 3.22 and gets 39.0%, with $p = 2$ 2.69 and 42.9%.

By kind of ballot. Approval wants the most bend: the best single power of the three approval ballots is 2.0 (40.2% fewer points missed; 32.9% at 1.5). The scales from 0 (0/1/2, 0/1/2/3, majority judgment, the Italian 0–4) want 1.55, and the scales with a neutral middle (−1/0/1, −1/0/1/2) the least, 1.35: their middle score is the voter's neutral, not a score halfway to the top. On −1/0/1 with $p = 2$ a candidate exactly halfway gets 0 (Section 3.8): 9.4% of the candidates of Voter Autrement 2022 are there (an opinion of 50, mostly), and 71% of them get the middle score, so they are missed by 0.92 points instead of 0.29. That is why the slider stops at 2.

A power per voter. Voters differ much more than kinds of ballot do: each voter's own best power would miss 52.4% fewer points than $p = 1$. That difference is not in their distances, though. A power chosen per voter from where the other candidates are on the way (by quintile of their mean part of the way, fitted on half the voters of each kind and tested on the other half) misses 22.6% fewer points, one power per kind of ballot 22.2% and $p = 1.5$ for all 20.0%. A voter's power is a matter of temperament, not of where the candidates are, so the slider has one power, the same for every voter.

Other utilities. Equation 44 is a utility t_i^p of the part of the way, scored in proportion. Other utilities with one parameter, scored the same way from 0 at the farthest to 1 at the closest, fit no better (one parameter for all 15 kinds):

utility	parameter	mean	worst kind
t_i^p	$p = 1.55$	−19.6%	+2.3%
$\exp(-k(1 - t_i))$	$k = 1.2$	−19.9%	+1.8%
$\exp\Big(-\big(r_i / \big(s r_{(C)}\big)\big)^2\Big)$	$s = 0.5$	−18.0%	−0.6%
$-\log\big(r_i + c\, r_{(C)}\big)$	$c = 0.5$	−17.6%	+0.6%
$-r_i^q$	$q = 0.6$	−15.2%	+4.8%

Table 6: The change of the points missed against $p = 1$ for other utilities of the distance, each with the one parameter that fits the 15 kinds of election ballot best.

The two convex utilities of the part of the way, the power and the exponential, are the best and nearly the same: both put the approval cut at about 0.63 of the way. Those of the distance itself, which also depend on how far the voter is from even the closest candidate, are worse. The power is the simpler, gives the borders of Equation 45 in closed form, and is the ballot of the definition at $p = 1$.

Ratings. The ratings given without a stake want no bend (Table 4): the best power of the products and of the psychological items is 0.95 to 1.05, and $p = 1.5$ fits them 3% to 31% worse. Only the parties of CSES, rated without a ballot, want 1.35, and gain 1%. The bend is a property of voting, of how sparingly people give the top scores of a ballot, and not of turning a fine scale into a coarse one.

Distances, not their squares. Equation 44 on the distances to an exponent misses by:

	$r^{0.5}$	$r^{0.75}$	r	r^2
perfumes	1.820	1.777	1.773	1.994
creams	2.213	2.213	2.185	2.304
CSES	2.904	2.868	2.902	3.171

The squares are the worst in all three; the distances themselves are best for the products and within 0.04 of the best for the parties.

Limits. The voting experiments are online and their participants chose to take part: 39% of the Voter Autrement 2017 voters above voted Mélenchon in the first round, 21% Macron and 16% Hamon. They gave their opinions after voting, so the opinions may have been fitted to the ballots, and no ballot of the experiments counted. The opinions are utilities, not distances in a plane: where utility is not linear in distance, the cut falls elsewhere in

the plane. The power was chosen on these data; the Voter Autrement rounds of 2017 and 2022 and the Italian survey agree on it, but they are not independent of the choice.

3.10. Majority judgment (`mj`)

Majority judgment (Balinski and Laraki) is only in the web UI as well, built from blocks as `Highest(Tally(Score(6, power=1.5), statistic="median"))`: the ballot of Score (Section 3.8), with the same two sliders, and another statistic of it. `Tally(ballot)` takes the mean, `statistic="mean"` by default; `statistic="median"` takes a score ballot's majority judgment, whose grades are the scores $0, \dots, L-1$ (`build/judgment.py`).

The majority grade. Let $w_i(k)$ be the share of the voters who give c_i the score k, $F_i(k) = w_i(0) + \dots + w_i(k)$ the share at most k and $S_i(k) = 1 - F_i(k)$ the share above k. The majority grade of c_i is its lower median,

$$\alpha_i = \min\{k : F_i(k) \geq 1/2\}, \tag{47}$$

the number of thresholds k above which a majority grades it, $S_i(k) > 1/2$. Where the voters split evenly, $F_i(k) = 1/2$, it is the lower of the two middle scores, and scores no voter gives are passed over. The highest majority grade wins.

Ties on ballots. On n ballots, Balinski and Laraki break a tie of majority grades by taking one majority grade out of each tied candidate's grades and the median of the rest, again and again, until the medians differ: the candidate whose median comes out higher is ahead. On a candidate's grades in order this takes them from the middle out, the lower of the two middle ones first, and the candidates are compared on these sequences, their majority values, at the first place where they differ.

Without ballots. The voters of a pixel are a distribution: there is no n, and one grade is no share of them. Taking out a share t of the voters is. The grades taken out first are the middle ones, so once a share t is out, the middle t of the voters, in the order of their grades, are gone, and the next grades to go are those at $(1-t)/2$ and $(1+t)/2$ of the way, the lower one first. That is decided at the thresholds between the scores. Where a majority grades c_i above k, $S_i(k) > 1/2$, its lower grade drops to k at $t = S_i(k) - F_i(k)$; elsewhere its upper grade rises above k at $t = F_i(k) - S_i(k)$. Two candidates c_i and c_j with different $S(k)$ have different grades at that end from

$$t_k = \begin{cases} 0 & \text{if their majorities are on different sides of } k \\ \min\big(|S_i(k) - F_i(k)|, |S_j(k) - F_j(k)|\big) & \text{otherwise} \end{cases} \tag{48}$$

on, and the one with the larger $S(k)$ has the higher grade there. So the threshold where they differ with the smallest t_k decides, the lower end first at the same t_k: the candidate with the larger $S(k)$ is ahead. A threshold with $t_k = 0$ is a difference of the majority grades, and the next ones by t_k are the tie-break, as deep as it goes. Two thresholds that disagree at the same t_k are only on borders, where the margin below is 0. Candidates with the same $S(k)$ at every threshold have the same distribution and tie; Highest takes the lowest index, as for any totals.

***n* ballots again.** Where the shares are multiples of $1/n$, the procedure on n ballots takes the grade of the lower end at t as its step nt, and that of the upper end as its step $nt + 1$. So the order above is that of the procedure on n ballots, for every such n, and for other shares the limit of it as n grows: it assumes no number of voters, and an electorate replicated has the same order. `tests/test_judgment.py` checks it against the procedure on random ballots, n from 1 to 30, with every kind of tie, also where only grades after the shares above and below the median tell two candidates apart. With two levels the one threshold is approval, and majority judgment is approval voting: the most approvals win, by the margin of Score with two levels.

Totals and margin. A lexicographic order is no number per candidate, so the totals of `Tally(..., statistic="median")` rank the candidates of each pixel instead: the first 0, each next one lower by the gap of its place, the smallest gap of a candidate above it and one below. Highest elects the first, with its smallest gap to another candidate as the margin; among finalists and in an elimination the totals are counted among the

remaining candidates, whose scores do not change. (Eliminating the totals at most their mean, `how="mean"`, is refused: they are only an order.) The gap of c_i and c_j is

$$\gamma_{ij} = \min_k \max\Big(\delta_k[k \text{ agrees}], \max_{m \text{ before } k, m \text{ agrees}} \min(\delta_m, (t_k - t_m)/2)\Big), \quad \delta_k = |S_i(k) - S_j(k)|, \qquad (49)$$

where a threshold agrees if it puts the two in their order, $[\cdot]$ is 1 if it does and 0 otherwise, and m comes before k at a smaller t, or at the same one at the lower end. γ_{ij} is 0 where the order of the two changes: where the δ of the deciding threshold vanishes, or another one reaches its t. The terms of the thresholds before k keep it continuous where the deciding threshold changes. If no candidate's shares change by as much as γ_{ij} in all, $\sum_k |\Delta w(k)| < \gamma_{ij}$ with the shares still adding up to 1, every S and F moves by less than half of it, and every δ and every $(t_k - t_m)/2$ by less than all of it, so the two keep their order. The winner therefore stays while the shares change by less than the margin, and the margin is 0 where the winner changes, as Section 8.5 needs. Where only the rule of the lower end puts a candidate behind, its gap is 0 and its total one step of rounding lower, so that Highest elects the candidate ahead, not the lower index.

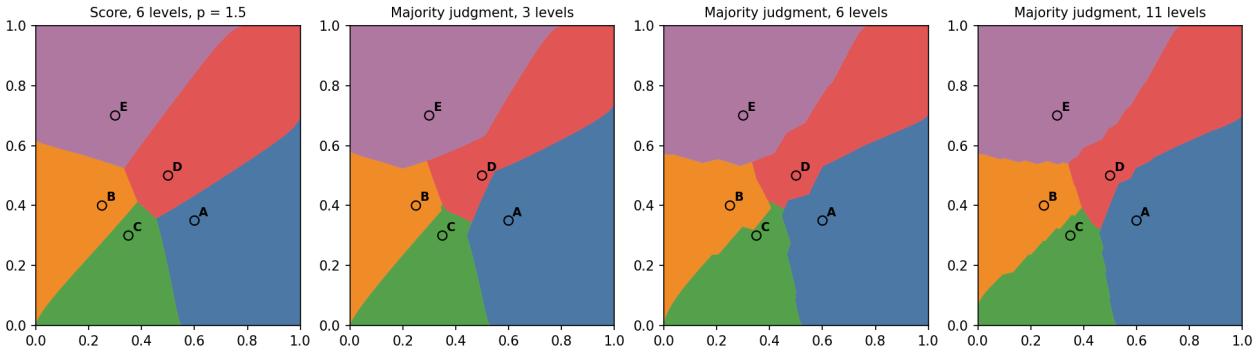


Figure 5: Majority judgment on the ballot of Score with $p = 1.5$, the default, at 3, 6 and 11 levels, next to Score with 6 (candidates A–E, Beta voters, `rms`, $D = 0.2$).

The diagram. The borders are curves in pieces, with a corner where another threshold decides, and more pieces with more levels (Figure 5). With six levels the two candidates ahead have the same majority grade, so that the tie-break decides, at 15% of the square, at 33% with three levels and at 8% with eleven. D wins 19% of the square against 24% under Score, A 30% against 26%; 8% of the square has another winner than under Score, and the part not won by the nearest candidate is 5%, as under Score.

Shares. $w_i(k)$ comes from the ballot table of Section 3.8: `tabulate` with `GRADES` gives the share of the voters of every rectangle at each score of each candidate, which is weighed with the voters of each pixel as in Equation 46, at the points where the borders are traced (`graded_shares` in `margin/shares.py`): L pairs of matrix products per candidate instead of one, from the same table. The part not given is $\sum_k (L - 1 - k) w_i(k)/(L - 1)$. Every share is a sum of terms of one sign, and F and S are each summed from their own end, so the few voters of a tail stay exact in relative terms: where nearly all voters give two candidates the top score, both have it as their majority grade, and the one that fewer voters give less is ahead, as under the mean. With $G = 320$ the shares and the order take about 0.2 s for 5 candidates and 0.3 s for 8, against 0.03 to 0.04 s for Score, the ballot table aside, which both need.

Strategic voters. The shares of every score add up over the voters like the mean scores, so strategic voters (Chapter 4) move them as they move the mean: a voter who changes a score of a frontrunner moves from that score to 0 (burying) or to the top score (compromising), and only the median of the result is not a sum. A burying and a compromising voter move a candidate's score for different voters, so the two add up. `moved_grades` in `score.py` sums, for a pair of frontrunners, the share of the voters of every rectangle at each score of the two that each strategy moves, from the ballot table by order. With burial and compromise the update takes about 0.4 to 0.9 s at $G = 160$ for 5 candidates, against 0.07 to 0.2 s for Score.

3.11. IRV set + KOTH + Score (`irv_set_koth_score`)

This method is only in the web UI as well, built from blocks as `Highest(Tally(Score(6, power=1.5)), among=Schedules(Tally(Plurality())) | koth)`: the highest mean score (Section 3.8) among the finalists $F = W \cup \{k\}$, the winners W of a family of elimination methods and the winner k of king of the hill.

The IRV set W. An elimination schedule splits the $C-1$ eliminations into rounds, $d_1 + \ldots + d_t = C-1$ with every $d_j \geq 1$. Round j drops the d_j candidates with the fewest first choices among the remaining candidates S, the shares s_i^S of IRV, counted again after every round (ties: the first candidate stays). A schedule is a choice of where, among the $C-2$ places between two eliminations, a round ends, so there are 2^{C-2} of them. One at a time is IRV and all at once FPTP. $C-2$ and then one is the two-round system: with complete rankings the first choices among two candidates are their pairwise shares π_{ij}, so the second round is their majority. `Schedules(Tally(Plurality()))` chooses the set W of the winners of all schedules.

The king k (`koth`) is the candidate with the most first choices or, if someone beats it head to head, the one with the most first choices among those who do. `Highest(totals, among=...)` then elects the highest total in F. The score ballot is that of Score, marked once among all candidates, so the scores compared in F are those Score compares. (A `Tally` of a ranked ballot is counted again among the finalists, as among the remaining candidates of a round.)

Properties. A candidate with more than half of the first choices has the most first choices among every set of remaining candidates, so it wins every schedule; it also beats everyone, so it is the king too and the only finalist. A candidate is in W only if it has the most first choices among some set that a schedule reaches. A centrist squeezed between two large flanks never does. With the candidates A–E, Beta voters (`rms`) and $D = 0.3$, at the median $(0.51, 0.51)$, candidate D has 19 % of the first choices against 28 % for A and 27 % for E, and 26 % among the three of them against 41 % and 34 %, so every schedule drops it and $W = \{A, E\}$. Yet D beats A by 59 to 41 % and E by 66 to 34 %, and it has the highest mean score. The king brings such a centrist back where it beats the candidate with the most first choices and has the most first choices of those who do, as D does here; where another challenger has more first choices, the centrist stays out. How often the king adds a finalist to W depends on how wide the voters are: with A–E it does at about 7 % of the pixels at $D = 0.3$, for Beta and normal voters alike, but at less than 0.1 % at $D \leq 0.2$, the default of the UI included. The king is the only finalist besides W, so wherever it changes the winner, k wins.

The two parts differ in burial, ranking a rival insincerely low. While a voter's first choice remains, the ballot counts for it whatever the order below, so burial cannot bring one's own first choice into W. It can make it the king: voters who rank a challenger j above the candidate with the most first choices can rank j below it instead, and if j then no longer beats it, the next challenger becomes k.

Computing W. A round keeps the m highest tallies of the set S it starts with, and what follows depends only on the set it keeps. Every set that some schedule reaches is therefore tallied once per pixel, in a compiled loop like that of IRV (Section 8.4): the sets are bit masks on a stack, and a set is pushed when it is first reached. With the tallies of S sorted, $s^S_{(1)} \geq s^S_{(2)} \geq \ldots$, the top one is in W (the schedule that keeps one candidate next), and keeping m leads to the set of the m highest. With 8 random candidates the 64 schedules reach on average 8 of the 247 sets of two or more candidates per pixel, and at most about 30. The winners change only where a round's decision flips, where two neighbouring tallies of a reached set tie, so

$$\mu_W = \min_S \min_{m=1}^{|S|-1} \left(s^S_{(m)} - s^S_{(m+1)} \right) \tag{50}$$

over the sets S that schedules reach. The margin of the method is the smallest of μ_W, the margin of king of the hill and the lead of the top mean score in F over the next one in F (∞ where F has one member). Like the margin of IRV, it also vanishes where some decision flips but the winner stays. The method needs the whole profile, like IRV, the first-choice and pairwise shares of king of the hill and the score shares of Score, and an update costs about as much as IRV and Score together.

3.12. Ties

Exact ties are broken by the order of the candidates: `argmax` and `argmin` take the first candidate. With exact shares a tie only happens on sets of zero area, e.g. for pixel centres exactly on a bisector under normal voters, where $\pi_{ij} = \frac{1}{2}$ and neither candidate beats the other (the dotted black lines in Figure 9). Majority judgment breaks a tie of majority grades by its own tie-break, and where the voters split evenly by its lower median (Section 3.10); only candidates with the same distribution of scores tie, and go to the first.

4. Strategic voting

Everywhere else every voter votes sincerely: they rank the candidates by distance and score them by distance. A strategy changes the ballots of some of the voters, not the method. `Bullet(method, share)`, `Burial(method, share)` and `Compromise(method, share)` (`build/strategic.py`) evaluate any method of Chapter 3 on voters of whom a share vote strategically at every pixel, the same share among the voters of every first choice, while the others vote sincerely. Every share a method reads is a sum over the voters, so the shares of these voters are those of the sincere voters and of the strategic ones, weighted by their shares. The three combine: `Bullet(Burial(Compromise(method, y), q), b)` is the method where b of the voters bullet vote, and of the others q bury and y compromise, each whatever the other does. A voter who bullet votes would gain nothing by either: their ballot ranks the frontrunner they like less among the last already, and their first choice alone first. The web UI has a slider for each, *Bullet voters* for b, *Burying voters* for q and *Compromising voters* for y, all at 0 at first: every voter sincere.

Bullet voting depends on the voter alone, so its shares are as smooth in the median as before, and the margins and borders are traced the same way (Section 8.5). Burial and compromise depend on who the frontrunners are, which jumps from one pixel to the next, and that adds borders of their own (Section 4.2).

4.1. Bullet voting (`Bullet`)

The ballot. A bullet voter whose first choice is c_k ranks c_k first and all other candidates equal below it. On a score ballot they give c_k the top score and every other candidate 0. The ballot is complete: every candidate has a place on it, the others all share the second one. A method reads equal candidates as follows:

- where it gives points by position, equal candidates share the mean of the points of the positions they take;
- head to head, a voter who ranks two candidates equal abstains;
- in an elimination, once the first choice is out, the ballot counts equally for all remaining candidates, which it ranks equal.

Shares. With s_i the first-choice shares (Equation 33), which bullet voting does not change, the bullet voters with first choice c_i rank c_i above c_j, those with c_j the other way round, and the rest abstain, half for each:

$$\pi'_{ij} = (1-b)\pi_{ij} + b\frac{1+s_i-s_j}{2}, \quad \pi'_{ij}+\pi'_{ji}=1, \quad \pi'_{ij}-\pi'_{ji} = (1-b)\big(\pi_{ij}-\pi_{ji}\big) + b\big(s_i-s_j\big). \quad (51)$$

The Borda score is still $\sum_{j\neq i}\pi'_{ij}$ (Section 8.1): a bullet ballot gives its first choice $C-1$ points and every other candidate the mean $(C-2)/2$ of the points $C-2, \dots, 0$ of the positions they share. Baldwin and Nanson take the same sum over the remaining candidates. On a score ballot the part of the top score that a bullet voter does not give c_i is 1 unless c_i is their first choice, so the part not given of Score (Section 3.8) becomes $(1-b)u_i + b\sum_{j\neq i}s_j$, and the strict score comparisons of STAR $(1-b)x_{ij} + b\big(s_i-s_j\big)$. The shares of every score of majority judgment (Section 3.10) become $(1-b)w_i(k)$, with bs_i more at the top score and $b\sum_{j\neq i}s_j$ more at 0. These need the first-choice shares next to the pairwise or score shares; `Bullet` adds them to what the method needs (Section 8.1).

The methods on the whole profile keep the rankings with $(1-b)P(r)$, and the bullet ballots with bs_k for every first choice c_k, taken from the profile itself.

- **IRV** (and every schedule of Section 3.11). A bullet ballot counts for its first choice while that remains. Once it is out, the ballot counts $1/|\mathcal{R}|$ for each of the remaining candidates $\mathcal{R}$. That adds the same to every remaining tally, so it changes no elimination and no gap, and the compiled rounds leave it out: the ballot acts as if it were exhausted.
- **DAC**. A bullet ballot for c_k ranks no candidate outside a set above one in it exactly when the set holds c_k, so it acquiesces to every such set, and (Equation 41)

$$a'_T = (1-b)a_T + b\sum_{k\in T}s_k. \quad (52)$$

The compiled loop adds these sums for all 2^C sets per pixel, each set's from that of the set without its lowest candidate.

Equal places and truncation. Leaving the others off the ballot would give the same winners everywhere but in Borda count and the methods built on it: Baldwin, Nanson and Black where there is no Condorcet winner. Head to head, unranked candidates abstain as equal ones do; in IRV the equal split changes nothing, as shown above; on a score ballot an unranked candidate gets 0 as well; and DAC counts a truncated ballot by the same rule of acquiescing. In Borda count a truncated ballot usually gives the others 0 points, here they share $(C-2)/2$ each, so a bullet voter's first choice gains less on them.

All voters. At $b = 1$ every method elects the candidate with the most first choices, the FPTP winner:

- by Equation 51 it beats every other candidate head to head, so the Condorcet methods, the runoffs of the two-round system and STAR, and king of the hill elect it;
- its Borda score $(C-1)/2 + (Cs_i - 1)/2$ and its mean score are the highest;
- IRV and every schedule drop the fewest first choices, never the most;
- in DAC the set of all eligible candidates but the one with the fewest first choices has the highest score among those that cut them, round after round.

The tests check this for all fifteen methods.

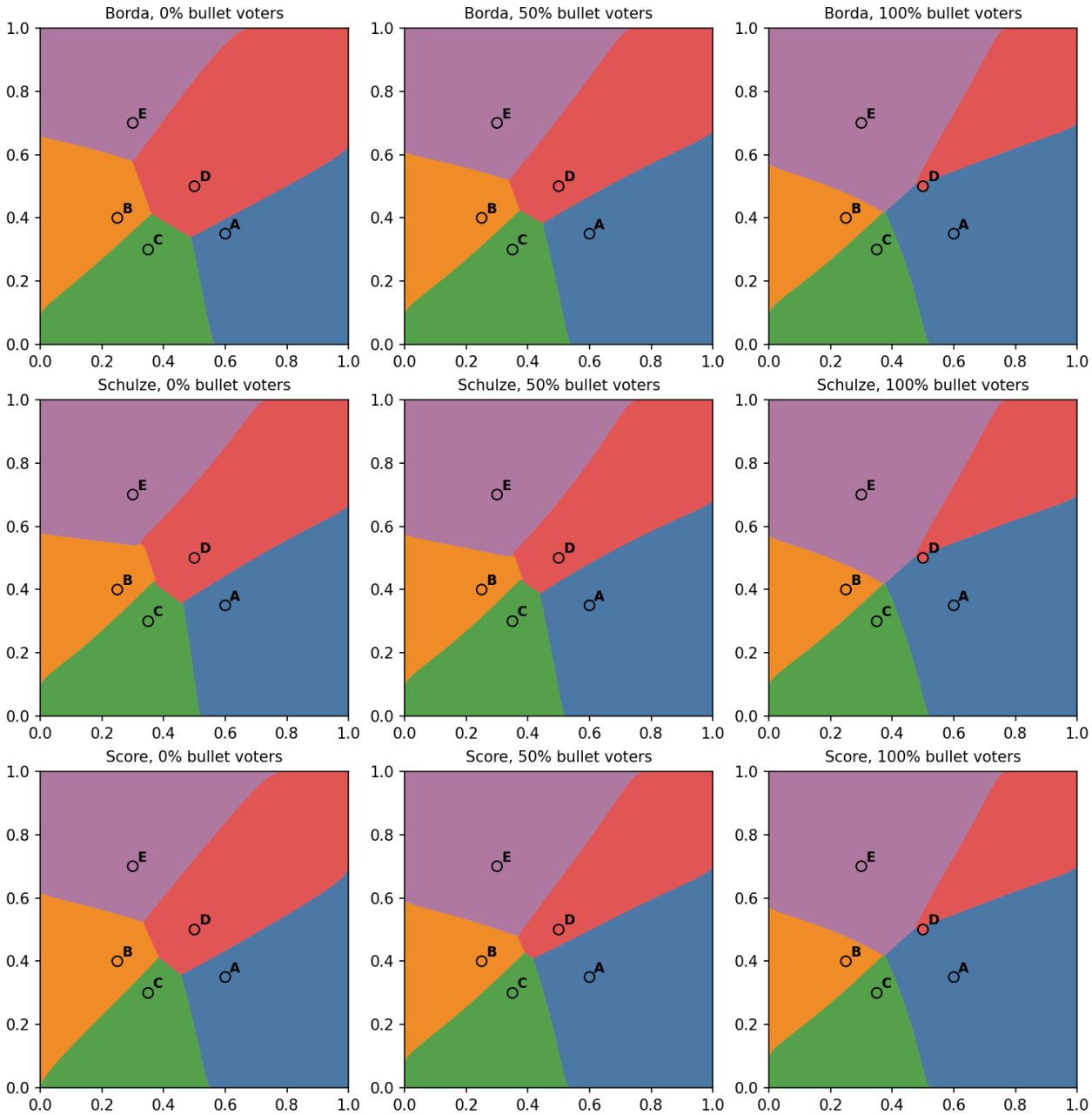


Figure 6: Borda count, Schulze and Score (six levels, $p = 1.5$) with no, half and all voters bullet voting (candidates A–E, Beta voters, `rms`, $D = 0.2$). At $b = 1$ all three are the FPTP diagram.

The default candidates. Bullet voting takes the second preferences out of the ballots, and the second preferences are what a centrist wins on. In Figure 6 the region of D, in the middle of A–E, shrinks from 31% of the square to 23% (Borda count), 25% to 21% (Schulze) and 24% to 19% (Score) at $b = 0.5$, and to the 14% of FPTP at $b = 1$. The part of the square with another winner than with sincere voters:

method	$b = 0.25$	$b = 0.5$
FPTP	0	0
Two-round	2.1%	4.7%
Baldwin, Nanson, Schulze, Minimax, KOTH	2.2%	4.9%
Condorcet, Black	2.3%	5.0%
STAR	2.4%	5.2%
IRV	3.3%	6.4%
IRV set + KOTH + Score	3.5%	7.1%
DAC	3.3%	7.6%
Score	4.5%	8.3%
Borda	5.4%	10.6%

FPTP reads only first choices, which bullet voting keeps. Borda count changes the most: it is the method that rewards the second place the most. DAC goes the other way: three quarters of the pixels that change at $b = 0.5$ go to D, whose region grows from 16% to 21%, before it falls to the 14% of FPTP at $b = 1$. A bullet ballot acquiesces to every set that holds its first choice, which gives the large sets with D in them the support of every bullet voter of their members.

4.2. Burial (`Burial`)

The ballot. Burial needs a rival to bury. At every pixel a poll names two frontrunners, by default the two with the most first choices, `Highest(Tally(Plurality()), n=2)`, the same for every method. A burying voter ranks the one of them they like less last, and on a score ballot gives it 0; the rest of the ballot stays as it is. No one buries their first choice, so the first choices do not change, and neither does the head-to-head of the two frontrunners: whoever ranks a above b still does. With q the share of the voters who bury and a, b the frontrunners of the pixel:

- **Profile.** Every ranking comes twice: as it is, with $(1 - q)P(r)$, and with the one of a and b it ranks lower moved last, with $qP(r)$. IRV, DAC and the schedules of Section 3.11 run on these $2R$ rankings.
- **Pairwise shares.** $\pi'_{ij} = \pi_{ij} + q\sum_r P(r)([i \text{ above } j \text{ in } r'] - [i \text{ above } j \text{ in } r])$, with r' the buried ranking. Only the pairs of a or b with a third candidate change. The sum needs the profile, so burial makes the pairwise methods read it (Section 8.1).
- **Score shares.** With t_{ij} the part of the top score that the voters who rank c_j above c_i give c_i, the part not given (Section 3.8) becomes $u_a + qt_{ab}$ for a and $u_b + qt_{ba}$ for b, and stays for the others: a burying voter takes back what they gave the frontrunner they bury. t is summed over the voter grid like u; a cell counts as one ballot only if its corners also rank the candidates in the same order, so that no bisector crosses it. Only t_{ab} and t_{ba} of the frontrunners of a pixel are needed there, so they are computed for each pair of frontrunners, at its pixels only. STAR's runoff reads the score comparisons, which take more (Section 4.4). Of the shares of every score of majority judgment (Section 3.10), q of those of the voters who rank c_b above c_a go from their score of c_a to 0, and likewise for c_b: the share of those voters at each score of the frontrunners is summed over the voter grid like t.

Poll lines. Where the second and the third most first choices tie, the frontrunners change, and with them whom the voters bury, so the winner can jump although every share is smooth. The method runs on the pixels of each pair of frontrunners apart, and the poll's margin, $s_{(2)} - s_{(3)}$ in the order of the first-choice shares, is a gap like any other: the margin is the smaller of the method's and the poll's. Like that of IRV, it also vanishes where the frontrunners change but the winner stays.

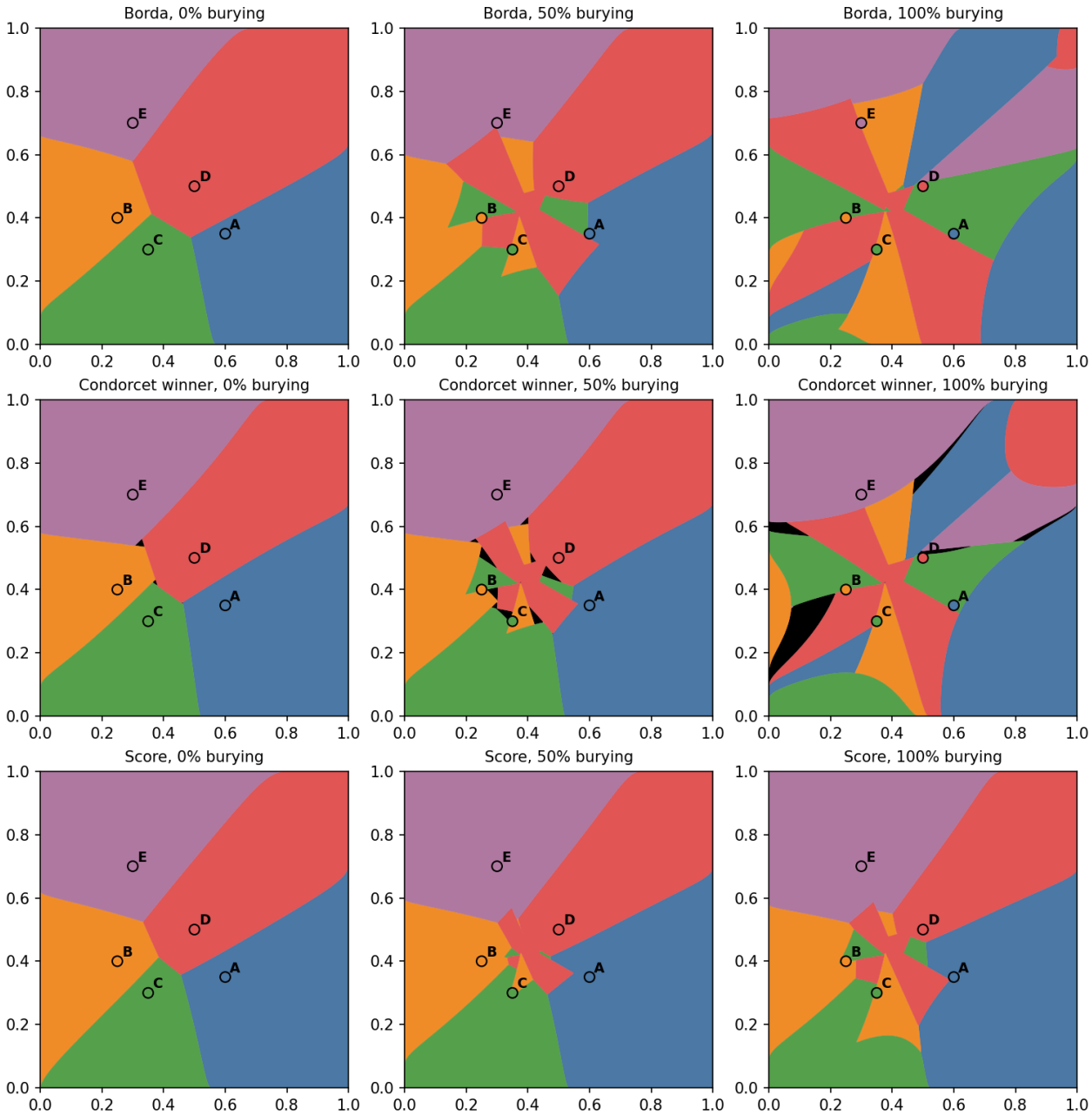


Figure 7: Borda count, the Condorcet winner and Score (six levels, $p = 1.5$) with no, half and all voters burying (candidates A–E, Beta voters, `rms`, $D = 0.2$). Black: no Condorcet winner.

The default candidates. FPTP and the two-round system do not change at all: the first choices stay, and the two-round finalists are the frontrunners, whose head-to-head burial keeps. IRV hardly changes: a ballot reaches the buried candidate only once all candidates above it are out. Elsewhere burial backfires. Where the supporters of each frontrunner bury the other, a third candidate, a dark horse, gains on both: in the bands of Figure 7 that run out of the middle of the square, the frontrunners are the two candidates on either side and the winner is a third. With Borda count a dark horse wins 13% of the square at $q = 0.5$ and 70% at $q = 1$, against 2% with sincere voters. Burial also makes Condorcet cycles: the Condorcet winner is missing on 0.6% of the square at $q = 0.5$ and 3.2% at $q = 1$. A buried candidate loses only what the burying voters gave it on a score ballot, so Score changes less than the ranked methods with many burying voters. The part of the square with another winner than with sincere voters:

method	$q = 0.25$	$q = 0.5$	$q = 1$
FPTP, Two-round	0	0	0
IRV	0.5%	1.4%	3.3%
KOTH	0.8%	3.4%	33%
IRV set + KOTH + Score	2.2%	4.7%	11%
Baldwin, Nanson, Schulze, Minimax	1.5%	5.5%	43–44%
Condorcet, Black	1.5–1.6%	5.9%	45%
DAC	2.2%	6.2%	27%
Score	3.4%	6.3%	14%
STAR	1.4%	3.8%	15%
Borda	7.6%	17%	69%

4.3. Compromise (`Compromise`)

The ballot. A compromising voter whose first choice is neither frontrunner ranks the one of them they like more first, and on a score ballot gives it the top score; the rest of the ballot stays. A voter with a frontrunner first ranks it first already and changes nothing. The frontrunners are those of the poll of Section 4.2, so the method runs on the pixels of each pair apart, with the poll's margin as a gap. With y the share of the voters who compromise:

- **Profile.** Every ranking comes twice: as it is, with $(1 - y)P(r)$, and compromised, with $yP(r)$.
- **First choices.** Unlike with burial, they change. The voters who rank a above b are those with a first and those with neither first who like a more, so y of the latter move their first choice to a:

$$s'_a = s_a + y(\pi_{ab} - s_a), \quad s'_b = s_b + y(\pi_{ba} - s_b), \quad s'_i = (1 - y)s_i \text{ for the others.} \tag{53}$$

  The shares come from the profile, so compromise makes every ranked method read it, FPTP too.
- **Pairwise shares.** From the profile, as for burial; the head-to-head of the frontrunners stays.
- **Score shares.** With g_{ij} the part of the top score that the voters who rank c_i above c_j, but neither first, do not give c_i, the part not given becomes $u_a - yg_{ab}$ for a and $u_b - yg_{ba}$ for b. g is summed over the voter grid like t, for each pair of frontrunners (Section 4.2); STAR's comparisons in Section 4.4. Of the shares of every score of majority judgment, y of those of the voters who rank c_a above c_b, neither first, go from their score of c_a to the top score, and likewise for c_b.

Order. A poll reads the first choices of the voters it is given, and compromise changes them, so it runs inside the other strategies: in `Bullet(Burial(Compromise(method, y), q), b)` every poll reads the sincere first choices. Burying and compromising move different candidates, so a voter can do both, and qy of the voters who do not bullet vote do: they rank one frontrunner first and the other last.

Empty tallies. Where all voters compromise, no voter has a third candidate first, and IRV's first rounds drop candidates with empty tallies. Two empty tallies tie, but that decides nothing: such a candidate has no ballots to pass on, and gets none, since it goes before any candidate with ballots, so their order changes no winner. The compiled rounds of IRV and of the schedules leave these ties out of the margin, which would otherwise be 0 on the whole square.

All voters. At $y = 1$ every first choice is a frontrunner, so FPTP elects the one of them more voters rank above the other, the winner of the two-round system, and so does IRV once the empty tallies are gone. The two-round system itself never changes: compromise only moves first choices to the frontrunners, who are its finalists already, and keeps their head-to-head. That is a limit of the model: the poll is exact, so no voter fears that the frontrunner they like more misses the runoff.

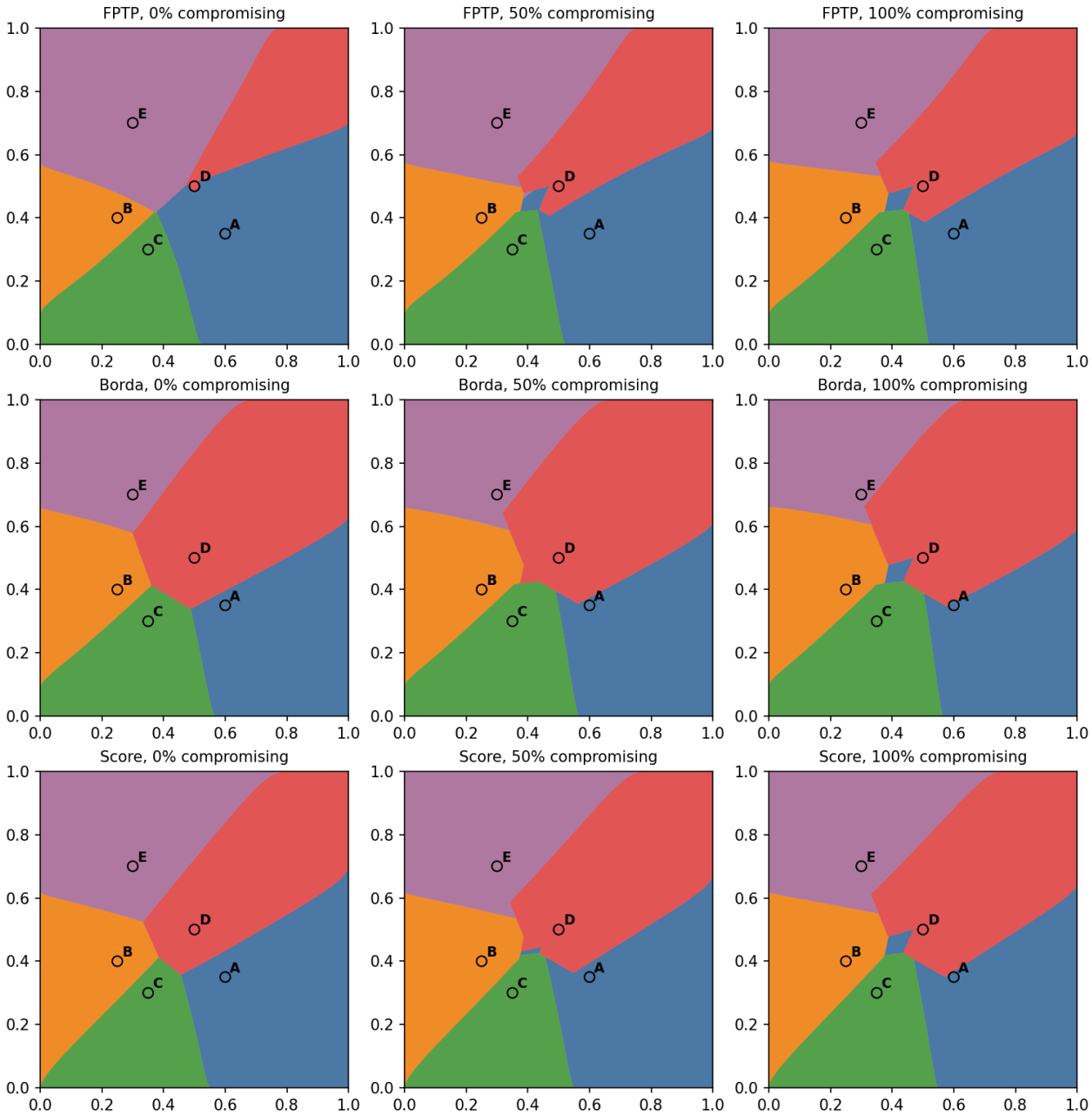


Figure 8: FPTP, Borda count and Score (six levels, $p = 1.5$) with no, half and all voters compromising (candidates A–E, Beta voters, `rms`, $D = 0.2$). At $y = 1$ FPTP is the two-round system.

The default candidates. Compromise takes first choices from the candidates outside the poll, the opposite of a spoiler, and FPTP changes the most. The centrist D, a frontrunner at many pixels, wins 14% of the square with sincere voters, 20% at $y = 0.5$ and 23.5% at $y = 1$, as in the two-round system (Figure 8). IRV changes only at the 1.9% of the square where its sincere winner is not a frontrunner, and the Condorcet methods hardly at all: they do not count first places. The part of the square with another winner than with sincere voters:

method	$y = 0.25$	$y = 0.5$	$y = 1$
Two-round	0	0	0
KOTH	0.4%	0.5%	0.5%
STAR	0.5%	0.7%	1.2%
Baldwin, Nanson, Schulze, Condorcet, Minimax, Black	0.7%	0.8%	1.2%
IRV set + KOTH + Score	1.7%	2.2%	3.4%
IRV	1.9%	1.9%	1.9%
Score	2.4%	4.2%	7.1%
Borda	2.7%	4.0%	6.6%
DAC	3.8%	6.5%	11%
FPTP	4.8%	8.3%	13%

4.4. STAR's runoff

STAR's runoff reads the score comparisons x_{ij}, the share of the voters who score c_i above c_j minus that who score c_j above c_i (Section 3.8). The runoff tie weight w adds w times the ordinal margin among voters who gave the pair the same score. Thus $w = 0$ is STAR, where they abstain, and $w = 1$ is an ordinary pairwise runoff. These ordinal comparisons must be computed from the same individual ballots as the score ties; their separate aggregate totals do not say which tied-score voters prefer which candidate. Burying and compromising move only the comparisons of a frontrunner with another candidate, but unlike the scores these do not add up over the strategies: a voter who buries b and compromises on a scores a above b whatever they did before, which neither change alone tells. So the comparisons are counted for each kind of voter on its own: of the voters who do not bullet vote, $(1 - q)(1 - y)$ vote sincerely, $q(1 - y)$ bury, $(1 - q)y$ compromise and qy do both, and each moves the comparisons by its own ballots,

$$x'_{ij} = x_{ij} + (1 - b) \sum_{\text{kind}} w_{\text{kind}} \sum_{k,l} \mu_k \left(m^{\text{kind}}_{ijkl} \right) \nu_l, \tag{54}$$

with m^{kind}_{ijkl} how much the comparisons of the ballots in rectangle (k, l) move (`moved_comparisons` in `score.py`), summed over the ballot table of Section 3.8. These depend on the pair of frontrunners and on q and y, so they are not shares the method gets beforehand: burying and compromising only note them (`Voters.compared`), and STAR's runoff computes them, for its pair of frontrunners and at its pixels only, when it reads them. With eight random candidates, whose pixels have twelve pairs of frontrunners, that takes about 0.25 s of the 0.9 s of a drag step with burying voters.

The default candidates. A burying voter ranks the frontrunner they like more above the one they bury already, so where the two frontrunners meet in the runoff only the voters who gave them the same score change their vote: STAR changes less than Score at $q = 0.5$, 3.8% of the square against 6.3%. Where a buried frontrunner meets a third candidate in the runoff instead, burial backfires: the voters who buried it now score that candidate above it. The centrist D gains: it wins 24% of the square with sincere voters, 26% at $q = 0.5$ and 28% at $q = 1$. Compromise hardly changes STAR, 0.7% of the square at $y = 0.5$: it gives the frontrunner a voter likes more the top score, which moves it up in the totals, but where the two frontrunners meet in the runoff that voter preferred it already.

5. Beta versus normal voters

With the same candidates the two voter models give visibly different diagrams, and the difference is largest where one would expect it least: for Condorcet methods. On votingmethods.net/yee (normal voters) the River method splits the square into one clean region per candidate, while Schulze on the Beta model has distorted regions and even pixels without a Condorcet winner (Figure 9). This chapter explains why.

All numbers below use the candidates A–E (Figure 2), the default spread rule `rms`, 300×300 pixels, 49×49 nodes and $D = 0.3$. This is more than the default 0.2 so that the effects are easy to see; at 0.2 they are the same in kind but smaller (Section 5.2.5). The numbers and figures are produced by `docs/figures.py`. How the other spread rule changes them is the subject of Chapter 6.

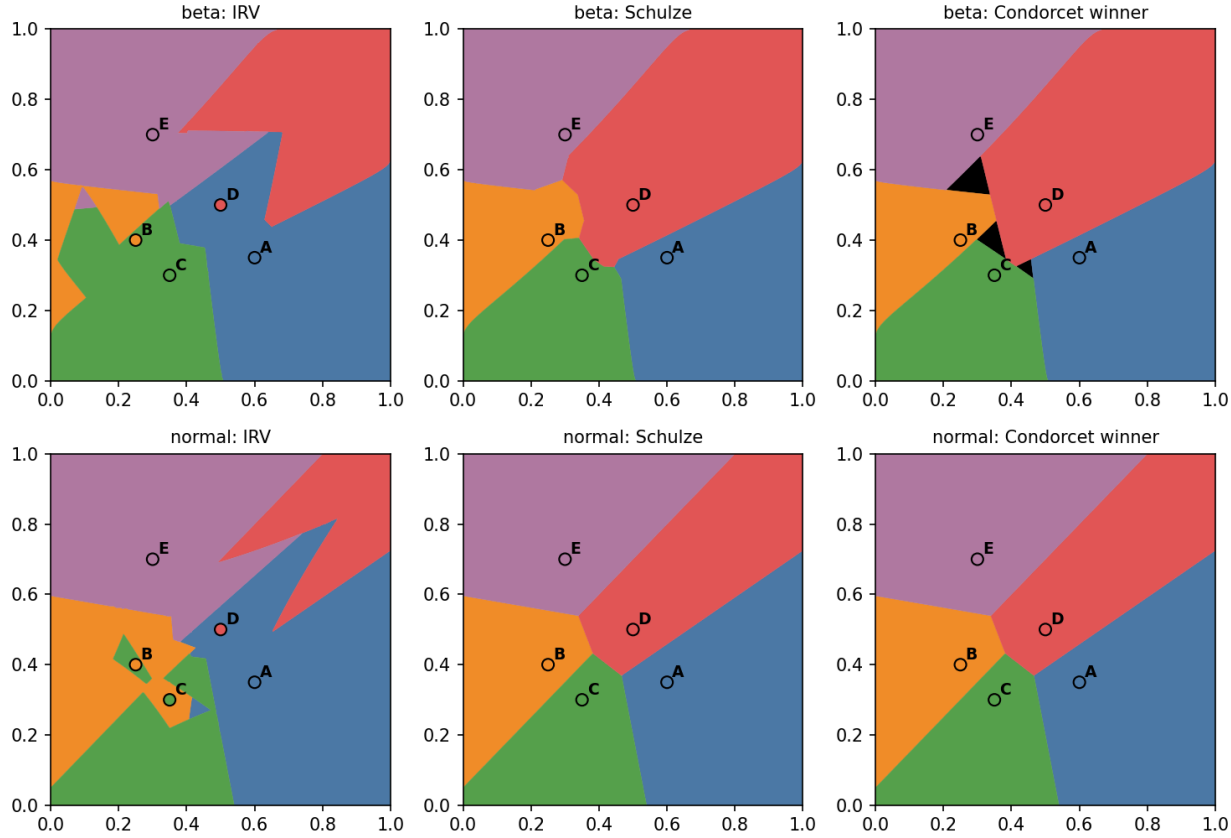


Figure 9: Beta voters (`rms`, top) and normal voters (bottom), $D = 0.3$. Normal Schulze and the normal Condorcet winner are exactly the Voronoi diagram of the candidates. Black: no Condorcet winner. The dotted black lines in the bottom right plot are pixels whose centre lies exactly on a bisector (a tie, not a cycle).

5.1. The two models

	Beta (`ranking_cells.py`)	**normal** (`normal.py`)
voters of pixel m	$X \sim \mathrm{Beta}(a_x, b_x)$, $Y \sim \mathrm{Beta}(a_y, b_y)$ independent	$(X, Y) \sim \mathcal{N}(m, \sigma^2 I)$
support	unit square	whole plane
m is	the median of each coordinate	mean and median, centre of symmetry
spread	$E\lvert X - \frac{1}{2}\rvert = D$ at the centre pixel, RMS from the median fixed (`rms`)	$E\lvert X - m_x\rvert = D$ everywhere, Equation 3
symmetric about m	only for $m = (\frac{1}{2}, \frac{1}{2})$	always

votingmethods.net uses a standard deviation of 0.4 by default ($D \approx 0.319$ here) and samples only 140 voters per pixel, so its diagrams also carry Monte Carlo noise (ragged IRV borders, small islands). The shares here are exact.

5.2. Pairwise majorities

By Equation 13 a voter at p prefers c_i to c_j exactly when $p \cdot \nu < \omega$ with $\nu = c_j - c_i$ and $\omega = \big(|c_j|^2 - |c_i|^2\big)/2$. The pairwise share (Equation 36) of the voters of pixel m who prefer c_i to c_j is therefore

$$\pi_{ij}(m) = P_m(X \cdot \nu < \omega), \tag{55}$$

and it depends only on the one-dimensional projection $X \cdot \nu$ of the voters onto the direction from c_i to c_j. Every Condorcet method only looks at these numbers.

5.2.1. Normal voters: the majority is decided by distance

The projection of an isotropic normal is normal, $X \cdot \nu \sim \mathcal{N}(m \cdot \nu, \sigma^2\ |\nu|^2)$, so

$$\pi_{ij}(m) = \Phi\left(\frac{\delta_{ij}(m)}{\sigma}\right), \quad \delta_{ij}(m) = \frac{\omega - m \cdot \nu}{|\nu|}, \tag{56}$$

where $\delta_{ij}(m)$ is the signed distance of m from the bisector, positive on the side of c_i. A majority prefers c_i to c_j if and only if $\delta_{ij}(m) > 0$, that is, if and only if $|m - c_i| < |m - c_j|$. Consequently:

1. The majority relation of every pixel is the order of the candidates by distance from m. It is transitive: **there are no Condorcet cycles.**
2. The Condorcet winner always exists and is the candidate nearest to m, so **every** Condorcet method (Schulze, River, Ranked Pairs, Minimax, …) draws exactly the Voronoi diagram of the candidates.
3. This holds **for every σ**. The only property used is that every line through m has half of the voters on each side, which is true for every distribution that is centrally symmetric about m.

This is why the River diagram of votingmethods.net looks like a Voronoi diagram; its borders agree with the bisectors (e.g. the D|A bisector meets $x = 1$ at $y = 0.725$ and the D|E bisector meets $y = 1$ at $x = 0.8$). The spread σ still matters for methods that use more than pairwise majorities (FPTP, IRV, Borda).

5.2.2. Beta voters: the median is a median only along the axes

For the Beta model m is the median of X and of Y separately. If ν is parallel to an axis, $X \cdot \nu$ is a multiple of one coordinate, its median is $m \cdot \nu$, and the line through m perpendicular to ν splits the voters exactly in half. For an oblique ν this fails: the median of $\nu_x X + \nu_y Y$ is in general **not** $\nu_x m_x + \nu_y m_y$, because medians are not additive for skewed variables. For every direction ν there is a different "effective centre" of the pixel, shifted from m towards the mean of the voters. Consequently:

1. The Condorcet winner is no longer the candidate nearest to m, so Condorcet regions are distorted.
2. Different pairs of candidates are compared from different effective centres, so the majority relation need not be transitive: **Condorcet cycles appear** (black in Figure 9). If some point were a median in every direction (a "total median", Davis, DeGroot and Hinich, 1972), the argument of the normal case would apply with that point in place of m; a product of skewed Beta marginals generally has none.

Chapter 9 draws every pixel at the geometric median of its voters instead of at m, which takes back part of this shift for every direction at once.

5.2.3. The shape of Beta voters

At $D = 0.3 > \frac{1}{4}$ the Beta is U-shaped even at the centre (Section 1.4). Towards a wall:

median	a	b	mean	$P(X < 0.1)$	$P(X > 0.9)$
0.50	0.602	0.602	0.500	0.175	0.175
0.70	0.743	0.454	0.621	0.095	0.298
0.90	0.796	0.265	0.750	0.051	0.500
0.98	0.648	0.151	0.812	0.049	0.640

As the median approaches the wall, half of the voters crowd into the strip between the median and the wall, while the other half forms a long tail towards the centre of the square. The mean lags behind the median by up

to 0.17: the voters of every pixel off the centre are skewed towards the middle of the square, and so are their effective centres.

5.2.4. Example: the pull towards the centre

The pixels below lie on A's side of the bisector of A $= (0.6, 0.35)$ and the centre candidate D $= (0.5, 0.5)$; δ is their distance from it. Share of voters preferring A (Beta: $4 \cdot 10^6$ sampled voters):

pixel m	δ	normal	Beta	Beta mean
(0.95, 0.55)	0.118	0.623	0.525	(0.786, 0.530)
(0.90, 0.60)	0.049	0.551	**0.473**	(0.750, 0.560)
(0.80, 0.45)	0.118	0.623	0.541	(0.684, 0.470)

In all three pixels A gets far less support than under normal voters, and in the second the median voter is closer to A, yet D wins the pairwise contest: the long tails of the Beta point towards the middle of the square, and so does the effective centre.

5.2.5. The whole diagram

Leaving out the 357 pixels whose centre lies exactly on a bisector (where both candidates are correct winners):

	Schulze $\neq$ Voronoi	no Condorcet winner	area of D (Voronoi: 0.208)
Beta, $D = 0.3$	12 705 pixels (14.2 %)	1 101 pixels (1.23 %)	0.312
Beta, $D = 0.2$	4 980 pixels (5.6 %)	48 pixels (0.05 %)	0.247
normal, any D	0	0	0.208

The centre candidate D gains most: the Beta electorates of the whole upper right part of the square are pulled towards the centre, where D is.

5.3. Curved borders

Under normal voters the borders of Condorcet methods are bisectors, i.e. straight lines. The borders of FPTP and IRV are curved in both models, and the Beta model adds bends near the walls of the square. The key is to look at a first-choice share as a function of the pixel: it is the Voronoi cell of the candidate, **blurred**.

5.3.1. First-choice shares are blurred Voronoi cells

The FPTP winner changes where two first-choice shares (Equation 33) are equal,

$$s_i(m) = s_j(m), \quad s_i(m) = P_m(X \in V_i) = \int_{V_i} f_m(p)\,\mathrm{d}p, \tag{57}$$

with V_i the Voronoi cell of c_i and f_m the voter density of pixel m. For normal voters $f_m(p) = \varphi_\sigma(p - m)$ is one Gaussian bump moved to m, and since it is symmetric,

$$s_i = \mathbb{1}_{V_i} * \varphi_\sigma, \tag{58}$$

the indicator of the cell (1 inside, 0 outside) convolved with a Gaussian of radius σ: the cell **blurred** like an out-of-focus photograph. The FPTP diagram is obtained by blurring every Voronoi cell and colouring each pixel by the brightest one; for $\sigma \to 0$ this is the Voronoi diagram itself. IRV does the same once per round, with the cells V_i^S of the candidates S still in the race. A border is therefore a **contour line** where two blurred cells are equally bright, and the question is when such a contour line is straight.

For Beta voters there is no single bump: f_m changes its shape with m (U-shaped, crowded against the walls, Figure 1 and Figure 13), so the blur itself varies across the square. This is the source of the bends near the walls below.

5.3.2. Straight versus round: blurring a corner

Half-plane: straight. With two candidates each cell is a half-plane, and by Equation 56 its blurred version is $\Phi(\delta/\sigma)$, a function of the distance δ from the bisector only. All its contour lines are parallel to the bisector, and the tie $s_i = s_j = \frac{1}{2}$ is the bisector itself.

Corner: round. Let a cell be the quadrant $Q = \left\{x < \frac{1}{2}, y < \frac{1}{2}\right\}$. The two coordinates of the normal are independent, so its blurred version is exactly

$$s_Q(m) = \Phi\left(\frac{\frac{1}{2} - m_x}{\sigma}\right) \Phi\left(\frac{\frac{1}{2} - m_y}{\sigma}\right). \tag{59}$$

Far from the corner one factor is 1 and the contour lines are straight lines parallel to the edges of Q. At the corner they bend round (Figure 10, middle). For example the contour $s_Q = \frac{1}{2}$ approaches the edges of Q far away, but on the diagonal it needs $\Phi(u)^2 = \frac{1}{2}$, $u = \Phi^{-1}\left(1/\sqrt{2}\right) = 0.545$: it passes $0.545\sqrt{2}\sigma \approx 0.77\sigma$ **inside** the corner. Blurring rounds the corner with a radius of the order of σ, exactly as blurring a photograph of a square gives a rounded square.

Vertices: the widest cell wins. A cell that is a wedge with angle α at a Voronoi vertex receives exactly $\alpha/(2\pi)$ of the voters of the pixel centred at that vertex (the Gaussian is rotationally symmetric). Where several cells meet, the one with the largest angle wins at the vertex, and the fronts of the others recede by a distance of the order of σ along round contour lines. Far from the vertices (distance $\gg \sigma$) only two cells are nearby and the border returns to their bisector.

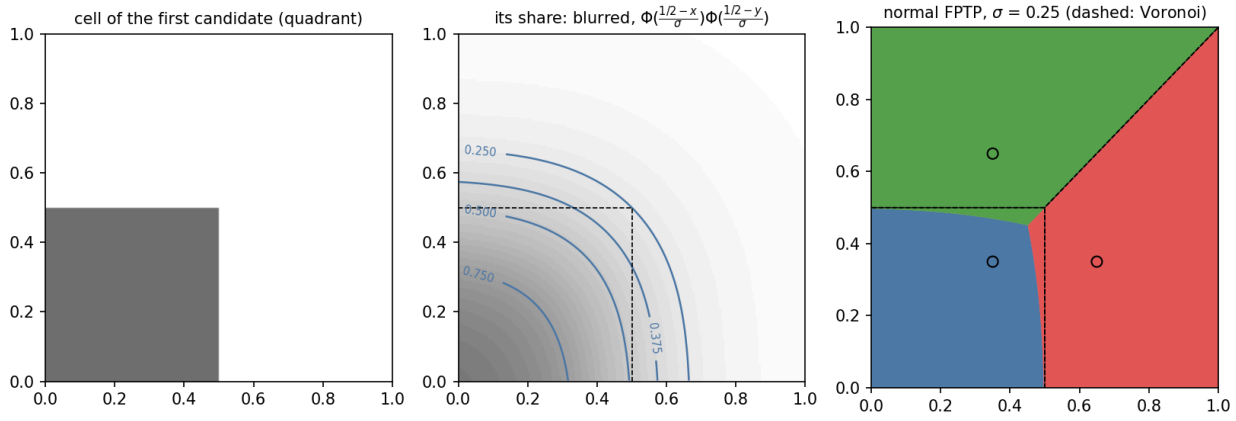


Figure 10: Three candidates whose cells meet at $\left(\frac{1}{2}, \frac{1}{2}\right)$ with angles 90°, 135°, 135° (normal voters, $\sigma = 0.25$). Left: the quadrant cell of the lower-left candidate. Middle: its share (Equation 59) with contour lines. Right: FPTP. At the vertex the shares are $\frac{1}{4}$, $\frac{3}{8}$, $\frac{3}{8}$, so the quadrant candidate loses its corner, along a round front.

5.3.3. Seven candidates

Figure 11 shows FPTP for seven candidates (B removed, three added on the left). With a small blur ($D = 0.1$) the diagram is the Voronoi diagram with rounded corners at the vertices. With $D = 0.3$ the blur is wider than several cells:

- the centre candidate $(0.5, 0.5)$ has a thin band as its cell. Blurred, a thin band stays dim, so under normal voters it wins only a small patch in the far corner of its band, not even its own position;
- borders appear between candidates whose cells do not even touch, e.g. $(0.3, 0.7)$ and $(0.6, 0.35)$. Such a border has no bisector to follow at all: it is purely a contour line of two blurred cells, and it is curved.

The Beta diagram shows the same effects, and in addition the blur changes from pixel to pixel: the centre candidate, helped by the pull towards the centre, wins a large region in the upper right, and the borders bend where the shape of the voters changes fastest, near the walls.

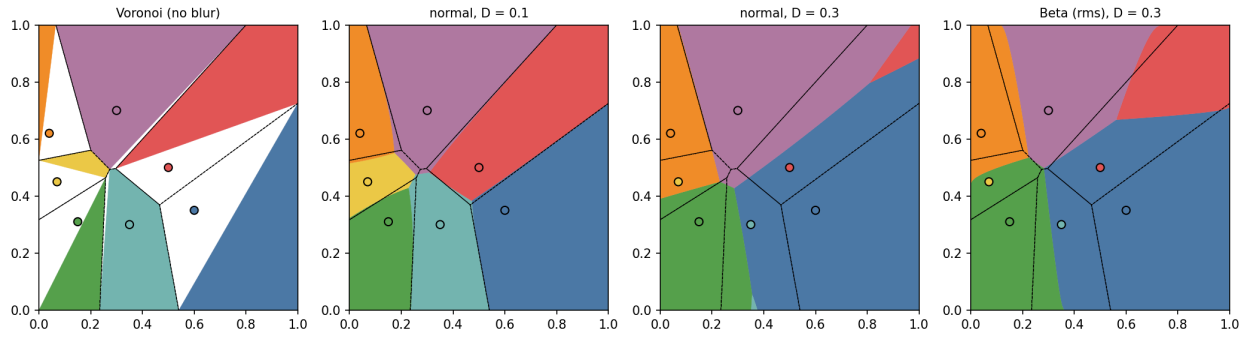


Figure 11: FPTP for seven candidates; dashed: Voronoi borders. From left: no blur (Voronoi), normal voters with $D = 0.1$ and $D = 0.3$, Beta voters (`rms`) with $D = 0.3$.

5.3.4. Flares at the walls (FPTP)

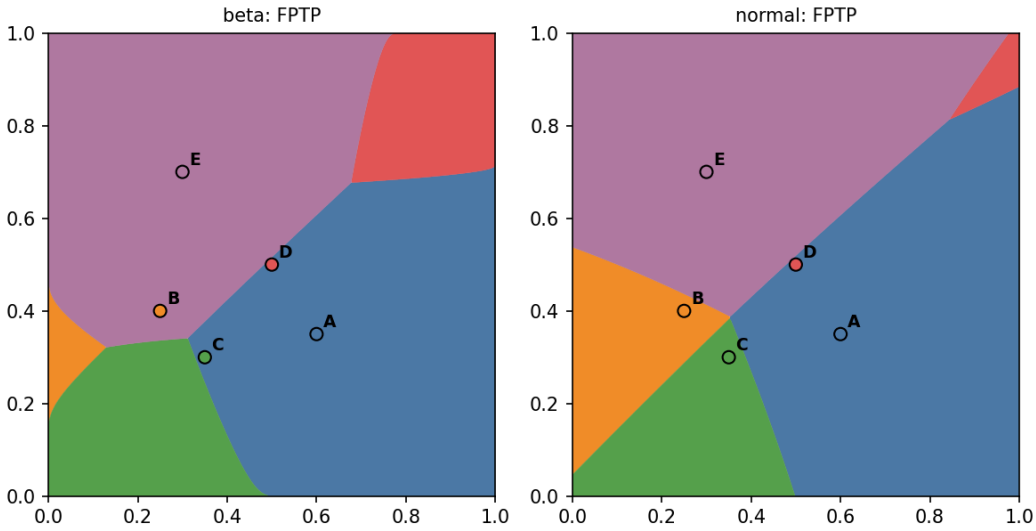


Figure 12: FPTP. With Beta voters the region of B flares out in the last few hundredths before the left wall; with normal voters it widens smoothly.

B $= (0.25, 0.4)$ is the leftmost candidate and its Voronoi cell widens towards the left wall: the bisector B|C ($y = x + 0.05$) falls as x decreases, and at $x = 0$ the cell spans $0.05 < y < 0.596$. The more voters sit close to $x = 0$, the more of them fall into the wide part of B's cell. Heights y of the borders of B's region in the column of median x ("–": B wins nowhere in the column):

median x	$P(X < 0.02)$	Beta B|E	Beta C|B	normal B|E	normal C|B
0.200	0.212	–	–	0.455	0.238
0.080	0.357	0.355	0.272	0.505	0.125
0.030	0.467	0.398	0.218	0.525	0.075
0.005	0.581	0.438	0.172	0.535	0.048

With Beta voters B's region only begins near the wall and then widens fast: over the last 0.075 the B|E border rises by 0.083, against 0.030 with normal voters. Figure 13 shows why. As the median approaches the wall, the half of the Beta voters on the wall side is squeezed into an ever thinner strip next to the wall, right into the wide part of B's cell ($P(X < 0.02)$ grows from 0.21 to 0.58), and B's share grows from 0.22 to 0.33 between $x = 0.2$ and $x = 0.02$. The normal cloud keeps its shape and only moves; its left half continues beyond $x = 0$, where B's cell simply keeps widening, so nothing piles up at the wall.

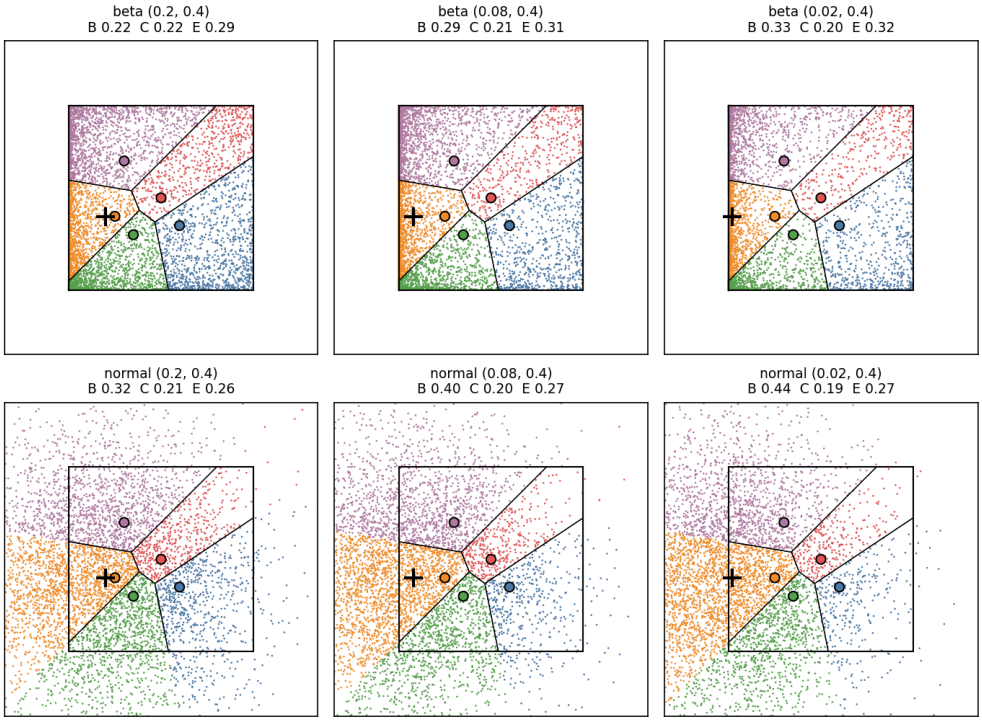


Figure 13: Voters of the pixels $(x, 0.4)$ approaching the left wall, coloured by their first choice (FPTP), with the Voronoi borders; titles: shares of B, C and E.

The flare is a property of any wide Beta, not of the spread rule: it is nearly the same under `mean_abs` (Chapter 6). It fades with less spread: at $D = 0.2$ the B|E border rises by only 0.030, like normal voters at $D = 0.3$.

5.4. Summary

- Normal voters are symmetric about the pixel centre, so pairwise majorities follow distance: no cycles, and every Condorcet method draws the Voronoi diagram, whatever σ.
- Beta voters have the pixel centre as the median of each coordinate only. Majorities between candidates on a diagonal are decided by the skew of the distribution, which pulls the effective centre towards the middle of the square: Condorcet regions are distorted (14 % of the pixels at $D = 0.3$, 6 % at $D = 0.2$) and cycles appear.
- A first-choice share is the candidate's Voronoi cell blurred by the voter distribution. Blurring keeps half-planes straight but rounds corners (radius of order σ), so FPTP and IRV borders are curved in both models.
- Beta voters change shape across the square and crowd against the walls, so the blur varies from pixel to pixel; near the walls this produces flares such as B's region in FPTP.

6. Choosing the spread rule

Section 1.5 defined two spread rules. They agree at the centre pixel and differ only in how the spread changes towards the walls. This chapter shows what that changes in the diagrams, explains which property of a rule keeps borders straight, and gives the evidence for the default `rms`. Unless stated otherwise the numbers use the candidates A–E, $D = 0.3$ and 300×300 pixels. `docs/figures.py` reproduces the figures and the numbers on the default candidates; the benchmark over random candidate layouts (Section 6.4) was run with separate scripts.

6.1. The two rules on the default candidates

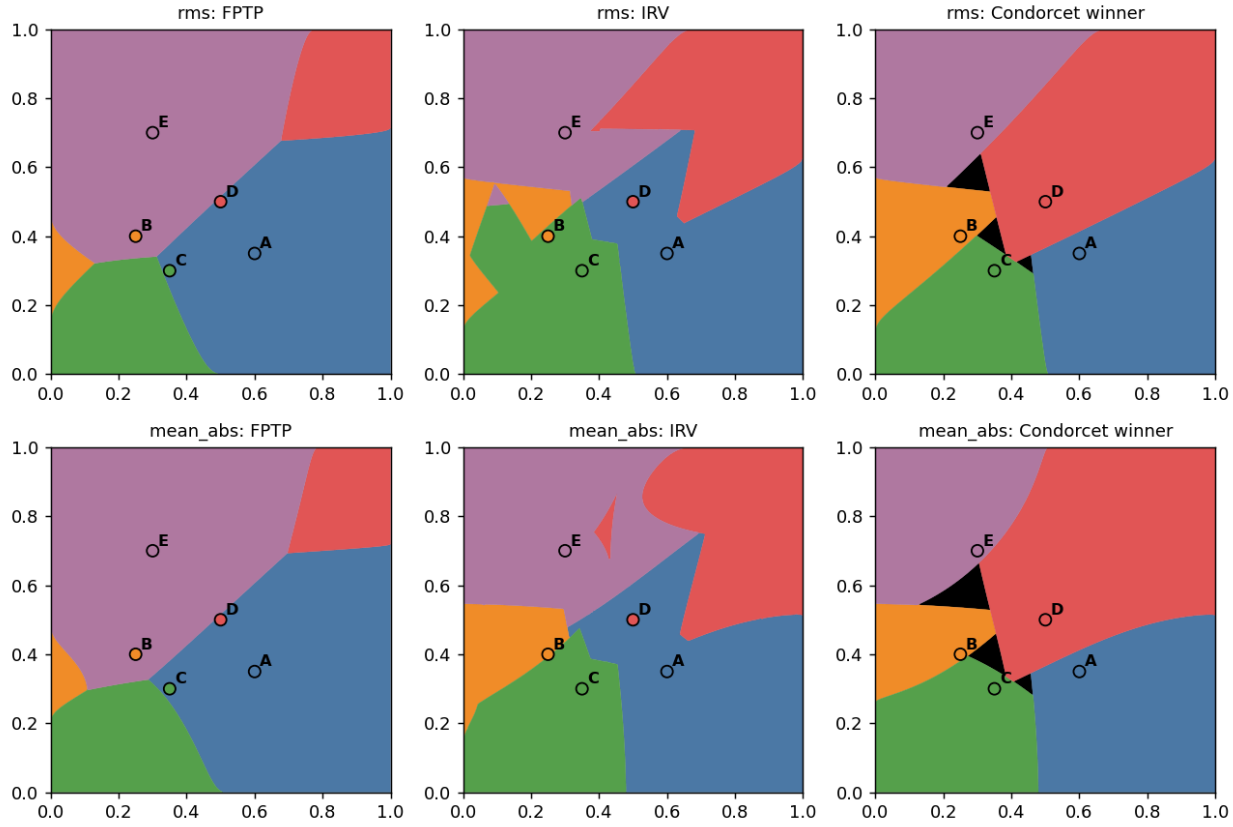


Figure 14: The two spread rules of Section 1.5 on the default candidates, $D = 0.3$. Only `mean_abs` has the round IRV edge. Black: no Condorcet winner.

voter model	cycle pixels	Schulze ≠ Voronoi	round IRV edge	flare of B
Beta, `rms`	1.23 %	14.2 %	no	0.083
Beta, `mean_abs`	2.06 %	20.2 %	yes	0.096
normal	0	0	no	0.030

(Flare of B: rise of the B|E border over the last 0.075 before the left wall, Section 5.3.4.) Compared with `rms`, `mean_abs` has 1.7 times as many cycle pixels, more distorted Condorcet regions and a round IRV edge. The FPTP flare is almost the same under both rules: it comes from the crowding of voters at the wall, which does not depend on the rule.

6.2. The round IRV edge of `mean_abs`

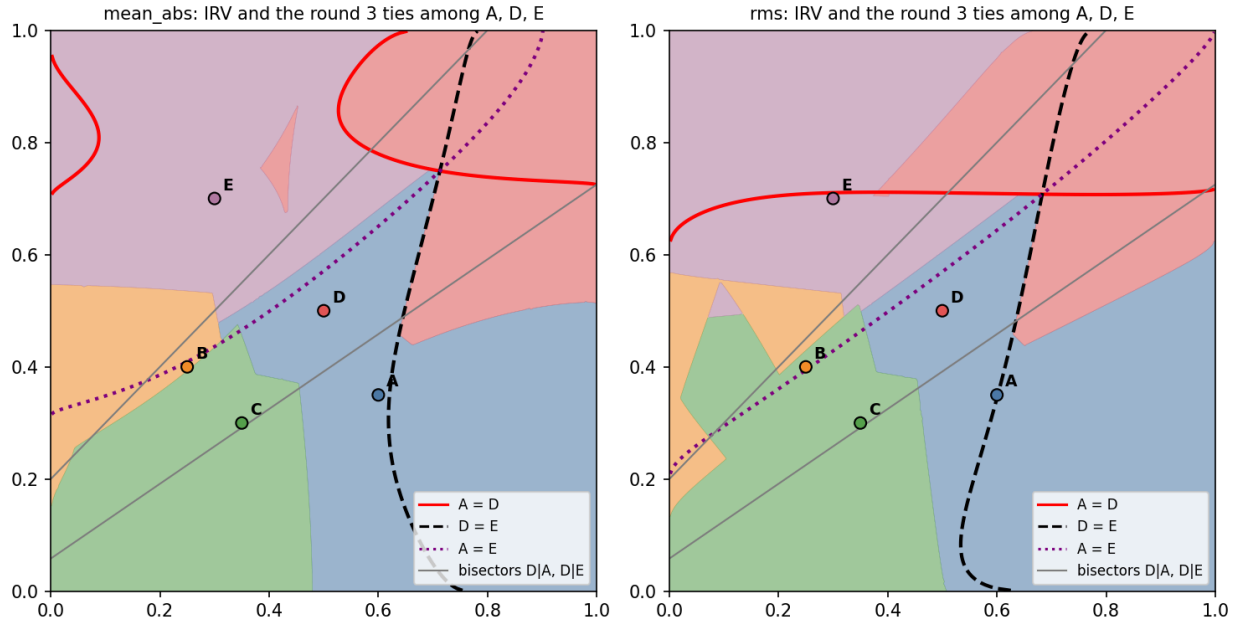


Figure 15: Beta IRV (faded) with the three round 3 tie curves among A, D, E, under `mean_abs` (left) and `rms` (right). The red curve $s_A = s_D$ bounds D's region. Gray: the bisectors D|A and D|E.

In the upper right part B and C are eliminated first. In round 3 A, D and E remain, and D's cell among them is the diagonal band between the bisectors D|A and D|E that widens towards the corner $(1, 1)$; A has the bottom right, E the top left. Whoever of A and D has fewer votes is eliminated:

- D out: the final is A against E, and E wins;
- A out: the final is D against E, and D wins near the band (the pull towards the centre again).

So the lower edge of D's region is the round 3 tie $s_A = s_D$ (red in Figure 15). Along the column $x = 0.55$ (round 3 shares; winner of the pixel):

	`mean_abs`				`rms`			
y	winner	s_A	s_D	$P(Y < 0.2)$	winner	s_A	s_D	$P(Y < 0.2)$
0.678	E	0.327	0.284	0.195	E	0.316	0.292	0.173
0.738	E	0.303	0.286	0.185	D	0.279	0.300	0.146
0.798	E	0.285	0.284	0.182	D	0.243	0.307	0.123
0.858	D	0.274	0.279	0.187	D	0.209	0.311	0.102
0.918	D	0.270	0.270	0.199	E	0.176	0.312	0.087
0.978	E	0.274	0.254	0.226	E	0.144	0.307	0.079

Under `rms`, going up the column, voters leave A's cell for D's band and E's part of the square, s_A falls steadily and the tie $s_A = s_D$ is crossed once, along a nearly straight curve. (At the top D still survives round 3 but loses the final to E.) Under `mean_abs` s_A stops falling and rises again: as the median of Y approaches 1, the fixed mean absolute deviation pushes the lower half of the voters down (Equation 9), $P(Y < 0.2)$ grows from 0.182 to 0.226, and those voters are in A's cell. D is ahead of A only in a middle band, so the tie curve closes around it into the round edge.

6.3. What keeps a border straight

Every rule is a curve $\kappa(m)$ (Section 1.5). The border between c_i and c_j is where the median of the projected voters $\nu \cdot X$ equals the bisector value. That median lies at some distance from $\nu \cdot m$, the pull of the pixel towards the centre, and the border is straight when the pull changes linearly along it.

Borders close to an axis (exact). For a small tilt ε, $F_{X+\varepsilon Y}(t) = E[F_X(t-\varepsilon Y)] = F_X(t) - \varepsilon f_X(t)E[Y] + O(\varepsilon^2)$, so

$$\operatorname{median}(X+\varepsilon Y) = \operatorname{median}(X) + \varepsilon E[Y] + O(\varepsilon^2), \tag{60}$$

and the pull is $\varepsilon(E[Y] - \operatorname{median}(Y))$. Such borders are straight exactly when median minus mean grows in proportion to the distance of the median from the centre, i.e. when $(m - EX)/\left(m - \frac{1}{2}\right)$ is constant. No small-spread assumption is needed (checked numerically). This ratio for several rules:

rule	$m = 0.6$	0.7	0.8	0.9	0.98
fixed $a + b$	0.40	0.39	0.38	0.35	0.27
`rms`	0.40	0.40	0.39	0.37	0.35
`mean_abs`	0.41	0.44	0.48	0.54	0.59

There are two ways to fail. With a fixed $a + b$ the far tail thins out near a wall, the mean catches up with the median and the pull stalls. With a fixed $E|X - m|$ the far half is pushed away (Equation 9) and the pull accelerates. `rms` lies in between, close to constant.

Diagonal borders. At 45° both coordinates are skewed at once. Exact border shapes bend one way for fixed $a + b$ and the other way for fixed $E|X - m|$, confirming the two failure modes. Where the balance between them lies depends on the position in the square, so no single curve $\kappa(m)$ keeps every border straight.

Why cycles survive. Straightness depends on how the pull changes with the position of the pixel, cycles on how it changes with the direction ν. Projections of two skewed coordinates in different directions do not share one effective centre, so the three borders of three candidates can each be straight and still miss each other. No spread rule removes cycles without removing the spread.

6.4. Benchmark

The **bend** of a rule is the largest distance of each pairwise majority border $\pi_{ij} = \frac{1}{2}$ from its chord, averaged over the pairs (units of the square; 0 for normal voters). At the same $D = 0.3$, on the default candidates:

rule	$\kappa(0.7)$	$\kappa(0.9)$	$\kappa(0.98)$	bend	cycle pixels
`mean_abs` (L_1)	1.012	0.554	0.307	0.046	2.06 %
$L_{1.5}$ from median	1.096	0.807	0.557	0.015	1.55 %
`rms` (L_2)	1.197	1.062	0.799	0.009	1.23 %
L_3 from median	1.430	1.546	1.250	0.010	0.82 %
L_4 from median	1.684	1.990	1.667	0.011	0.59 %
fixed $a + b$	1.204	1.204	1.204	0.024	1.21 %
SD about the mean	1.086	0.779	0.557	0.015	1.59 %
far-side mean distance	1.826	1.624	1.045	0.012	0.43 %
far-side RMS	2.112	2.078	1.514	0.010	0.31 %
far-side median distance	1.444	1.096	0.622	0.026	0.75 %
SD of $\operatorname{logit}(X)$	1.320	2.262	8.106	0.038	0.98 %

"Far-side" measures use only the half of the voters towards the centre of the square. Too little decrease of κ (fixed $a + b$) and too much (`mean_abs`) both bend borders. But a comparison at the same D mixes the rule with the amount of spread: cycles and bend both grow with the spread, and a rule that makes pixels near the walls narrower (larger κ) gets fewer cycles and less bend together. The fair comparison is the bend at the **same share of cycle pixels**.

The benchmark therefore swept $D \in \{0.2, 0.25, 0.3, 0.35, 0.4\}$ for every rule and compared the bend at equal shares of cycle pixels, interpolating in D. The score is the bend of the *visible* Condorcet borders (where both

candidates beat all others), on layouts of 3–6 candidates drawn uniformly from $[0.1, 0.9]^2$. On a first set of 24 layouts most rules bend more than `rms` (bend relative to `rms`): fixed $a + b$ 2.83, `mean_abs` 2.41, RMS of $\arcsin\sqrt{X}$ 2.00, far-side RMS 1.84, L_3 1.35, $L_{2.5}$ 1.21, $L_{2.25}$ 1.10. The power means below 2 were checked on 30 new layouts, with 95 % bootstrap intervals over layouts, at three cycle levels (those of `rms` at $D = 0.25, 0.3, 0.35$):

rule	$D = 0.25$ level	$D = 0.3$ level	$D = 0.35$ level
$L_{1.75}$	1.01 [0.96, 1.05]	0.92 [0.88, 0.98]	0.77 [0.73, 0.80]
$L_{1.5}$	1.14 [1.05, 1.24]	0.94 [0.83, 1.08]	0.67 [0.60, 0.77]

They bend as much as `rms` or more at the lowest level and less at the higher ones. In absolute terms the bend of `rms` is small: at $D = 0.3$ its visible borders bend by 1.5 pixels on average (90th percentile 3.1 pixels, the single worst border 4.9 pixels).

6.5. Other ways to fix the spread

- **Median absolute deviation.** For every Beta, $\operatorname{median}|X - m| < \min(m, 1 - m)$: the near half lies entirely within that distance. A fixed median absolute deviation is therefore impossible within D of every wall.
- **Normal voters clamped to the square** (moved onto the nearest edge when they would leave it) have neither the round IRV edge nor a strong flare (0.023), few cycles (0.10 %) and Schulze $\neq$ Voronoi on 7.9 % of the pixels. But they put point masses on the edges, are not a Beta, and their deviation shrinks near the walls.

6.6. Conclusion

- **`rms`** is the default, and the only rule of the web UI. It fixes a quantity with a clear meaning, pushes the far half of the voters out only by $\sqrt{2}$ near a wall, has no round IRV edge, and its borders bend by 1.5 pixels on average at $D = 0.3$; of the rules compared, only power means slightly below 2 bend less, and only at larger spreads.
- **`mean_abs`** is kept to reproduce and compare results made with it. By Equation 9 it pushes the far half of the voters out twice as far near a wall, which bends borders the most, adds cycles and makes the round IRV edge.

7. Shapes of win regions

A rule of thumb about Yee diagrams links the shape of the regions with monotonicity: methods that are not monotone, such as IRV, draw regions that are not convex and sometimes fall into several pieces. This chapter makes the link precise. The shape of a region is decided by the method and the voter model together, and the rule of thumb holds only in special cases. Monotone methods can have non-convex regions and regions in several pieces, and non-monotone methods can have convex regions.

7.1. Regions, shapes and monotonicity

The **win region** R_i of candidate c_i is the set of pixel centres $m \in (0,1)^2$ where c_i wins. Apart from exact ties, which only happen on curves (Section 3.12), every method decides its winner by strict inequalities between continuous functions of m: first-choice shares, pairwise shares, IRV totals or Schulze path strengths. So R_i is an open set. Three shapes, from the strongest to the weakest:

- R_i is **convex** if the segment between any two of its points lies in R_i.
- R_i is **star-shaped** around a point q if the segment from q to any point of R_i lies in R_i. Star-shaped around the candidate's own position is the natural property between the other two.
- R_i is **connected** if it is not the union of two disjoint non-empty open sets. For an open set in the plane this is the same as **path-connected**: any two of its points are joined by a continuous path inside it.

A convex region is star-shaped around each of its points, and a star-shaped region is connected. On the pixel grid a region is **in pieces** when its pixels fall into groups that do not touch, not even at a corner.

Curved borders. Two neighbouring regions can both be convex only if the border between them is straight: two disjoint open convex sets are separated by a line (the separation theorem), so their common border lies on that line. Hence **a curved border always has a non-convex region on at least one side**, and every method that draws curved borders has non-convex regions.

Monotonicity. A method is **monotone** if a winner stays the winner when some voters raise them on their ballots and the order of the other candidates stays the same. FPTP, Borda and Schulze are monotone, and a Condorcet winner stays the Condorcet winner when raised. IRV is not monotone: raising the winner can change who is eliminated first, and so give the winner a stronger opponent in the final.

7.2. Why monotonicity does not fix the shape

Monotonicity compares two electorates that differ in one way only: some voters raise one candidate. Moving the pixel is a different change. Every voter moves, and the rankings change among all candidates, including pairs that do not involve the winner. Even moving the pixel towards a candidate is not the same as raising that candidate, because the voters beyond the candidate move away from it. So monotonicity says nothing about the winners of neighbouring pixels.

Another property that might seem to fit does not help either. A **consistent** method (FPTP and Borda are consistent) that elects c_i in two electorates also elects c_i when the two electorates are pooled. But the voters of the midpoint between two pixels are not the pooled voters of the two pixels: for normal voters they are one Gaussian bump at the midpoint, not two bumps.

7.3. Where the regions are convex

7.3.1. Condorcet methods and symmetric voters

For normal voters, and more generally for voters that are centrally symmetric about the pixel centre, every Condorcet method draws the Voronoi diagram (Section 5.2.1), whose regions are convex polygons. This includes Condorcet methods that are not monotone, such as Baldwin's and Nanson's methods: they elect the Condorcet winner whenever there is one, and for these voters there always is one. So a non-monotone method can have convex regions. Their convexity comes from the Condorcet criterion together with the symmetry of the voters, not from monotonicity.

7.3.2. Candidates on a line: FPTP versus IRV

Let the candidates lie on a line with unit direction u. Their Voronoi cells are strips perpendicular to the line, $V_i = \{p : p \cdot u \in I_i\}$, with intervals $I_1 < I_2 < \dots$ in the order of the candidates. For normal voters the projection $X \cdot u$ is $\mathcal{N}(t, \sigma^2)$ with $t = m \cdot u$, so every share depends only on t, and every region is a union of strips.

FPTP regions are single strips. For $t < t'$ the ratio of the two projected densities,

$$L(x) = \frac{\varphi((x - t')/\sigma)}{\varphi((x - t)/\sigma)} = \exp\left(\frac{(t' - t)x - (t'^2 - t^2)/2}{\sigma^2}\right), \tag{61}$$

increases in x. Moving the pixel from t to t' multiplies the share of cell k by the average of L over I_k, and for $i < j$ the interval I_j lies to the right of I_i, so the share of c_j grows by a larger factor than that of c_i: the ratio $s_j(t)/s_i(t)$ never decreases. Hence c_i beats c_j on a half-line of t, to the left of some point for $j > i$ and to the right of some point for $j < i$. The FPTP region of c_i is the intersection of these half-lines, one interval of t: a strip, which is convex. The argument only uses that L increases, which holds whenever the voters of a pixel are one fixed log-concave density moved to the pixel. Beta voters are not of this kind, and near the walls they are not even stochastically ordered (Section 7.6), so the argument does not carry over to them.

IRV splits the middle candidate. Take L $= (0.2, 0.5)$, M $= (0.5, 0.5)$ and R $= (0.8, 0.5)$ with normal voters, $D = 0.3$ ($\sigma = 0.376$). At the centre, M is the nearest candidate and the Condorcet winner, but it has the fewest first choices, $2\Phi(0.15/\sigma) - 1 = 0.310$ against 0.345 each for L and R. So M is eliminated first: the *centre squeeze*. At $x = 0.4$ the shares are L 0.445, M 0.300 and R 0.254, so R is eliminated first and M beats L in the final. M therefore wins IRV on two strips, $0.35 < x < 0.46$ and $0.54 < x < 0.65$, but not between them, and the regions of L and R are in two pieces as well (Figure 16). FPTP gives L and R one strip each and M nothing.

Here the rule of thumb is exactly right: the monotone method keeps every region in one piece and the non-monotone one does not. Both properties of IRV have the same cause: who meets whom in the final depends on who is eliminated first.

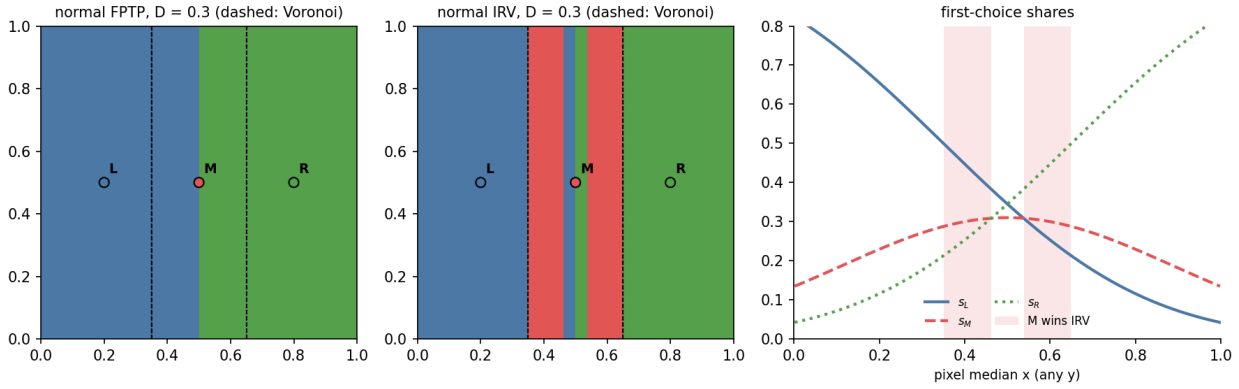


Figure 16: Three candidates on a line, normal voters, $D = 0.3$. FPTP gives every candidate at most one strip. In IRV the middle candidate M is squeezed out near the centre, where it has the fewest first choices (right), so M wins two separate strips, and so do L and R.

7.4. Monotone methods with non-convex regions

How often. The random layouts below count a region as non-convex when its convex hull contains pixels that are more than 3 pixels away from the region; pixelation alone cannot cause that. Table 7 counts, on 100 random layouts of 3–6 candidates in $[0.05, 0.95]^2$ (150×150 pixels, `docs/figures.py search`), the layouts with at least one such region, and the layouts with a region in pieces.

		normal voters		Beta voters (rms)	
method	monotone	non-convex	in pieces	non-convex	in pieces
FPTP	yes	42 / 21	0 / 0	34 / 47	0 / 0
Borda	yes	35 / 24	0 / 0	53 / 52	0 / 0
Schulze	yes	0 / 0	0 / 0	8 / 49	0 / 0
IRV	no	90 / 95	25 / 45	94 / 98	26 / 45

Table 7: Layouts out of 100 with at least one non-convex region, or with a region in pieces (more than one piece of at least 20 pixels), at $D = 0.2$ / $D = 0.3$.

Normal voters. The FPTP and Borda borders are contour lines of blurred cells and are curved (Section 5.3), so by the argument above some of their regions are not convex. The dent is often only a few pixels deep. Schulze draws the Voronoi diagram and is always convex.

Beta voters and Schulze. Under Beta voters Schulze regions are not convex either. Figure 17 shows an example with $c_1 = (0.457, 0.407)$, $c_2 = (0.254, 0.289)$, $c_3 = (0.334, 0.641)$ and $c_4 = (0.235, 0.464)$, `rms` and $D = 0.3$. c_1 wins at both ends of the segment, c_4 in most of it. The notch is not a bend of a pairwise border: along the whole segment c_4 beats c_1 head to head, with 0.508 to 0.516 of the voters, and in the middle c_4 is the Condorcet winner. The two ends lie in pockets of Condorcet cycles; in each the fourth candidate loses to the other three:

- at the upper end $c_1 > c_3 > c_4 > c_1$, with 0.512, 0.528 and 0.508 of the voters;
- at the lower end $c_1 > c_2 > c_4 > c_1$, with 0.530, 0.537 and 0.516.

In a cycle of three candidates Schulze elects the candidate beaten by the weakest of the three defeats. Next to the tie line of c_1 and c_4 the weakest defeat is that of c_1 by c_4, barely above one half. So Schulze gives c_1 the part of each pocket along that line, on c_4's side of it. In general Schulze splits a cycle pocket among its three candidates, each winning the part next to the tie line of its own defeat. Every Schulze region therefore reaches across its pairwise borders into the neighbouring pockets, and between two such pockets the region has a notch. Normal voters have no pockets: their three pairwise borders meet in one point, the Voronoi vertex.

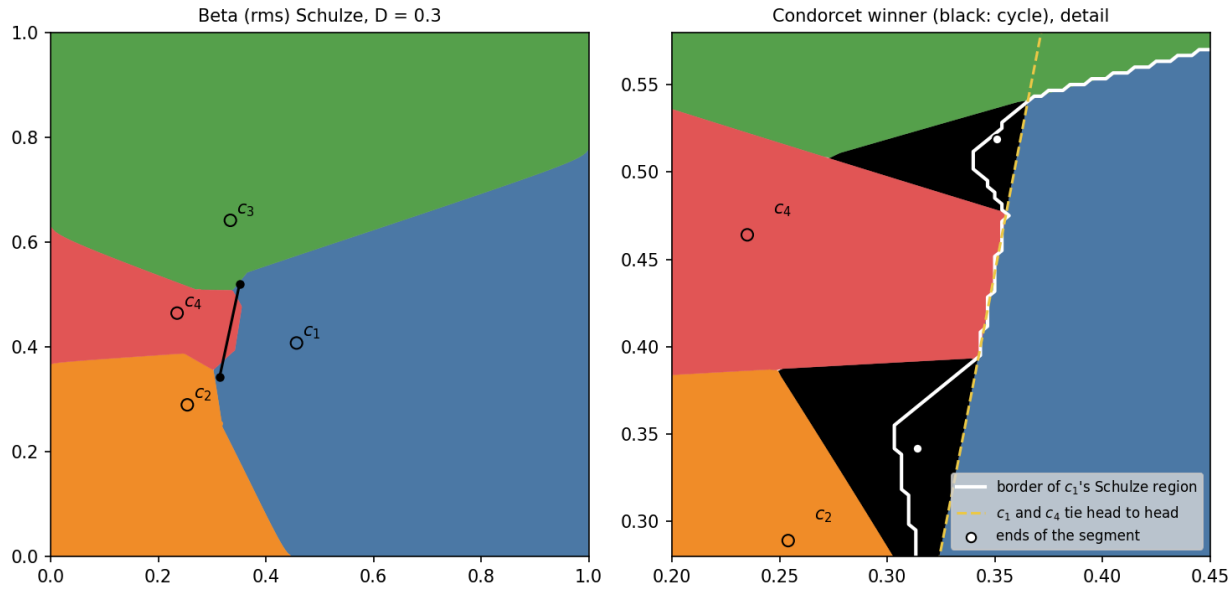


Figure 17: Left: Schulze with Beta voters (`rms`, $D = 0.3$). c_1 (blue) wins at both ends of the segment, c_4 (red) in between. Right: detail with the Condorcet winner (black: cycles). c_1's Schulze region, to the right of the white line, reaches across the tie line of c_1 and c_4 (yellow) into two cycle pockets.

7.5. Monotone methods with regions in pieces

FPTP can split a region, already under normal voters. Take seven candidates: A $= (0.5, 0.5)$; N $= (0.5, 0.6)$ and S $= (0.5, 0.4)$ just above and below it; and NW $= (0.3, 0.95)$, NE $= (0.7, 0.95)$, SW $= (0.3, 0.05)$, SE $= (0.7, 0.05)$ far above and below. A's Voronoi cell is the thin strip $0.45 < y < 0.55$.

With normal voters and $D = 0.12$ ($\sigma = 0.150$), N and S are strongest in the middle column: at A's own position A has 0.260 of the first choices and N and S have 0.320 each, and along the whole column $x = \frac{1}{2}$ A trails the best rival by at least 0.060. Further out, the voters above the strip are split between N and NW or NE, and those below between S and SW or SE. Every rival is weaker there, and A wins. So A's region is in two pieces, $0.13 < x < 0.28$ and $0.72 < x < 0.87$ (both at $0.46 < y < 0.54$), separated by the column where A never wins (Figure 18). Beta voters (`rms`, same D) give the same picture, with pieces at $0.08 < x < 0.25$ and $0.75 < x < 0.92$; along the column A trails by at least 0.081.

A does not win at its own position, so its region is not even star-shaped around A. The regions of NW, NE, SW and SE are not convex either: the domes of N and S bite into them. Layouts like this one are rare. In the random layouts of Table 7 no region of a monotone method is in pieces, while IRV has regions in pieces in a quarter to a half of them.

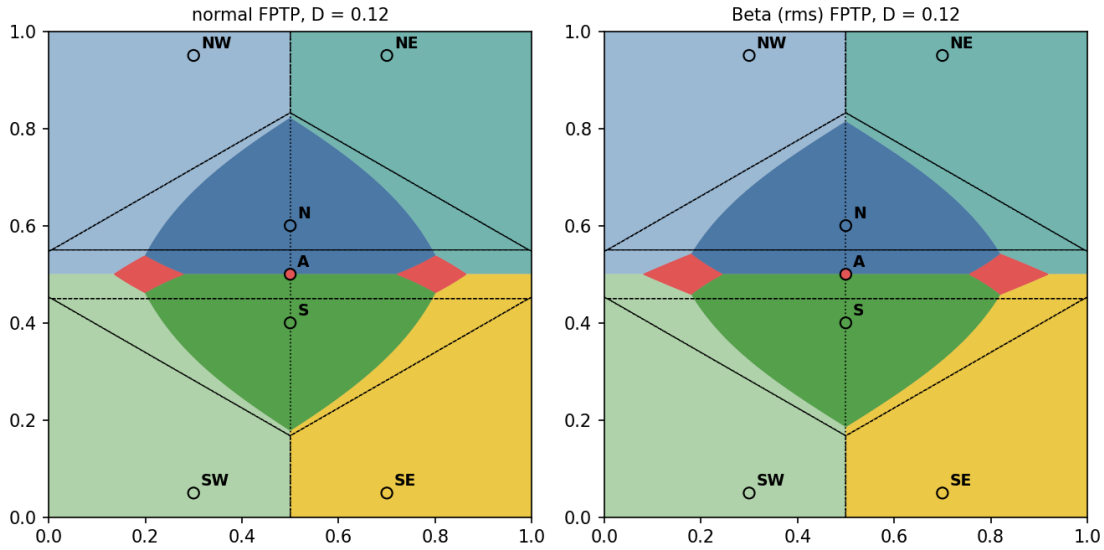


Figure 18: FPTP with normal and Beta voters, $D = 0.12$. Dashed: Voronoi borders; dotted: the column $x = \frac{1}{2}$, where A (red) never wins. A's region is in two pieces.

7.6. Condorcet methods under Beta voters

For Condorcet methods under Beta voters, the random layouts gave non-convex regions but never regions in pieces. Whether Beta voters can split a Condorcet region is open. This section collects what is known.

Two candidates. Every method then elects the majority winner, and the region of c_i is $\{\pi_{ij} > \frac{1}{2}\}$, bounded by one bent border. Suppose that the voters of a pixel become stochastically larger in each coordinate as its median grows, i.e. that $P(X \le t)$ never increases with the median, for any t. Then $\nu \cdot X$ becomes stochastically larger as m_x grows if $\nu_x > 0$ and smaller if $\nu_x < 0$, and likewise for m_y. So π_{ij} is monotone in each coordinate of the pixel, and with every pixel the region contains the whole rectangle between that pixel and one corner of the square. Any two points of the region are then joined through that corner: the region is connected, and its border is the graph of a monotone function. Normal voters have this property, because they only shift.

Beta voters do not quite have it. As the median moves towards a wall, the far half of the voters is pushed out (Section 1.5), and $P(X \le t)$ grows for some t. The table gives, for medians up to $1 - 1/800$ (the last pixel of a 400×400 grid), the median from which $P(X \le t)$ first grows, and the largest increase of $P(X \le t)$ between two medians, over all t.

	$D = 0.2$		$D = 0.3$	
rule	grows from	largest rise	grows from	largest rise
`rms`	0.773	0.011	0.900	0.018
`mean_abs`	0.673	0.097	0.654	0.146

So the argument does not apply. Numerically, no two-candidate region was in pieces: 0 of 900 layouts (9 directions of the bisector times 100 positions) for each of `rms` and `mean_abs` at $D = 0.2$, 0.3 and 0.4.

More candidates. The Condorcet region of c_i is the intersection of its pairwise regions. For it to fall into pieces, two of its borders would have to cross each other twice. They are bent by only a few pixels, so this would need two nearly parallel borders close together.

7.7. Summary

- Convex regions come from the voter model and the method together. Under normal voters every Condorcet method, monotone or not, draws convex regions, and FPTP does for candidates on a line.
- Monotone methods can have non-convex regions (FPTP and Borda under both models, Schulze under Beta voters, Figure 17) and regions in pieces (FPTP, Figure 18).
- So neither the shape nor the connectedness of a region proves anything about monotonicity. What holds is weaker: in the random layouts only IRV split regions, and for candidates on a line FPTP can never split a region, while IRV does.

8. Real-time diagrams in the web UI

The web UI redraws the diagram while a candidate is dragged. Chapter 2 computes the share of every ranking for every pixel, which is exact and serves every method, but takes too long for that:

step ($D = 0.2$, `rms`, 49×49 nodes)	5 candidates	8 candidates
ranking shares at the nodes (`margin/shares.py`)	333 ms	2878 ms
bisector arrangement (`ranking_cells`)	8 ms	99 ms
interpolation to a 300×300 evaluation grid	12 ms	88 ms
IRV / Schulze on the grid	61 / 64 ms	703 / 358 ms

Almost all of the time goes to the shares at the $N \times N$ nodes, which do not depend on the resolution of the evaluation grid. Drawing the diagram as curves (Section 8.5) is the right output, but on its own it saves little. The speed comes from computing fewer integrals (Section 8.1), keeping those a drag does not change (Section 8.2), making each one cheaper (Section 8.3), and compiling the loops that remain (Section 8.4). This chapter describes `margin/` (`shares.py`, `beta_tables.py` and `regions.py`; `geometric.py` is the subject of Chapter 9) and the methods built from blocks in `build/`, which `web/app.py` uses for the UI. `docs/figures.py` uses these same polygon regions and exports the diagrams as PNG/SVG. The tests validate the margin pipeline against analytical and Monte Carlo references.

8.1. What each method needs (`margin/shares.py`)

The arrangement of all $C(C-1)/2$ bisectors has $O(C^4)$ cells and edges: 43 cells and 65 slanted edges for 5 candidates, 272 cells and 468 slanted edges for 8. Most methods need much less than the share of every cell.

Pairwise shares. Schulze, Minimax, Black and the Condorcet winner only use π_{ij} (Equation 36). Borda does too: $C - 1 - \mathrm{pos}_r(i)$ is the number of candidates below c_i in ranking r, so

$$\mathrm{score}_i = \sum_r P(r)(C - 1 - \mathrm{pos}_r(i)) = \sum_{j \neq i} \pi_{ij}. \tag{62}$$

Baldwin and Nanson take the same sum over the remaining candidates in every round (Equation 37), so they need nothing else either.

Each π_{ij} is the share of one half-plane, the part of the square on c_i's side of the bisector: a convex polygon with one slanted edge. For 8 candidates that makes 28 slanted edges instead of 468. For normal voters no integral is needed at all: $\pi_{ij} = \Phi\big(\delta_{ij}(m)/\sigma\big)$ (Equation 56).

First-choice shares. FPTP only uses s_i (Equation 33), the share of the Voronoi cell of c_i: C convex polygons with about $3C$ edges in all. For normal voters the cells are taken in the box of Section 2.5 and summed with Owen's T.

The whole profile. IRV needs the first choices among every set of remaining candidates. In a benchmark with 8 candidates it visited 106 of the 247 sets of two or more, so computing sets lazily saves little, and IRV uses the full arrangement. So does IRV set + KOTH + Score (Section 3.11), which needs the same first choices for its schedules, and DAC (Section 3.7), whose coalitions are the first k candidates of every ranking.

All three are sums of edge terms: the Green integrals of Section 2.3 for Beta voters and the signed triangles of Section 2.5 for normal voters.

Bullet voters (Section 4.1) need the shares of their method, and next to pairwise or score shares also the first-choice shares; the profile has its own. **Burying voters** (Section 4.2) need the first-choice shares of the poll and the profile for pairwise shares, and **compromising voters** (Section 4.3) the profile for first-choice and pairwise shares. Next to score shares both need the ballots of the voter grid, from which what burying takes back, t_{ij}, what compromising adds, g_{ij}, the shares of every score of the frontrunners that they move (Section 3.10) and how STAR's comparisons move (Section 4.4) are computed for each pair of frontrunners, at its pixels only, multiplying only the rows and columns of the voters' shares of the rectangles that those pixels have. These read who ranks whom above whom, so their ballot table also keeps one order of distance per rectangle: its

rectangles with one ballot must also have the same order at their four corners, and the ones a bisector crosses are averaged too. That table takes about 1.6 times as long as the one of the part not given alone, and gives the part not given as well.

Score shares. Score uses the part of the top score the voters do not give each candidate. The regions of these voters have curved borders, so their shares have no edge terms: they are summed over a grid of rectangles with the exact share of the voters in each, at the $G + 2$ points per axis where the borders are traced and not at the nodes (Section 3.8). The voters' shares of the rectangles depend only on the voter model and G. They are kept; computing them takes up to 0.2 s for Beta voters. The ballot table is computed again at every step of a drag: for five to eight candidates it takes about 60 to 150 ms with six levels and 30 to 90 ms with two; with more levels more rectangles have a border inside and hold the ballots of 8×8 points. The fine grid after the drag reuses it, and then the matrix products are most of the work. STAR takes one table for its totals and its comparisons, and majority judgment the share of every score of every candidate from it, L products per candidate (Section 3.10).

8.2. Dragging: an edge cache

An edge term depends only on the edge and the voter model. The terms are therefore cached with the endpoints as key (rounded to 10^{-9}, the smaller endpoint first, the sign from the orientation, as in Section 2.2), up to 64 MB, least recently used out first. Dragging c_k moves only the $C - 1$ bisectors through c_k; the half-planes and Voronoi edges of all other pairs keep their endpoints and come from the cache. For pairwise shares 4 of 10 half-planes are recomputed with 5 candidates, 7 of 28 with 8. The arrangement gains less: every other bisector is cut by the moving ones, so for 5 candidates about 60 of its 89 edges are new after each step.

8.3. Tabulated Beta CDF (`margin/beta_tables.py`)

An edge integral evaluates G_j at Q points for every pair of nodes, $QN^2 \approx 58\,000$ calls of `betainc` at about 400 ns each. The N Beta distributions of the nodes depend only on D and the spread rule, so their CDFs can be tabulated once.

The CDF is tabulated against $s = \operatorname{logit} y$, with the exact slope

$$\frac{\mathrm{d}G}{\mathrm{d}s} = g(y)\, y(1-y). \tag{63}$$

Near a wall $G(y) = y^a/(aB(a,b))(1 + O(y)) = e^{as}/(aB(a,b))(1 + O(y))$: an exponential in s, smooth even for $a, b < 1$, where the density itself is infinite. A cubic Hermite interpolant with the exact slope on a uniform grid ($s \in [-40, 40]$, step 0.01) is therefore accurate everywhere ($4 \cdot 10^{-9}$ for the narrowest voters). Below the table ($y < 4 \cdot 10^{-18}$) the power law $y^a/(aB(a,b))$ itself is used, which is exact to double precision there; above it there is no double $y < 1$. (A table of logit G instead is smooth enough for a step of 0.02, but evaluating it costs an `exp` per point and node, which was most of the time of the compiled kernel.) Two numerical details matter:

- Of G and $1 - G$ the smaller is computed directly and the other as its complement. Using $1 - G(y) = G'(1 - y)$ with the mirrored Beta is not enough: $1 - y$ rounds to 1 below $y \approx 10^{-16}$, where $G \approx y^a$ is still 0.02 for $a = 0.1$.
- The same grid in s serves all nodes, so a point's interval and its position in it are computed once and reused for all N nodes. The remaining work per point and node is one cubic, with the coefficients of all nodes in one interval stored together.

The quantile F^{-1} has a table in logit–logit coordinates, logit x against logit u, for $u \in [\operatorname{expit}(-30), \operatorname{expit}(30)]$ with step 0.02. It is asymptotically linear at both ends ($x \approx (aBu)^{1/a}$ near 0), so outside the table it is extended linearly. A u-interval of width w changes an edge integral by at most w, because the integrand is bounded by 1, so values below 10^{-13} do not matter.

Over all offered deviations and spread rules the tables agree with `betainc` to $4 \cdot 10^{-9}$ and with `betaincinv` to $2 \cdot 10^{-8}$ in x (tests: 10^{-8} and 10^{-7}), and edge integrals agree with the exact ones to $5 \cdot 10^{-9}$. With the compiled kernel of Section 8.4 an edge takes about 0.2 ms instead of 23 ms. A table is built in about 0.2 s, in parallel over the nodes.

8.4. Compiled kernels (numba)

With the tables in numpy an edge still took 1.3 ms: about 13 passes over temporary arrays of QN^2 elements, and threads ran edges only 1.5 times faster than one, since numpy holds the interpreter lock between its many small operations. The loops that remain are therefore compiled with numba:

- the edge integrals of Section 2.3 on the tables, every form of the edge (vertical, horizontal, split, u and v) in one kernel for a whole batch of edges; the lookup, the cubic and the quadrature sum happen in one pass (about 2 ns per point and node);
- the triangles of normal voters with Owen's T (Section 2.5);
- the clipping of a polygon by a bisector (`ranking_cells`), which with a vectorized test of which cells a bisector crosses takes the arrangement of 8 candidates from 120 ms to 5 ms;
- the rounds of IRV (Chapter 3), and the schedules of IRV set + KOTH + Score, each set of remaining candidates tallied once (Section 3.11);
- the rounds of DAC, on the scores of all sets of candidates as bit masks (Section 3.7);
- which candidates a round of Baldwin or Nanson drops (`build/rounds.py`); their Borda scores (Equation 37) are one `einsum` of numpy per round;
- the distance of every sample of one boundary to the nearest segment of the other (Chapter 10): both directions, about 700 samples against 1200 segments each at $G = 320$, take 3 to 9 ms instead of 55 to 130 ms in numpy.

The kernels release the interpreter lock but are not parallel themselves; `threads.py` runs chunks of their work (edges, pixels) in a pool of threads. A kernel with numba's own parallel loops would abort, with numba's default threading layer, as soon as two threads call it at once, as two requests of the web UI can. Two details decided the speed: indexing the coefficient table element by element (an array view per point and node costs more than the cubic), and not calling `exp` in the innermost loop (this numba has no vectorized `exp`), hence the table of G itself. The first run compiles the kernels; numba caches them on disk, and the UI loads them at start.

Compute per drag update (in process, best of several runs, $G = 160$), with the kernels and before them:

	candidates	FPTP	Borda	Schulze	IRV
Beta, $D = 0.2$	5	5 (12) ms	11 (23) ms	15 (26) ms	14 (170) ms
	8	7 (27) ms	22 (41) ms	30 (61) ms	79 (984) ms
Beta, $D = 0.05$	5	7 (28) ms	17 (48) ms	18 (48) ms	24 (366) ms
	8	10 (52) ms	31 (95) ms	39 (96) ms	162 (2308) ms
normal, $D = 0.2$	5	6 ms	10 ms	14 ms	31 ms
	8	9 ms	23 ms	27 ms	118 ms

8.5. Borders as zero sets (`build/`, `margin/regions.py`)

The shares are smooth in the median m; only the winner jumps. Each method compares continuous functions of the shares, so its borders are where such a comparison is a tie. The methods of the web UI return, next to the winner, a **margin** $\mu \geq 0$ that is continuous in the shares and 0 on every border between two winners. It is the smallest gap in a comparison that could change the winner, so it can also vanish where such a comparison ties but the winner stays; that makes no border, because the sign of ψ_c below changes only with the winner:

method	margin μ
FPTP, Borda	lead of the top score over the second
Condorcet winner	$\min_{j\neq w}(\pi_{wj}-\pi_{jw})$; in a cycle $-\max_i \min_{j\neq i}(\pi_{ij}-\pi_{ji})$
Schulze	$\min_{e\neq w}\max_f(p_{fe}-p_{ef})$
Minimax	lead of the top weakest link over the second
Black	as the Condorcet winner; in a cycle the smaller of that and the margin of Borda
IRV	smallest gap between the two lowest tallies over all rounds
DAC	smallest gap over all rounds between the set taken and its highest rival
Baldwin	smallest gap between the two lowest Borda scores over all rounds
Nanson	smallest distance of a Borda score from the mean over all rounds
Score	lead of the top mean score over the second
Majority judgment	smallest gap γ of Equation 49 between the winner and another candidate
IRV set + KOTH + Score	the smallest of μ_W of Section 3.11, the margin of KOTH and the lead of the top mean score in F

All methods are built from blocks (`yeelab.build`), and the margin is never put together by hand: every block that decides computes the gap of its decision, and the margin is the smallest of them. `Highest` has the lead of the top score (with `among`, among the candidates chosen, and their margin), `Eliminate` the gap of each round, `Schedules` that of every round of every schedule, `DescendingCoalitions` that of every round, `Unbeaten` the gaps below and `Conditional` those of the condition and selected method. Baldwin and Nanson are in Section 3.5, Minimax and Black in Section 3.6, DAC in Section 3.7, the score methods in Section 3.8 and IRV set + KOTH + Score in Section 3.11.

IRV. On each side of a curve where some round's two lowest tallies tie, the gap of that round tends to 0. So μ is continuous, and it vanishes on every curve where an elimination changes, whether or not the winner changes there.

Unbeaten. The Condorcet winner and Schulze both elect a candidate that no one beats, from links $\ell_{ij}=-\ell_{ji}$ in which c_i beats c_j where $\ell_{ij}>0$: the margins of Equation 39, or the strongest paths below. Let

$$\text{beaten}_e=\max_{f\neq e}\ell_{fe} \tag{64}$$

be the strongest link into c_e, negative if c_e beats everyone and positive if someone beats it. The winner w has the smallest beaten_e (ties: the first). It changes only where another candidate becomes unbeaten, that is, where some beaten_e with $e\neq w$ crosses 0, so the margin is how far the closest of the others is from that:

$$\mu=\min_{e\neq w}\max(\text{beaten}_e,0). \tag{65}$$

Weak and strict winners. What the winner must be depends on the links. On the strongest paths, "beats" ($p_{ij}>p_{ji}$) has no cycles, as Schulze proved, so someone is unbeaten, $\text{beaten}_w\leq 0$, and the winner is a *weak* winner: $p_{we}\geq p_{ew}$ for all e, as in Chapter 3, which may tie some candidates. The margins $\ell_{ij}=\pi_{ij}-\pi_{ji}$ can form a cycle. There the winner must be a *strict* winner, a Condorcet winner with $\text{beaten}_w<0$ who beats everyone; where the smallest beaten_w is not negative, the point is a cycle and marked `CYCLE`. That decision flips where beaten_w crosses 0, so $|\text{beaten}_w|$ is a gap as well:

$$\mu=\min\Big(\min_{e\neq w}\max(\text{beaten}_e,0),\,|\text{beaten}_w|\Big). \tag{66}$$

For antisymmetric links the second term is never larger than the first. With a Condorcet winner, $\text{beaten}_e\geq\ell_{we}\geq\min_f\ell_{wf}=-\text{beaten}_w$ for every other e. In a cycle, $\text{beaten}_e\geq\text{beaten}_w\geq 0$. So $\mu=|\text{beaten}_w|$: the narrowest win of the Condorcet winner, and, in a cycle, how far every candidate is from beating everyone, as in the table. For Schulze's links the gap is left out. There beaten_w is 0 on whole areas, where the widest paths

from w to another candidate and back share their weakest link, and the margin would vanish on an area where no border runs.

Strongest paths. With complete rankings $\pi_{ij} + \pi_{ji} = 1$, so the margin $\ell_{ij} = 2\pi_{ij} - 1$ orders the links exactly like the winning votes π_{ij} of Chapter 3, and the winners are the same. Unlike winning votes, the link strengths $\max(\ell_{ij}, 0)$ are continuous in π, and so are the path strengths p_{ij} built from them. `StrongestPaths` returns the links $p_{ij} - p_{ji}$, which make $\text{beaten}_e = \max_{f \neq e}(p_{fe} - p_{ef})$, the expression of the table.

Where a Condorcet winner w exists it is the Schulze winner: $p_{we} \geq \ell_{we} > 0$, and no link leads into w, so $p_{ew} = 0$. The widest paths are therefore only computed at the points without a Condorcet winner. Elsewhere the winner and margin of the margins themselves stand in: the margin is positive, at most the margin on the paths (as $p_{we} \geq \ell_{we}$), and 0 on the border of the Condorcet region, so it vanishes on the same borders.

Conditional. `Conditional(condition, when_true, when_false)` chooses `when_true` where the condition is true, and `when_false` elsewhere. Method-result comparisons such as `fptp == two_round` and `two_round != condorcet` create deferred Boolean conditions; combine them with `&` or `|`. Their input shares are collected from the methods in the expression. A Winner block used directly as the condition is true where it elects a candidate; Black uses the Condorcet method this way, with Borda as `when_false`. The result margin includes the smallest method margin in the condition and the selected method's gap. For Black, the margin is the Condorcet margin outside cycles and the smaller of the Condorcet and Borda margins inside them. Use `.same_as(other)` when a regular Python Boolean comparison of method definitions is needed; the `==` and `!=` operators build deferred conditions for use by `Conditional`:

$$\mu = \begin{cases} \mu_{\text{first}} & \text{where first elects someone} \\ \min(\mu_{\text{first}}, \mu_{\text{second}}) & \text{in a cycle of first} \end{cases}. \tag{67}$$

In a cycle μ_{first} is how far the point is from having a Condorcet winner, so it vanishes on the border of the cycle, where `second` hands over to `first`, and μ_{second} on the borders `second` draws inside the cycle.

8.5.1. Example: margins at two medians

The examples use the candidates A–E (Figure 2) and the defaults of the UI: Beta voters, `rms` and $D = 0.2$. Table 8 lists the exact shares at two medians: $m_1 = (0.5, 0.5)$, the position of D, and $m_2 = (0.32, 0.545)$, in a small pocket where B, D and E form a cycle. Table 9 gives the winners and margins. The shares are rounded to four decimals, while the margins come from the unrounded shares, so a difference of two entries can be off by one in the last digit.

median		A	B	C	D	E
m_1	s_i	0.2697	0.1164	0.1303	0.2565	0.2271
	score_i	2.1051	1.6919	1.8157	2.7301	1.6572
	π_{Aj}	–	0.5783	0.5819	0.3636	0.5812
	π_{Bj}	0.4217	–	0.4436	0.3095	0.5172
	π_{Cj}	0.4181	0.5564	–	0.3111	0.5301
	π_{Dj}	0.6364	0.6905	0.6889	–	0.7143
	π_{Ej}	0.4188	0.4828	0.4699	0.2857	–
m_2	s_i	0.1243	0.2218	0.1227	0.1730	0.3582
	score_i	1.2748	2.3343	1.8258	2.3824	2.1827
	π_{Aj}	–	0.3342	0.3539	0.2084	0.3784
	π_{Bj}	0.6658	–	0.6587	0.5206	0.4891
	π_{Cj}	0.6461	0.3413	–	0.3949	0.4435
	π_{Dj}	0.7916	0.4794	0.6051	–	0.5062
	π_{Ej}	0.6216	0.5109	0.5565	0.4938	–

Table 8: Shares at m_1 and m_2 (Beta, `rms`, $D = 0.2$): first choices s_i (Equation 33), Borda scores and pairwise shares π_{ij} (Equation 36), candidate i in the row, j in the column.

method	winner at m_1	μ	winner at m_2	μ
FPTP	A	0.0132	E	0.1363
Borda	D	0.6250	D	0.0481
Baldwin	D	0.0347	E	0.0190
Nanson	D	0.1051	E	0.0046
Condorcet winner	D	0.2727	none (cycle)	0.0125
Schulze	D	0.2727	E	0.0093
Minimax	D	0.5455	E	0.0093
Black	D	0.2727	D	0.0125
IRV	D	0.0138	E	0.0016

Table 9: Winners and margins at m_1 and m_2, as returned by the methods of the web UI.

At m_1. D sits at the median of the voters, but A has the most first choices, so FPTP elects A with $\mu = s_A - s_D = 0.2697 - 0.2565 = 0.0132$. The Borda scores are the row sums of π (Section 8.1), and D leads A by $2.7301 - 2.1051 = 0.6250$. D beats every candidate head to head, most narrowly A: the Condorcet margin is $\pi_{DA} - \pi_{AD} = 0.2727$, and Schulze and Black, which elect the Condorcet winner, use the same margin. Minimax elects D too: its weakest link is 0.2727, A's is the best of the others, -0.2727, and μ is the lead 0.5455. IRV eliminates B, C, E and A in turn:

round	A	B	C	D	E	out	gap
1	0.2697	0.1164	0.1303	0.2565	0.2271	B	0.0138
2	0.2697		0.2129	0.2622	0.2552	C	0.0423
3	0.3636			0.3507	0.2857	E	0.0650
4	0.3636			0.6364		A	0.2727

The gap between the two lowest tallies is smallest in the first round, between B and C, so $\mu = 0.0138$. Baldwin drops E, B, C and A, the first with the smallest gap, $1.6919 - 1.6572 = 0.0347$ to B. Nanson drops B, C and E

at once, whose scores are below the mean 2, with A closest to it ($\mu = 2.1051 - 2 = 0.1051$), and then A. Both elect the Condorcet winner D.

At m_2. B, D and E each beat A and C, but among themselves they form a cycle: B beats D ($\pi_{BD} = 0.5206$), D beats E (0.5062) and E beats B (0.5109). The narrowest head-to-head results $\min_{j \neq i}(\pi_{ij} - \pi_{ji})$ are -0.0218 for B, -0.0412 for D and -0.0125 for E (A and C lose by much more). No candidate beats everyone, and the Condorcet margin is 0.0125, E's deficit against D.

Schulze has to resolve the cycle. Its links $\pi_{ij} - \pi_{ji} > 0$ among B, D and E are B⟶D 0.0412, D⟶E 0.0125 and E⟶B 0.0218. A and C beat none of the three, so the widest paths between B, D and E stay among them: $p_{BE} = \min(0.0412, 0.0125) = 0.0125$ through D, $p_{DB} = \min(0.0125, 0.0218) = 0.0125$ through E, and $p_{ED} = \min(0.0218, 0.0412) = 0.0218$ through B. E is unbeaten, since $p_{EB} = 0.0218 > p_{BE}$ and $p_{ED} = 0.0218 > p_{DE} = 0.0125$, so E wins. For the margin, each other candidate e is beaten by $\max_f(p_{fe} - p_{ef})$: B only by E, by $0.0218 - 0.0125 = 0.0093$; D by B, by $0.0412 - 0.0125 = 0.0287$ (and by E, by 0.0093); A and C by 0.5832 and 0.3175. The smallest of these, $\mu = 0.0093$, says that B is the candidate closest to being unbeaten.

Minimax elects E as well, whose worst defeat, 0.0125, is the smallest (B's is 0.0218, D's 0.0412); $\mu = -0.0125 - (-0.0218) = 0.0093$ is its lead over B. Black has no Condorcet winner to elect here and takes the Borda winner D, with $\mu = \min(0.0125, 0.0481) = 0.0125$, the smaller of the Condorcet margin and Borda's: the cycle would end, and D lose its place to a Condorcet winner, if E's deficit against D closed.

Borda elects D, but Baldwin and Nanson resolve the cycle like Schulze. They drop A and C first (Baldwin one at a time) and are left with B, D and E, whose Borda scores among the three are 1.0097, 0.9856 and 1.0046, with mean 1 (Equation 37). Both drop D, and then B, which E beats. Nanson's margin $\mu = 1.0046 - 1 = 0.0046$ is how close E came to going out with D; Baldwin's $\mu = 1.0046 - 0.9856 = 0.0190$ how close E came to being the lowest instead of D.

In the first round of IRV, A (0.1243) and C (0.1227) nearly tie for last place, so $\mu = 0.0016$. E wins whichever of the two goes out first: m_2 lies next to a curve where μ vanishes but the winner does not change. Figure 19 shows μ over the whole square, with m_1 and m_2 marked. The FPTP margin vanishes only on the borders. The IRV margin also vanishes on many curves that are not borders: there two candidates tie for last place in some round, but the winner does not change.

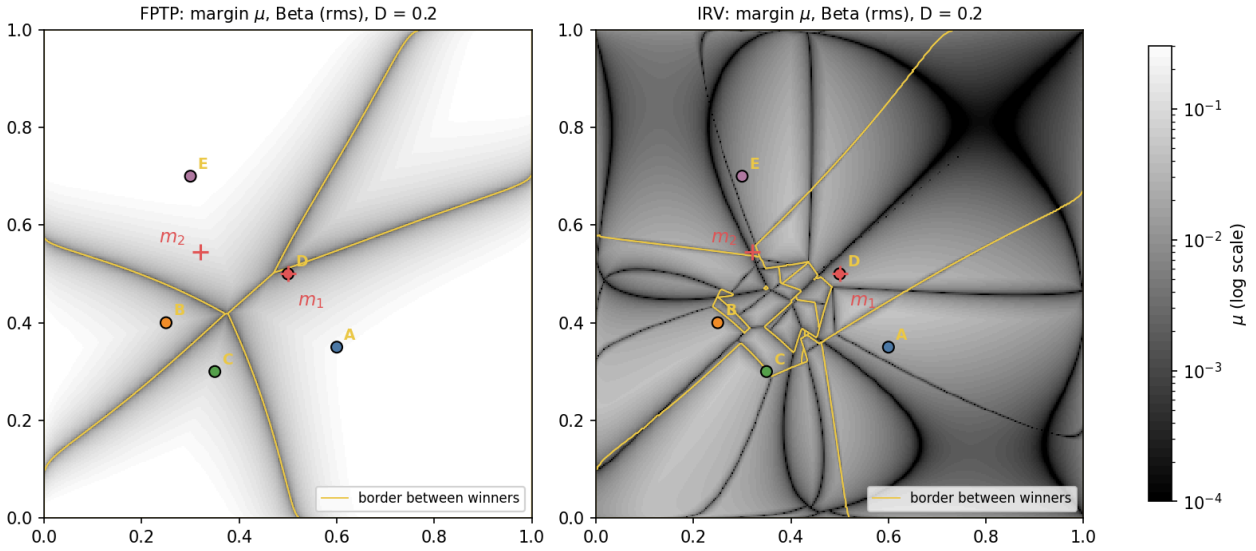


Figure 19: The margin μ of FPTP (left) and IRV (right) on the grid of $G = 320$ points per axis (Beta, `rms`, $D = 0.2$), on a log scale: black is below 10^{-4}. Yellow: the borders between winners, traced as in Section 8.5.2. The crosses mark m_1 (at D) and m_2.

8.5.2. Tracing the regions

For each winner c let

$$\psi_c = \begin{cases} \mu & \text{where } c \text{ wins} \\ -\mu & \text{elsewhere} \end{cases}. \tag{68}$$

ψ_c is continuous, and the region of c is $\{\psi_c > 0\}$. It is traced by marching squares (`contourpy`) on a grid of $G \times G$ points: the centres $\left(k + \frac{1}{2}\right)/G$ plus both walls, where the medians are clamped to the outermost pixel centres, as the nodes are. On a grid edge from a point where c wins with margin α to a point where c' wins with margin β, both ψ_c and $\psi_{c'}$ cross zero at $\alpha/(\alpha + \beta)$ of the way. So neighbouring regions share their border points and leave no gap along a border. For example, for IRV at $G = 160$ the neighbouring grid points $(0.296875, 0.384375)$ and $(0.303125, 0.384375)$ are won by B with $\alpha = 0.00255$ and by C with $\beta = 0.00350$. Both the ring of B and the ring of C get the vertex 0.4215 of the way along, at $x = 0.299509$ (Figure 20, near the bottom tip of C's island). A gap is left only in a grid cell where three winners meet: there each of the three regions ends at a straight segment between two edge crossings, and the small triangle between the three segments belongs to none of them.

The shares come from the interpolant of Section 2.4 (those of the score methods are computed at the grid points themselves, Section 3.8), so the grid only has to be fine enough not to miss slivers; its borders do not have the steps of a pixel image. The UI uses $G = 160$ while dragging and $G = 320$ once the candidate is dropped. The margins at grid points next to a change of winner are $O(1/G)$, which the tests check.

8.5.3. Polygons

Every region is a list of polygons, because regions can be in pieces (Section 7.5) and can have holes (an island of another winner). Each polygon is an outer ring and its holes. Can the order of a ring's points tell the region from its complement? In principle yes: with the convention that the region lies to the left of each edge, a counter-clockwise triangle is the triangle and a clockwise one is everything else. GeoJSON and the nonzero fill rule use this convention for holes. But a canvas never fills a lone clockwise ring as its outside; it fills the inside with winding number -1. So the complement is sent explicitly, as an outer ring (counter-clockwise) with the triangle as a hole (clockwise). The UI fills with the even-odd rule, which does not depend on the orientation at all. This matters because flipping y for the canvas reverses every orientation. Voronoi diagrams need no grid: their cells are the polygons of Section 8.1.

Each ring is sent as a flat list $[x_0, y_0, x_1, y_1, \ldots]$ with six decimals. It is closed: the last point repeats the first, so a ring with k vertices has $2(k + 1)$ numbers. Table 10 lists the IRV regions of Figure 19 as the UI receives them while dragging ($G = 160$), with the signed (shoelace) area of every ring, which is positive for a counter-clockwise ring. Figure 20 draws them.

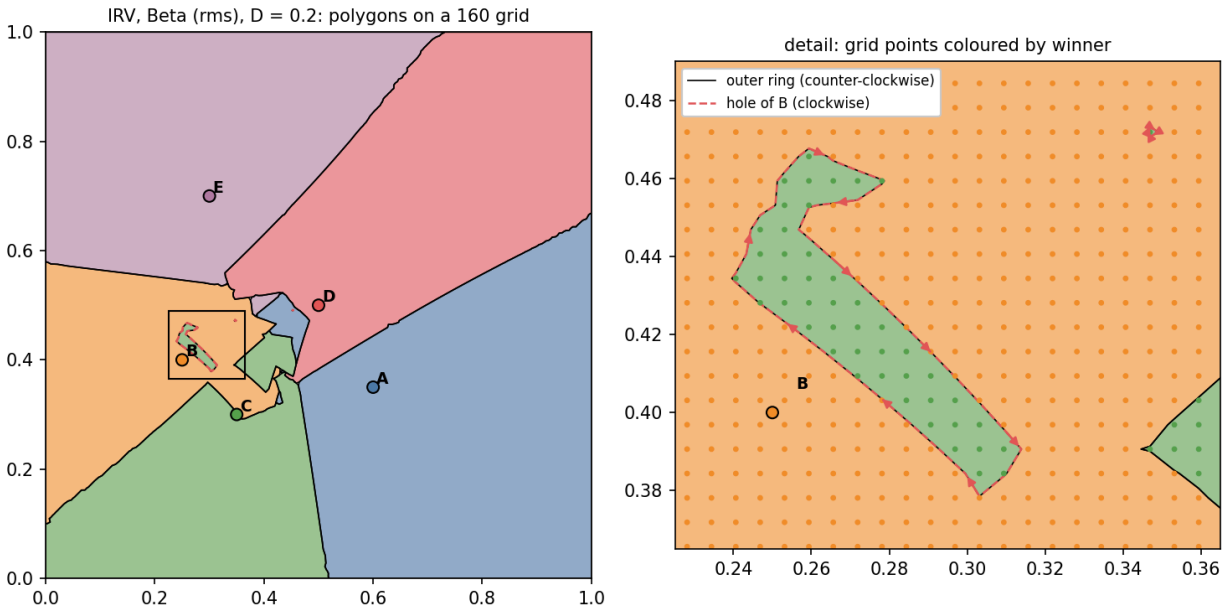


Figure 20: The polygons of Table 10, filled with the even-odd rule as in the UI. Solid black: outer rings; dashed red: holes. Right: the frame on the left, with the grid points coloured by their winner. The arrows run clockwise along B's two holes: the island of C, and at the top right a sliver of C that the grid catches at only one point.

winner	polygon	outer ring	holes
A	1	389 (+0.267367)	
	2	10 (+0.000155)	
	3	4 (+0.000020)	
	4	76 (+0.005068)	4 (−0.000001)
B	1	356 (+0.117606)	60 (−0.001887), 4 (−0.000004)
C	1	391 (+0.138644)	
	2	60 (+0.001887)	
	3	4 (+0.000004)	
D	1	443 (+0.233848)	
	2	4 (+0.000001)	
E	1	32 (+0.001143)	
	2	385 (+0.236058)	

Table 10: The IRV regions of the UI's default diagram while dragging (candidates A–E, Beta voters, `rms`, $D = 0.2$, $G = 160$): the number of vertices of every ring and, in brackets, its signed area.

B's region is one polygon with two holes. The larger hole is the island of C next to B. C's region lists the same 60 vertices again as a polygon of its own, counter-clockwise, so its area is $+0.001887$ where B's hole has -0.001887. The area of B's region is the sum over its rings, $0.117606 - 0.001887 - 0.000004 = 0.115715$. The rings with 4 vertices are diamonds around a single grid point, the most the grid shows of a sliver thinner than its step. The smaller hole of B is such a sliver of C, near $(0.347, 0.472)$. A and E are in several pieces, as IRV regions often are (Section 7.5).

The signed areas of all rings add up to 0.99991 instead of 1. The missing 0.00009 lies in the grid cells where three winners meet (Section 8.5.2). It shrinks with the grid, to 0.99998 at $G = 320$.

8.6. Timings

Round trip of one update over HTTP while dragging one candidate ($G = 160$; median of 15 steps, $D = 0.2$, `rms`), which adds a few milliseconds to the compute times of Section 8.4. Timings on the test machine, a laptop with performance and efficiency cores, vary by up to a factor of three between runs.

	candidates	FPTP	Borda	Condorcet	Schulze	IRV
Beta	5	13 ms	14 ms	16 ms	16 ms	17 ms
	8	10 ms	21 ms	29 ms	26 ms	75 ms

IRV is still the slowest: it needs every cell of the arrangement, a drag changes about two thirds of their edges, and the shares of about 300 rankings are interpolated to the grid. All edges of the arrangement lie on the $C(C-1)/2$ bisectors, though. If the integral of ω along each bisector were tabulated once, as a function of the position on it, every edge would be a difference of two values, and the cost would grow with the number of lines (28 for 8 candidates) instead of edges (468). This is not implemented.

9. Pixels at the geometric median

A Beta pixel is the median of its voters along x and along y. That is a median only in the directions of the axes, which is the root of the distorted Condorcet regions of Chapter 5. The web UI draws every election at the **geometric median** of its voters by default (*Point in diagram is: the geometric median*), a centre that does not depend on the axes. This chapter defines it, describes the part of the square that the diagram then covers, shows what it does to the borders between the candidates, and how it is computed (`margin/geometric.py`). The numbers and the figure are produced by `docs/figures.py`.

9.1. The map g

The geometric median of the voters $V = (X, Y)$ of a pixel is the point with the smallest mean distance to them. For the voters with the medians $m = (m_x, m_y)$ along the axes,

$$g(m) = \arg\min_p d_m(p), \quad d_m(p) = E_m\ |V - p|\ . \tag{69}$$

(In this chapter g is this map, as in the code, and not the density of Y.) The voters do not lie on one line, so d_m is strictly convex and the minimum is unique. In one dimension the point with the smallest mean distance is the median. In the plane it is a different point from the pair of medians along the axes, and unlike that pair it turns with the voters when the plane is rotated.

A diagram of geometric medians shows at the point p the election of the voters with $g(m) = p$. The voters, their shares, the methods and the margins are those of the pixel m; only the point at which the election is drawn moves from m to $g(m)$. g depends on the voter model alone (D and the spread rule), never on the candidates.

- **Normal voters** are symmetric about their pixel, so $g(m) = m$ and the diagram is the usual one.
- **Symmetry.** For Beta voters g leaves the centre of the square in place and commutes with mirroring an axis and with swapping the axes, as the voters do. A median of $\frac{1}{2}$ along one axis stays $\frac{1}{2}$.
- **Towards the centre.** Everywhere else g moves the pixel towards the centre of the square, the more the closer the pixel is to a wall. In the table below it lies between the median and the mean of the voters.
- **Both axes at once.** The first coordinate of g depends on m_y too: the same voters along x are moved less when Y is crowded against a wall as well. So g cannot be found one axis at a time, as the median and the mean can.
- **One to one.** Two different electorates of the model are never drawn at the same point. This is checked numerically: the pixel grid moved by g keeps the order of its neighbours along both axes and never folds over, for $D = 0.05$, 0.25 and 0.4 and both spread rules (`tests/test_geometric.py`).

First coordinate of g for the medians $\left(m, \frac{1}{2}\right)$ and (m, m), and the mean of X (`rms`, $D = 0.3$; the last row is the outermost pixel):

median m	g at $\left(m, \frac{1}{2}\right)$	g at (m, m)	mean
0.50	0.500	0.500	0.500
0.70	0.662	0.665	0.621
0.90	0.823	0.844	0.750
0.98	0.896	0.938	0.812
$599/600$	0.930	0.983	0.838

9.2. The coloured region

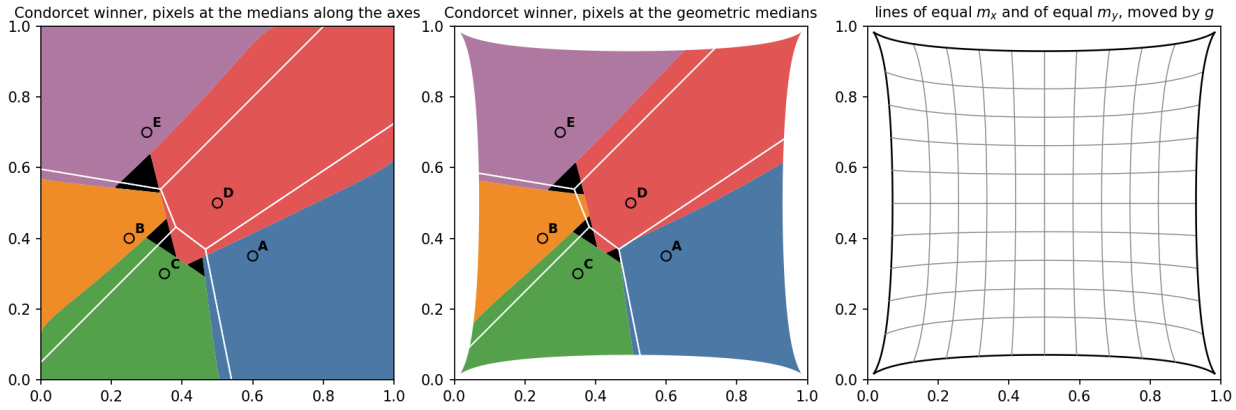


Figure 21: Beta voters (`rms`, $D = 0.3$). Left and middle: the Condorcet winner with pixels at the medians along the axes and at the geometric medians. White lines: the Voronoi borders (bisectors); black: no Condorcet winner. Right: the lines of equal m_x and of equal m_y for the values $1/600$, 0.1, ..., 0.9, $599/600$, moved by g; the thick outline belongs to the outermost medians.

With pixels at geometric medians the diagram no longer fills the square (Figure 21). The coloured region is exactly the set of points of the unit square that are the geometric median of some electorate of the model. Each of its points belongs to exactly one electorate, because g is one to one.

It is not a part cut out of the usual diagram. Every election of the usual diagram is in it and none is missing; each is only moved from its medians along the axes to its geometric median. The usual diagram has shrunk into the region as a whole, and nothing was cropped.

"Of the model" matters. The region is not a property of the geometric median as such, but of this family of voters:

- **The shape of the voters:** independent Beta distributions along the axes, with the chosen D and spread rule. So the outline changes with the *Deviation* slider, and with the rule.
- **The range of the medians:** from $1/600$ to $1 - 1/600$, the outermost pixel centres of the UI's 300 pixels per axis. Medians closer to a wall would fill the corners (under `rms`), but narrow the strip in the middle of a wall only slowly.
- **Other voters** can have their geometric median anywhere in the square. Normal voters have it at their pixel and cover the whole square.

The table gives the width of the empty strip, that is the distance of g from the wall for the outermost medians, in the middle of a wall and at a corner, and the part of the square that stays empty. The last two columns are the widths if the medians reached to 10^{-6} from the walls.

		medians from $1/600$			from 10^{-6}	
rule	D	middle	corner	empty	middle	corner
`rms`	0.1	0.018	0.007	6.8 %	0.007	0.0001
	0.2	0.038	0.011	13.4 %	0.019	0.0002
	0.3	0.070	0.017	22.5 %	0.044	0.0004
	0.4	0.125	0.032	35.9 %	0.094	0.0012
`mean_abs`	0.2	0.064	0.025	22.7 %	0.044	0.0011
	0.3	0.141	0.087	45.9 %	0.120	0.0270

Why a strip stays empty. At the minimum of d_m the unit vectors from p towards the voters cancel,

$$\nabla d_m(p) = E_m\left[\frac{p - V}{|V - p|}\right] = 0. \tag{70}$$

A median counts voters, while the geometric median balances directions. A pixel next to the wall $x = 0$ has half of its voters between the wall and m_x, which is what holds the median there. But these voters are spread along the wall in y. Seen from a point near the wall they lie up and down the wall, so their unit vectors nearly cancel and have almost no component towards the wall. The other half pulls away from the wall with its full weight. The balance is therefore reached inside the square, and further inside the further out that half lies. This is the push of Equation 9, which is why `mean_abs` leaves a wider strip than `rms`. In a corner both coordinates are crowded against their walls, so many voters sit next to the pixel in every direction and hold g there: the strip is narrowest in the corners.

No spread rule can fill the square as long as the voters keep a spread at the wall. At a point p of the wall $x = 0$ the derivative of d_m across the wall is $-E_m[X/|V - p|]$, which is negative unless all the voters lie on the wall. So for voters inside the square the geometric median reaches a wall only if their spread across it vanishes there. Normal voters fill the square because they may leave it.

Nor does a thinner far tail close the strip. The pull counts voters, the spread counts squared distances, so putting the spread into a few voters on the opposite wall narrows the strip. This was tried as **ZOIB voters** (a zero-or-one inflated Beta, 5.5 % of a pixel's voters on the opposite wall at $D = 0.2$): the strip in the middle of a wall narrowed from 0.038 to 0.016. But the voters below g still lie along the wall, and their pull towards it vanishes as g approaches it, while the far ones keep pulling with their full weight, so the strip stays. ZOIB voters are not used: they do not make the diagram easier to interpret. They are there only to fill the picture, nothing about real voters calls for them, and they bend the borders next to the walls.

9.3. The borders between the candidates

By Chapter 5 a majority prefers c_i to c_j where the median of the projected voters $V \cdot \nu$ lies on c_i's side. That median, the effective centre of the pixel in the direction ν, is shifted from $m \cdot \nu$ towards the mean of the voters, and $g(m)$ is shifted the same way (Section 9.1). Drawing the election at $g(m)$ therefore takes back part of the shift in every direction at once, and the borders move towards the bisectors (Figure 21).

The table measures how far the pairwise majority borders $\pi_{ij} = \frac{1}{2}$ of all ten pairs of the candidates A–E lie from their bisectors, at the points where a border crosses a line of the 300×300 pixel grid: the mean distance and, in brackets, the largest.

rule	D	pixels at the medians along the axes	pixels at the geometric medians
`rms`	0.2	0.0165 (0.054)	0.0106 (0.029)
	0.3	0.0411 (0.106)	0.0255 (0.060)
`mean_abs`	0.2	0.0208 (0.089)	0.0130 (0.054)
	0.3	0.0549 (0.223)	0.0317 (0.132)

About 60 % of the mean distance and a little more than half of the largest remain. For `rms` that is 3.2 pixels instead of 5.0 on average at $D = 0.2$, and 7.7 instead of 12.3 at $D = 0.3$. For the border between D and E at $D = 0.25$ the tests check the same: 0.045 to 0.075 from the bisector with pixels at the medians along the axes, 0.025 to 0.039 with pixels at the geometric medians.

What the geometric median does not do:

- **It does not give the Voronoi diagram.** $g(m)$ is not a median in every direction: a slanted line through it need not split the voters in half, and a product of skewed Beta distributions has no point through which every line does (Chapter 5). The rest of the bend is the skew of the voters, and it stays.
- **The cycles stay.** The elections are the same, so every election without a Condorcet winner is still there, drawn at another point (black in Figure 21).
- **Areas are not comparable.** g compresses the diagram most near the walls, so the share of the coloured region that a candidate wins is not its share of the pixels.

- **The spread rule still refers to the medians along the axes.** `rms` fixes $\sqrt{E(X-m_x)^2}$, not the distance of the voters from $g(m)$. Fixing the spread around $g(m)$ instead ($\sqrt{E(X-g_x)^2}$ or $E|V-g|$) makes the coloured region smaller. And since g depends on both axes, the parameters of X then depend on m_y too, so the shares are no longer edge integrals of one Beta per column and one per row (Section 2.3), and take a much slower computation: two to four times as long per diagram while a candidate is dragged, and about 6 s instead of 0.2 s to prepare the voters of each deviation.

A spread around the geometric median gains little that shows in the diagrams and costs time, so the UI uses the geometric median only as the point where an election is drawn; the voters keep the `rms` rule around the medians along the axes.

9.4. Computing g (`margin/geometric.py`)

The mean distance. $d_m(p)$ is a sum over the rectangles of the voter grid of Section 3.8. The rectangle $[x_k, x_{k+1}] \times [x_l, x_{l+1}]$ holds the exact share $\mu_k \nu_l$ of the voters, so the infinite density of a Beta with $a < 1$ at a wall does no harm. Its voters count at their exact mean $(\overline{x}_k, \overline{y}_l)$ (the first moment of $\mathrm{Beta}(a,b)$ between two lines is $a/(a+b)$ times the share of $\mathrm{Beta}(a+1,b)$ between them), at the distance

$$\sqrt{(\overline{x}_k - p_x)^2 + (\overline{y}_l - p_y)^2 + s_{kl}^2}, \quad s_{kl}^2 = \frac{(x_{k+1}-x_k)^2 + (x_{l+1}-x_l)^2}{24}, \tag{71}$$

where s_{kl}^2 is the variance of a uniform distribution over the rectangle along one axis, averaged over the two axes. For a square cell this is the mean distance of its voters from p to second order in the size of the cell. It also keeps d_m smooth and convex: with plain distances the voters next to a corner, half of them in a few cells, would trap the minimum at the mean of a cell.

Minimisation. Newton's method, started at m, takes about five steps. A step that would leave the square is replaced by Weiszfeld's step, the mean of the voters weighted by one over their distance, which never leaves it.

Accuracy. Against a voter grid with cells a quarter as wide, g changes by less than $5 \cdot 10^{-5}$ for the narrowest voters of the UI ($D = 0.05$), by $2 \cdot 10^{-5}$ at $D = 0.1$ and by $3 \cdot 10^{-6}$ from $D = 0.25$ on. The geometric median of a million sampled voters, itself accurate to about $3 \cdot 10^{-4}$, agrees with it within the $1.5 \cdot 10^{-3}$ of the tests.

Nodes. g is smooth in m, so it is computed at the $N \times N$ nodes of Section 2.4 and interpolated in $\operatorname{logit} m$, as the shares are. A quarter of the nodes is enough; the others are their mirror images. The interpolant is within $4 \cdot 10^{-5}$ of g computed at the point itself. The table of a model is built once, in about 0.2 s, and kept.

Drawing (`margin/regions.py`). The regions are traced as in Section 8.5.2, on the grid moved by g: the grid point with the medians m is drawn at $g(m)$, and a crossing is placed along the moved edge by the same linear interpolation of ψ_c. The grid points at and next to a wall share the outermost median and would be drawn at the same point, so only one of them is kept. The moved grid ends short of the walls, and so do the regions. The Voronoi diagram is traced on the moved grid as well, with the nearest candidate as the winner and the difference of the two smallest squared distances as the margin. That difference is linear along a straight edge, so its borders still lie exactly on the bisectors.

Moving between the two diagrams. When the choice of what a pixel is changes, the UI moves the diagram instead of jumping. For 0.7 s it asks for the regions with every grid point drawn at $q + t(g(m) - q)$, where q is the point at which the usual diagram draws it (m, or the wall for the outermost medians) and t eases from 0 to 1 or back (`shift` of `POST /api/regions`). So every election travels on a straight line between its two points, and the grid does not fold over on the way (tests). The steps are traced on the coarser grid, like a drag, at about 15 to 45 ms each for five candidates, and the last one on the fine grid. Where the system asks for less motion the diagram jumps.

Hovering (`pixels_at`, `GET /api/geometric`). To show the voters of a hovered point the UI needs the inverse of g: for the centre of every pixel, the pixel whose voters have their geometric median nearest to it (a k-d tree over g of all pixels), or none if the centre lies in the strip, beyond the curve that g follows along the outermost pixels of a side. The UI gets one quarter of the square per deviation and mirrors it.

Candidates stay in the coloured region (`outline`). With the lookup comes the border of the coloured region, the curve g follows along the outermost pixels, counter-clockwise. A candidate in the strip would stand where no electorate has its geometric median, so with pixels at geometric medians the UI keeps every candidate within the border: one dragged or added beyond it, or left beyond it when the diagram moves to geometric medians or a larger deviation widens the strip, goes to the closest point of the border. It stays there when the pixels go back to their medians along the axes.

10. Comparing a diagram with Voronoi

The web UI reports three errors for a method's diagram, using the Voronoi diagram of the same candidate positions as the reference. All are shown as percentages, are zero for the Voronoi diagram itself, and smaller values indicate closer agreement. The assignment error and the assignment error inside the candidates' convex hull are $100 - S$ for the scores S below; the hull-restricted one is the same metric evaluated only within the convex hull, so the two values are related rather than independent. The boundary distance is a mean in units of the unit square's width.

The UI computes the errors only for the diagram on the fine grid ($G = 320$, Section 8.5.2), once a candidate is dropped or a slider let go. While a candidate is dragged it shows none: tracing the Voronoi diagram and comparing the two take longer than most methods themselves.

The errors have a true zero, so two methods are compared by their ratio: an error of 2 % against 6 % is three times smaller. For small regrets the assignment loss below is nearly linear ($\ell \approx r/L$), so this holds approximately for the assignment errors too. Errors near zero are dominated by the grid resolution, and there a ratio says little.

10.1. Assignment regret (`margin/metrics.py`)

At each evaluation point x, let $d_{\min(x)}$ be its distance to the nearest candidate, and $d_{a(x)}$ its distance to the candidate assigned the point by the method diagram. The regret is the extra distance paid by that assignment:

$$r(x) = d_{a(x)} - d_{\min(x)}. \tag{72}$$

This is more informative than counting mismatched labels. At a Voronoi boundary the two nearest candidates have nearly equal distances, so switching their labels has very little regret. Assigning a point to a much farther candidate has larger regret. If the method reports a cycle rather than a candidate, the point receives the maximum loss.

To put regrets of different candidate layouts on a comparable scale, let L be the median of all pairwise candidate distances. The bounded loss and the importance weight at x are

$$\ell(x) = \frac{r(x)}{r(x) + L}, \quad w_{a(x)} = \frac{1}{1 + \left(\frac{d_{\min(x)}}{L}\right)^2}. \tag{73}$$

Thus ℓ is zero for a perfect assignment and approaches one for a very poor one. The weight decreases with distance from the nearest candidate, so mistakes in remote parts of the square matter less. Let $A(x)$ be the area represented by the grid point (the trapezoidal area weight of the evaluation grid). The whole-square score is

$$S_a = 100\left(1 - \frac{\sum_{x \in \Omega} A(x) w_{a(x)} \ell(x)}{\sum_{x \in \Omega} A(x) w_{a(x)}}\right). \tag{74}$$

Here Ω is the full evaluated region. To obtain the inside-hull score, use exactly the same formula and weights, but sum only over grid points inside the convex hull H of the candidate positions:

$$S_H = 100\left(1 - \frac{\sum_{x \in H} A(x) w_{a(x)} \ell(x)}{\sum_{x \in H} A(x) w_{a(x)}}\right). \tag{75}$$

If the candidates are collinear, their convex hull has no area and the inside-hull score is not defined; the UI displays `n/a`. The UI displays the errors $100 - S_a$ and $100 - S_H$.

10.2. Symmetric mean distance between boundaries

Let B_a be the shared borders between the method's win regions and B_v the shared Voronoi borders. The outer edge of the square is not included: it is not a boundary between two winners. Distances are measured in both directions, so extra borders in one diagram and missing borders in the other are both represented.

For a sampled point x on one boundary and a line segment of the other boundary with endpoints a and b, the nearest point on that segment is the perpendicular projection clamped to the segment:

$$t = \max\left(0, \min\left(1, \frac{(x-a)\cdot(b-a)}{\|}b-a\|^2\right)\right), \quad p = a + t(b-a), \quad d(x,[a,b]) = \|x-p\|. \tag{76}$$

The distance $d(x, B)$ to a whole boundary is the minimum over all its segments. This is a true point-to-segment distance, not the distance to the nearest sampled vertex: when the perpendicular foot falls within a segment, it gives the perpendicular distance; outside the segment it gives the distance to the nearer endpoint.

The boundaries are sampled uniformly by arc length, at approximately 0.005 units per sample. Each sample represents an equal small piece of boundary length. For the whole-square metric, its weight is further reduced toward the square's edge by

$$w_{b(x)} = \exp(-4\ \|x - (0.5, 0.5)\|^2). \tag{77}$$

This weight is largest at the centre and smaller toward the corners. For each direction, take the weighted mean distance and then give the two directions equal weight. The reported boundary distance is

$$D_b = \frac{1}{2}\left(\frac{\sum_{x\in B_a} \Delta s w_{b(x)} d(x, B_v)}{\sum_{x\in B_a} \Delta s w_{b(x)}} + \frac{\sum_{x\in B_v} \Delta s w_{b(x)} d(x, B_a)}{\sum_{x\in B_v} \Delta s w_{b(x)}}\right). \tag{78}$$

The UI displays $100 D_b$ as a percentage of the square's width.

The boundary overlay in the UI draws the method's boundary in black and the Voronoi reference boundary in blue. The candidate hull can be shown separately as a dashed outline. There is no tolerance band and no binary inside/outside decision in the boundary metric.